Part A: Object-Oriented Design

Object
Oriented
Analysis &
Design



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ **Object-Oriented Analysis and Design (OOAD)**
- ✓ **History**
- ✓ **Scenario-1: DUMS**
- ✓ **Scenario-2: Agricultural Automation System (AAS)**
- ✓ **Conceptual Design**
- ✓ **Technical Design**
- ✓ **OOAD Thinking**
- ✓ **CRC:Class, Responsibility and Collaborator**

Object-Oriented Analysis and Design (OOAD)

- A method for analyzing and designing a system by thinking in terms of objects (real-world entities) rather than just functions or logic.
- It helps us move from requirements (what the system should do) to a design (how the system should be built).
- Object-Oriented Analysis and Design is about planning before coding.
- It has two parts:
  - Conceptual Design (what to build)
  - Technical Design (how to build it)
- Using object-oriented principles and tools like CRC cards ensures your software will be structured, maintainable, and reusable.





Why?

- ✓ Part of the foundation for becoming an experienced software architect.
- ✓ Upon completion of this course, we will be able to:
  - ➢ Explain object-oriented analysis and design.
  - ➢ Engage in object-oriented modelling.
  - ➢ Explain design principles for object-oriented programming





Why People First Used Object-Oriented Programming

- **Software was getting too complex**
- In the 1960s, programs were written mostly in **procedural languages** (like Fortran, COBOL, C).
- These worked fine for small programs, but as projects grew into *hundreds of thousands of lines*, it became very hard to maintain.
- Problems:
  - Data and functions were separated (functions acted on global data → messy).
  - Hard to reuse code (you had to copy-paste functions).
  - Difficult to model *real-world entities*.





OOP solved these problems

- ✓ OOP grouped **data + behavior** together inside "objects".
- ✓ Objects could represent **real-world things** (e.g., a *Car* object has data like color, speed and functions like accelerate(), brake()).
- ✓ Reusability: You could reuse and extend classes without rewriting everything.
- ✓ Abstraction: You could hide details and show only what mattered.
- ✓ This made large systems easier to design, understand, and maintain.





Who Invented OOP? (History)

- ✓ **1960s – Simula 67 (Norway):** First OOP language, simulations (like simulating ships, traffic, factories). Classes, objects, and inheritance → the foundations of OOP.
- ✓ **1970s – Smalltalk (USA):** popularized the term *Object-Oriented Programming*.
- ✓ fully OOP (everything is an object)., Influenced GUI development and later languages.
- ✓ **1980s – C++ (Denmark): Bjarne Stroustrup** extended C with classes → **C++**., OOP into mainstream software engineering.
- ✓ **1990s – Java (USA):** Sun Microsystems introduced **Java** (by James Gosling). Platform-independent, heavily OOP, designed for the internet era. Became very popular for enterprise software.

- ✓ Reason OOP was used first: To manage software complexity by modeling real-world concepts as "objects".
- ✓ Inventors:Simula 67 → Dahl & Nygaard (first OOP language).
- ✓ Smalltalk → Alan Kay (spread OOP philosophy).
- ✓ C++ & Java → made OOP widely practical.





System Scenario-1: "Dhaka Urban Management System (DUMS)"

- Dhaka has 3 major problems:
  - House allocation (overpopulation, unequal distribution)
  - Garbage management (waste disposal everywhere) https://www.youtube.com/watch?v=TcWPIXDQhz8
  - Traffic control (congestion, poor flow) https://www.youtube.com/watch?v=b_mCu2_RQg4
- We will design a software system to monitor and manage these problems.

System Overview

"Dhaka Urban Management System (DUMS)" is a centralized software platform designed to help city authorities manage three major urban problems in Dhaka: **housing allocation, garbage management, and traffic control**. Citizens, government officials, garbage workers, and traffic authorities will use the system through a web or mobile application.

The system will collect real-time data, store it in a central database, and use AI/ML-based decision support to improve housing distribution, waste collection, and traffic flow.





System Scenario-1: "Dhaka Urban Management System (DUMS)"

- ✓ Goal: A centralized platform to manage housing allocation, garbage collection, and traffic control in Dhaka.
- ✓ Housing Allocation Module
  - Citizens apply for housing permits.
  - Government reviews applications based on availability.
  - Automated allocation based on income, family size, and location /area.
- ✓ **Garbage Management Module**
  - Track garbage collection zones.
  - Assign cleanup schedules.
  - Citizen complaints for uncollected waste.
- ✓ **Traffic Control Module**
  - Real-time traffic monitoring and Dynamic signal adjustments.
  - Accident/blockage reporting.

Main Actors

- ✓ Step 1: Understanding the problem domain
- ✓ We identify the main objects (entities) from the problem

Actor	Role
Citizen	Applies for housing, reports garbage issues, reports traffic accidents/blockages
Housing Authority	Reviews housing applications and approves allocation
City Authority/Admin	Monitors the whole system and makes policy decisions
Garbage Management Officer	Creates collection schedules and assigns trucks
Garbage Truck Driver	Collects waste from assigned zones
Traffic Control Officer	Monitors congestion and handles accidents/blockages
AI/ML System	Suggests housing allocation, garbage routes, and traffic signal timing
External Systems	Dhaka City Corporation, BRTA, traffic sensors, GPS systems

Use Case 1: Housing Allocation

Use Case Name: Apply and Allocate Housing Permit

Goal : To allocate available housing fairly based on income, family size, and preferred location.

Primary Actor : Citizen

Supporting Actors: Housing Authority, AI/ML Allocation System, City Authority System

Preconditions: The citizen must be registered in the DUMS platform. Housing data must be available in the system database.

Main Scenario

- The citizen logs into the DUMS mobile or web app.
- The citizen selects “Apply for Housing Permit.”
- The citizen submits personal information, income level, family size, current address, and preferred area.
- The system verifies the application and stores it in the database.
- The AI/ML component analyzes housing availability, applicant income, family size, and location demand.
- The system generates a ranked allocation list.
- The Housing Authority reviews the recommendation.
- If the application is valid, the authority approves the allocation.
- The citizen receives a notification with housing permit details.
- The database is updated with the new allocation record.

Alternative Scenario: If no house is available in the preferred area, the system suggests nearby alternative areas.

Output: A fair and automated housing allocation decision.



Use Case 2: Garbage Management

- **Use Case Name: Report and Manage Uncollected Garbage**
- **Goal:** To ensure timely garbage collection and reduce waste disposal problems in Dhaka.
- **Primary Actor:** Citizen
- **Supporting Actors:** Garbage Management Officer, Garbage Truck Driver, Dhaka City Corporation API
- **Preconditions:** Garbage zones, bins, trucks, and collection schedules must be registered in the system.
- **Main Scenario**
  - ✓ A citizen notices uncollected garbage in their area.
  - ✓ The citizen opens the DUMS app and selects “Report Garbage Issue.”
  - ✓ The citizen uploads a photo, location, and short description.
  - ✓ The system identifies the garbage zone using GPS/location data.
  - ✓ The complaint is stored in the database and sent to the Garbage Management Officer.
  - ✓ The system checks the nearest available garbage truck.
  - ✓ The AI/ML routing system suggests the most efficient truck route.
  - ✓ The Garbage Management Officer assigns the task to a truck driver.
  - ✓ The truck driver collects the waste and updates the task status.
  - ✓ The citizen receives a notification that the complaint has been resolved.
- **Alternative Scenario:** If no truck is available, the complaint is placed in the next available collection schedule.
- **Output:** Cleaner city zones and faster response to waste complaints.

Use Case 3: Traffic Control

- **Use Case Name: Monitor Traffic and Adjust Signals Dynamically**
- **Goal:** To reduce traffic congestion and improve vehicle flow in Dhaka.
- **Primary Actor:** Traffic Control Officer
- **Supporting Actors:** Vehicle, Traffic Signal, Citizen, BRTA API, AI Traffic System
- **Preconditions:** Traffic signals, road sensors, cameras, and vehicle flow data must be connected to the system.
- **Main Scenario**
  - ✓ The traffic monitoring system collects real-time data from road sensors, cameras, and GPS-enabled vehicles.
  - ✓ The system detects heavy congestion at a major intersection.
  - ✓ The AI/ML traffic module analyzes vehicle density and road flow.
  - ✓ The system automatically adjusts traffic signal timing.
  - ✓ Green-light duration is increased for the congested road.
  - ✓ The Traffic Control Officer monitors the result through the central dashboard.
  - ✓ If congestion decreases, the signal timing returns to normal.
  - ✓ The system stores traffic flow data for future analysis.

Alternative Scenario: If a citizen reports an accident or road blockage, the system alerts the Traffic Control Officer and suggests alternative routes.

Output: Improved traffic flow, reduced congestion, and faster accident response.

Integrated System Scenario

- **Scenario Title: A Day in Dhaka Managed by DUMS**

- ✓ In the morning, a citizen named Rahim logs into the DUMS app and applies for a housing permit because his family is living in an overcrowded area. The system collects his income, family size, and preferred location. The AI-based housing module analyzes available houses and suggests a suitable allocation. The Housing Authority reviews and approves the application.
- ✓ Later, Rahim notices a large amount of garbage near his neighborhood. He reports it through the same app by uploading a photo and location. The garbage module identifies the zone, assigns the nearest garbage truck, and updates the complaint status after collection.
- ✓ During the evening rush hour, traffic congestion increases near a major road intersection. The traffic control module detects the congestion using real-time monitoring data. The system dynamically changes the traffic signal timing to improve vehicle flow. At the same time, if an accident is reported, the system alerts traffic officers and suggests alternative routes.
- ✓ Through one centralized platform, DUMS helps the city authority make faster, fairer, and data-driven decisions for housing, garbage, and traffic management.





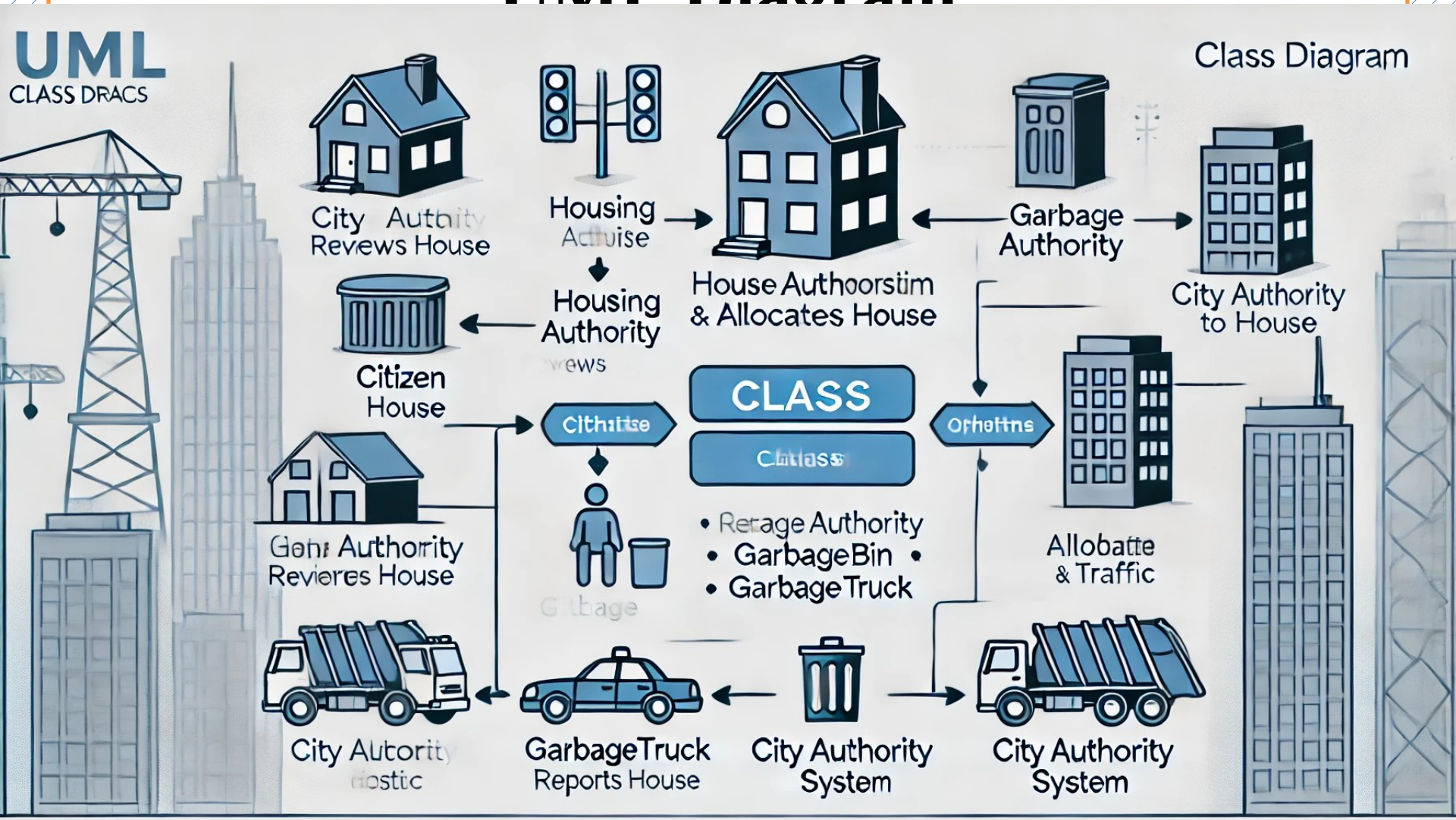
Summary of Use Cases

Module	Main Use Case	Main Benefit
Housing Allocation	Apply and allocate housing permits	Fair distribution of housing
Garbage Management	Report and collect uncollected waste	Cleaner city environment
Traffic Control	Monitor traffic and adjust signals	Reduced congestion
Central Dashboard	Monitor all city services	Better urban decision-making

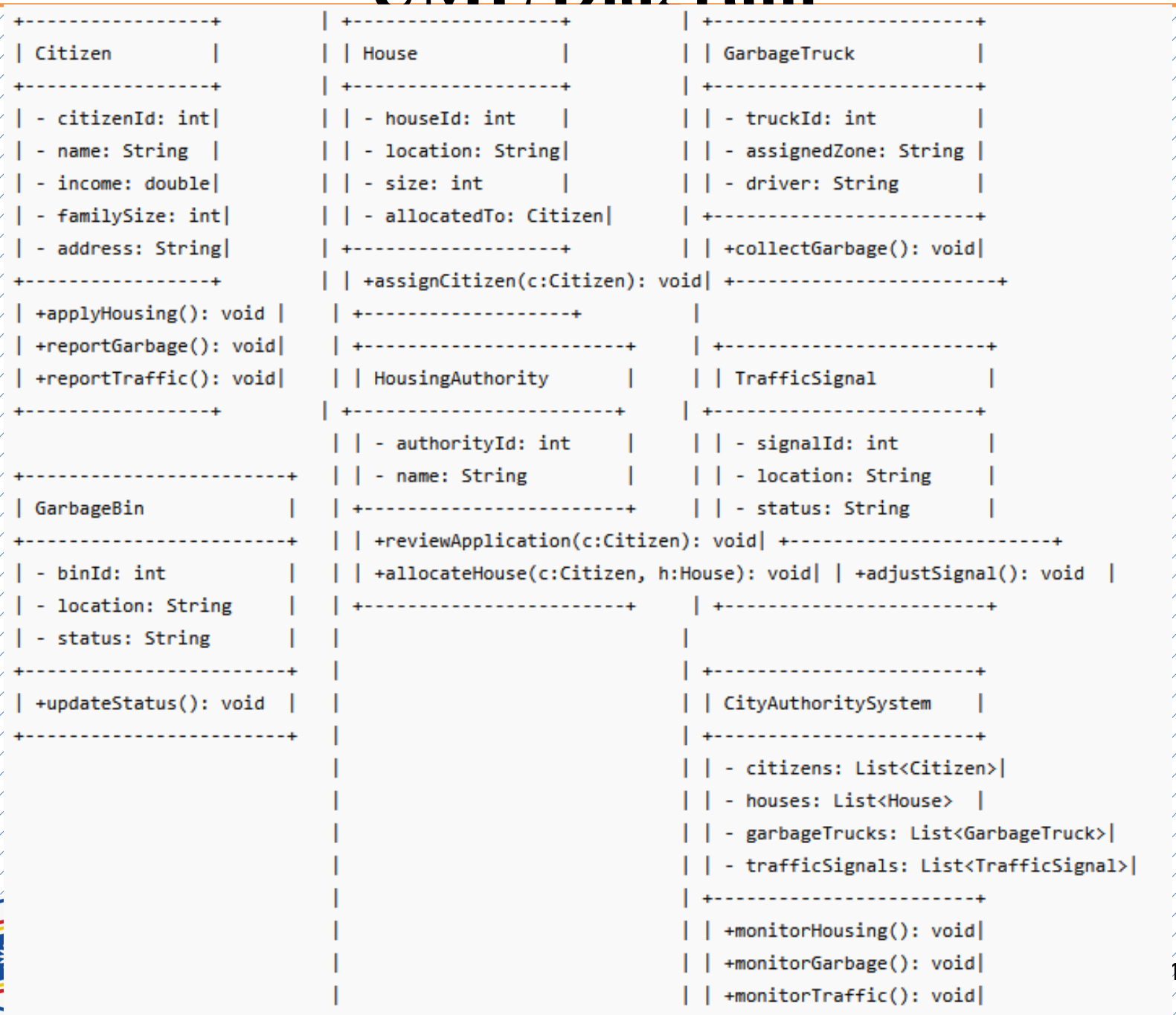
Final Use Case Statement

The “Dhaka Urban Management System” allows citizens and government authorities to work through one centralized digital platform. Citizens can apply for housing, report garbage problems, and submit traffic complaints, while city authorities can monitor real-time data, assign resources, and make AI-supported decisions. This system helps Dhaka manage overpopulation, waste disposal, and traffic congestion in a smarter and more organized way.

UML Diagram


UML
CLASS DRACS
Class Diagram
City Authority
Reviews House
Housing
Housing
Authority
House Authoorstim
& Allocates House
Garbage
Authority
City Authority
to House
Citizen
House
CLASS
Garbage Authority
Revieres House
• Garbage Authority
• GarbageBin •
• GarbageTruck
Allobatte
& Traffic
City Authority
GarbageTruck
Reports House
City Authority
System
City Authority
System

UML Diagram


Citizen
- citizenId: int
- name: String
- income: double
- familySize: int
- address: String
+applyHousing(): void
+reportGarbage(): void
+reportTraffic(): void
GarbageBin
- binId: int
- location: String
- status: String
+updateStatus(): void
House
- houseId: int
- location: String
- size: int
- allocatedTo: Citizen
+assignCitizen(c:Citizen): void
HousingAuthority
- authorityId: int
- name: String
+reviewApplication(c:Citizen): void
+allocateHouse(c:Citizen, h:House): void
GarbageTruck
- truckId: int
- assignedZone: String
- driver: String
+collectGarbage(): void
TrafficSignal
- signalId: int
- location: String
- status: String
+adjustSignal(): void
CityAuthoritySystem
- citizens: List<Citizen>
- houses: List<House>
- garbageTrucks: List<GarbageTruck>
- trafficSignals: List<TrafficSignal>
+monitorHousing(): void
+monitorGarbage(): void
+monitorTraffic(): void

Sequential Model (Example for Housing Allocation)

Scenario: Citizen applies for housing → AI system suggests allocation → Housing Authority approves → Citizen notified.

Steps (Sequence Diagram in Text):

```
Citizen -> DUMS_UI: Submit Housing Application
DUMS_UI -> Database: Store Application Data
Database -> AI_Module: Analyze eligibility & generate
allocation suggestion
AI_Module -> HousingAuthority: Send suggested allocation
HousingAuthority -> AI_Module: Approve or adjust suggestion
HousingAuthority -> Database: Update house allocation
Database -> DUMS_UI: Notify citizen of allocation
DUMS_UI -> Citizen: Display allocated house
```

Similarly, garbage collection and traffic control sequences follow:

- **Garbage:** Citizen reports → System identifies zone → Assign truck → Truck collects → Citizen notified
- **Traffic:** System monitors sensors → Detect congestion → Adjust traffic signal → Notify officer → Log for analytics





Java Code Skeleton (Basic OOP Implementation)

Simplified Java version to illustrate the core functionality:

```
class Citizen {
    int citizenId;
    String name;
    double income;
    int familySize;
    String address;

    public Citizen(int id, String name, double income, int familySize, String address) {
        this.citizenId = id;
        this.name = name;
        this.income = income;
        this.familySize = familySize;
        this.address = address;
    }

    public void applyHousing(HousingAuthority ha, CityAuthoritySystem city) {
        System.out.println(name + " applied for housing.");
        ha.reviewApplication(this, city);
    }

    public void reportGarbage(String location) {
        System.out.println(name + " reported garbage at: " + location);
    }
```

```
class House {
    int houseId;
    String location;
    int size;
    Citizen allocatedTo;

    public House(int id, String loc, int size) {
        this.houseId = id;
        this.location = loc;
        this.size = size;
    }

    public void assignCitizen(Citizen c) {
        this.allocatedTo = c;
        System.out.println("House " + houseId + " assigned to " + c.name);
    }
```

```
class GarbageTruck { (if added)
    int truckId;
    String assignedZone;
    String driver;
```

Java Code Skeleton (Basic OOP Implementation)

Simplified Java version to illustrate the core functionality:

```
class HousingAuthority {
    int authorityId;
    String name;

    public HousingAuthority(int id, String name) {
        this.authorityId = id;
        this.name = name;
    }

    public void reviewApplication(Citizen c, CityAuthoritySystem city) {
        System.out.println("Housing Authority reviewing application of " + c.name);
        for (House h : city.houses) {
            if (h.allocatedTo == null) {
                h.assignCitizen(c);
                break;
            }
        }
    }
}

public class DUMSSystem {
    public static void main(String[] args) {
        CityAuthoritySystem city = new CityAuthoritySystem();
        city.addHouse(new House(1, "Gulshan", 1200));
        city.addHouse(new House(2, "Banani", 900));
        Citizen rahim = new Citizen(101, "Rahim", 15000, 4, "Old Dhaka");
        city.addCitizen(rahim);
        HousingAuthority ha = new HousingAuthority(1, "Dhaka Housing Authority");
        rahim.applyHousing(ha, city);
        city.monitorHousing();
```

```
class CityAuthoritySystem {
    List<Citizen> citizens = new ArrayList<>();
    List<House> houses = new ArrayList<>();

    public void addCitizen(Citizen c) {
    }

    public void addHouse(House h) {
        houses.add(h);
    }

    public void monitorHousing() {
        System.out.println("Monitor
        for (House h : houses) {
            if (h.allocatedTo != nul
                System.out
        }
    }
```

Others Module

- ✓ System Integration (High-Level Workflow)
- ✓ User Interface (Web/Mobile App):
  - ➢ Citizens apply for housing, report garbage issues, or submit traffic complaints.
  - ➢ Government officials review data and take action.
- ✓ Database:
  - ➢ Stores applicant details, housing units, garbage zones, and traffic signals.
- ✓ AI/ML Components:
  - ➢ Predictive housing allocation.
  - ➢ Optimal garbage truck routing and Smart traffic light automation.
- ✓ APIs for External Systems:
  - ➢ Dhaka City Corporation (garbage trucks).
  - ➢ BRTA (traffic management).

System Scenario-2: "Agricultural Automation System (AAS)"

- ✓ Agriculture is one of the most important sectors in Bangladesh. Farmers face several challenges:
  - ❖ Inefficient irrigation management
  - ❖ Excessive fertilizer usage
  - ❖ Pest and weed attacks
  - ❖ Labor shortages
  - ❖ Low crop yield prediction accuracy
  - ❖ Delayed harvesting decisions
- ✓ The **Agricultural Automation System (AAS)** is a centralized smart farming platform designed to automate the rice cultivation process using **IoT sensors, AI/ML, drones, and automated machinery.**
- ✓ The **system can be accessed through web and mobile applications by farmers, agricultural officers, field workers, and farm managers.**
- ✓ The system **continuously collects field data from sensors and cameras and tomatically controls irrigation, fertilization, pest control, and harvesting erations.**

System Goal

- ✓ Develop a smart farming platform that:
  - ❖ Monitors crop growth
  - ❖ Controls irrigation automatically
  - ❖ Detects diseases and pests
  - ❖ Optimizes fertilizer application
  - ❖ Predicts harvesting time
  - ❖ Increases crop production





AAS Main Modules

1.Field Preparation Module

- ✓ **Responsibilities:**
  - ❖ Register farm land
  - ❖ Analyze soil condition
  - ❖ Measure pH level
  - ❖ Detect moisture level
  - ❖ Prepare cultivation schedule
- ✓ **Input:**
  - ❖ Soil sensor data
  - ❖ Farmer information
- ✓ **Output:**
  - ✓ Land readiness report

2.Planting Module

- ✓ **Responsibilities:**
  - ❖ Manage seed information
  - ❖ Schedule planting
  - ❖ Control automatic seed planter
- ✓ **Input:**
  - ❖ Seed type
  - ❖ Field information
- ✓ **Output:**
  - ❖ Planting confirmation

AAS Main Modules

3.Irrigation Control Module

- ✓ **Responsibilities:**
  - ❖ Monitor soil moisture
  - ❖ Control water pumps
  - ❖ Manage irrigation schedules
- ✓ **Input:**
  - ❖ Moisture sensors
  - ❖ Weather forecast
- ✓ **Output:**
  - ❖ Automatic irrigation decision

4. Fertilization Module

- ✓ **Responsibilities:**
  - ❖ Monitor nutrient levels
  - ❖ Calculate fertilizer requirements
  - ❖ Schedule fertilizer application
- ✓ **Input:**
  - ❖ Soil nutrients
  - ❖ Crop growth stage
- ✓ **Output:**
  - ❖ Fertilizer recommendation

AAS Main Modules

5. Pest & Weed Control Module

✓ **Responsibilities:**

- Detect pests using camera images
- Detect weeds
- Recommend pesticide usage
- Control spraying drones

✓ **Input:**

- Drone images
- Camera feeds

✓ **Output:**

- Pest treatment plan

6. Growth Monitoring Module

✓ **Responsibilities:**

- Monitor plant growth
- Calculate crop health index
- Estimate yield

✓ **Input:**

- Drone images
- Sensor data

✓ **Output:**

- Growth report

AAS Main Modules

7. Harvesting Module

- ✓ **Responsibilities:**
  - ❖ Predict harvesting date
  - ❖ Control harvesting machines
- ✓ **Input:**
  - ❖ Growth stage
  - ❖ Weather condition
- ✓ **Output:**
  - ❖ Harvest schedule

8. Post-Harvest Module

- ✓ **Responsibilities:**
  - ❖ Store harvested rice
  - ❖ Manage transportation
  - ❖ Track warehouse inventory
- ✓ **Input:**
  - ❖ Harvest quantity
- ✓ Output:
  - ❖ Storage report

AAS Main Modules

9. Central Control System

- ✓ Responsibilities:
- ✓ Coordinate all modules
- ✓ Store data
- ✓ Generate reports
- ✓ Run AI prediction models





AAS Main Modules

Step-1: Understanding the Problem Domain

- ✓ **Main Actors**
  - ❖ **Human Actors**
    1. Farmer
    2. Agricultural Officer
    3. Farm Manager
    4. Drone Operator

  - ❖ **System Actors**
    1. Soil Sensor
    2. Moisture Sensor
    3. Weather API
    4. AI/ML Prediction Engine
    5. Irrigation Pump
    6. Drone Sprayer
    7. Harvesting Machine





Use Case 1:
Smart Irrigation Control

- **Goal**
  - Automatically irrigate fields based on soil moisture and weather conditions.
- **Primary Actor**
  - Farmer
- **Supporting Actors**
  - Moisture Sensor
  - Weather API
  - Irrigation Pump
  - AI Irrigation System
- **Preconditions**
  - Sensors are connected.
  - Irrigation pumps are registered.

Main Scenario

1.Moisture sensors continuously monitor soil.
2.Soil moisture drops below threshold.
3.Sensor sends data to AAS.
4.Weather API provides forecast.
5.AI module analyzes moisture and weather.
6.System decides irrigation is necessary.
7.Pump receives activation command.
8.Water is supplied automatically.
9.Moisture reaches target level.
10.Pump stops.
11.Farmer receives notification.

- **Alternative Scenario**
  - If rain is expected within a few hours, irrigation is postponed.
- **Output**
  - Efficient water management.

Use Case 2
Pest Detection and Control

Goal
Automatically detect and control pest attacks.
Primary Actor
Agricultural Officer
Supporting Actors
Drone Camera
AI Vision System
Drone Sprayer
Preconditions
Drone system operational.

Main Scenario
Drone captures field images.
Images uploaded to AAS.
AI detects pest infection.
Infection severity calculated.
Treatment recommendation generated.
Agricultural Officer approves.
Sprayer drone receives task.
Pesticide sprayed.
Treatment report stored.
Farmer notified.

Alternative Scenario
If no pest is detected, monitoring continues.
Output
Reduced crop loss.

Use Case 3
Harvest Prediction and Scheduling

Goal
Predict optimal harvesting time.
Primary Actor
Farm Manager
Supporting Actors
Growth Monitoring Module
Weather API
Harvesting Machine
Preconditions
Crop growth data available.

Main Scenario
Growth module analyzes crop maturity.
Weather forecast retrieved.
AI predicts harvesting date.
Harvest schedule generated.
Manager reviews schedule.
Harvesting machine assigned.
Harvest operation completed.
Storage information updated.

Alternative Scenario
If severe weather is expected, harvesting date is adjusted.
Output
Timely harvesting and reduced crop damage.

Integrated Scenario: A Day in Smart Farming Managed by AAS

- ✓ In the morning, soil moisture sensors detect that water levels are below the required threshold. The Irrigation Control Module consults weather forecasts and automatically activates irrigation pumps.
- ✓ Later, drones fly over the rice field and capture images. The Pest & Weed Control Module analyzes the images and detects a pest outbreak in one section of the farm. The Agricultural Officer approves the treatment recommendation, and a spraying drone applies pesticide automatically.
- ✓ In the afternoon, the Growth Monitoring Module evaluates crop maturity and predicts that harvesting should begin within five days. The Harvesting Module schedules harvesting machinery and updates the farmer.
- ✓ Throughout the day, the Central Control System collects data from all modules and provides real-time reports through the farmer's mobile application.
- ✓ The AAS enables data-driven, automated, and efficient farming operations while improving crop yield and reducing labor costs.

OOAD – Main Classes (Domain Model)

- ✓ Farmer
- ✓ AgriculturalOfficer
- ✓ FarmManager

- ✓ Field
- ✓ Crop
- ✓ Sensor
- ✓ SoilSensor
- ✓ MoistureSensor

- ✓ IrrigationController
- ✓ FertilizerController
- ✓ PestController
- ✓ GrowthMonitor
- ✓ HarvestManager

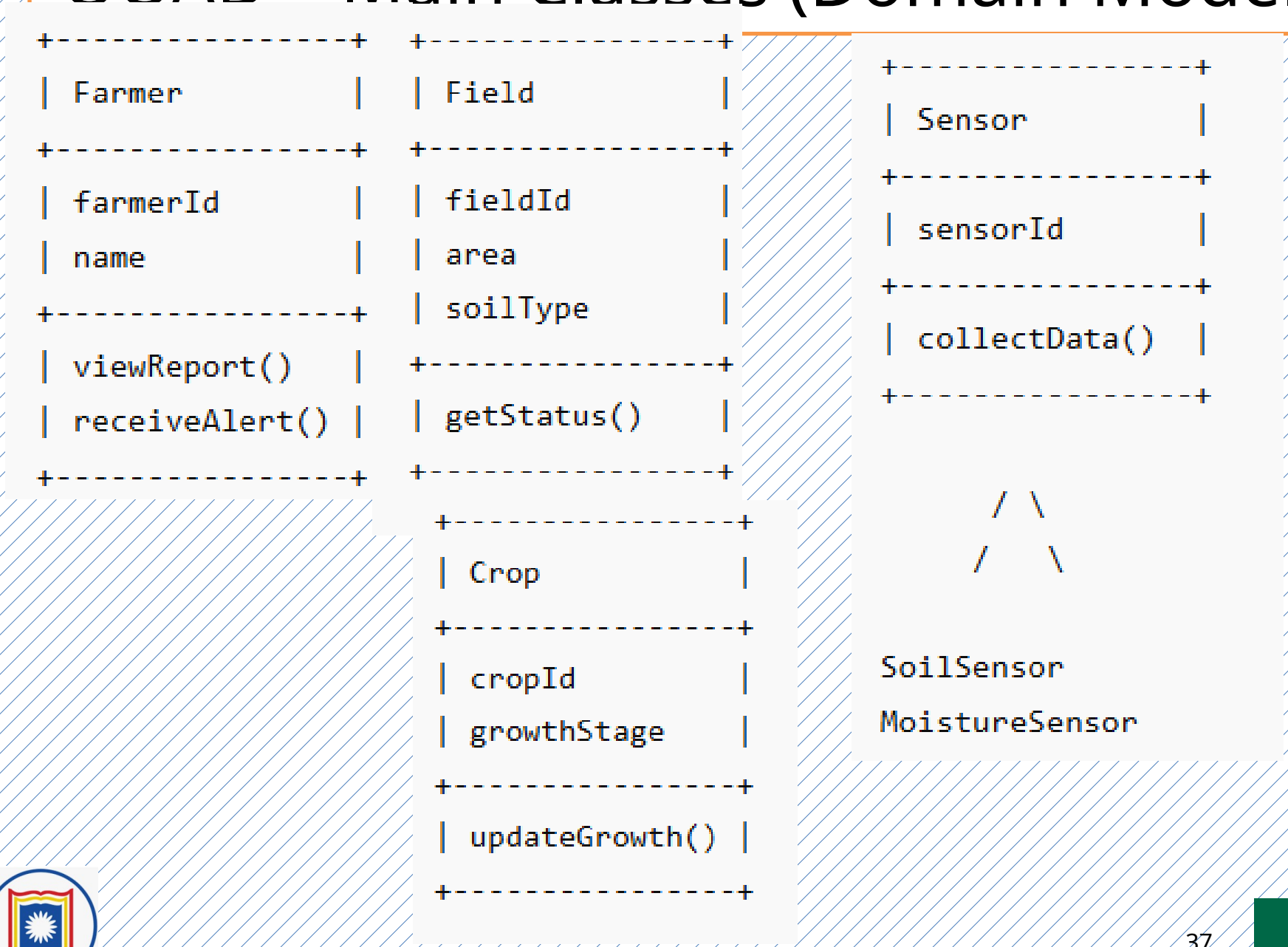
- ✓ Drone
- ✓ SprayerDrone
- ✓ HarvestMachine

- ✓ WeatherService
- ✓ AIEngine
- ✓ ReportGenerator





OOAD – Main Classes (Domain Model)



Suitable Design Patterns for AAS

Module	Design Pattern
Central Control System	**Singleton**
Sensor Notification	**Observer**
AI Decision Engine	**Strategy**
Report Generation	**Factory**
Irrigation Controller	**Command**
Drone Control	**Adapter**
Module Integration	**Facade**





Example UML Sequence Diagram (Irrigation)

- Farmer
  |
- MoistureSensor
- |
- AIEngine
- |
- IrrigationController
- |
- WaterPump
- Sensor -> AIEngine : sendData()
- AIEngine -> Controller : irrigate()
- Controller -> Pump : start()
- Pump -> Controller : completed()
- Controller -> Farmer : notify()

Simple Python Demonstration ttern)

```
class Farmer:
    def update(self, message):
        print("Farmer Notification:", message)
class MoistureSensor:
    def __init__(self):
        self.observers = []
    def attach(self, observer):
        self.observers.append(observer)
    def notify(self, message):
        for observer in self.observers:
            observer.update(message)
    def check_moisture(self, level):
        if level < 30:
            self.notify(
                "Soil moisture is low. Irrigation started."          )
```

- ✓ farmer = Farmer()
- ✓ sensor = MoistureSensor()
- ✓ sensor.attach(farmer)
- ✓ sensor.check_moisture(25)
- ✓ Output:
  - ❖ Farmer Notification:
    Soil moisture is low. Irrigation



Real world object and Benifit

- ✓ The benefit of representing real-world entities—such as Farmer, SensorModule, and IrrigationController—in the Automated Agricultural System as objects in your code.
- ✓ Modeling **Farmer**, **SensorModule**, and **IrrigationController** as objects helps create a clear, modular structure. Each object manages its own functions—such as monitoring soil data or controlling irrigation—making the system easier to update, automate, and integrate. This improves reliability, simplifies troubleshooting, and supports smart, data-driven farming.





Why "Good software design begins before coding"?

- ✓ **Key Concept:** Design is a blueprint; coding is implementation.
- ✓ If the design is weak, the software will be hard to understand, extend, or maintain.
- ✓ A good design ensures:
  - Clear structure
  - Easier debugging and testing
  - Flexibility for future changes
- ✓ Before writing code for housing allocation, we:
  - Identified core problems (overpopulation, garbage, traffic).
  - Defined requirements (e.g., "Applicants must submit income proof").
  - Designed classes (Applicant, HousingUnit) and workflows.





Establishing Initial Software Requirements

- **Key Concept:** Gather what the system must do (functional) and should do (non-functional).
- DUMS Example:
- **Functional Requirements:**
  - Citizens can apply for housing online.
  - Garbage trucks are assigned based on zone complaints.
- **Non-Functional Requirements:**
  - Scalable to 10M users (Dhaka's population).
  - Real-time traffic signal updates (<1 sec latency).





OOAD Main Activities : 1. Conceptual Design

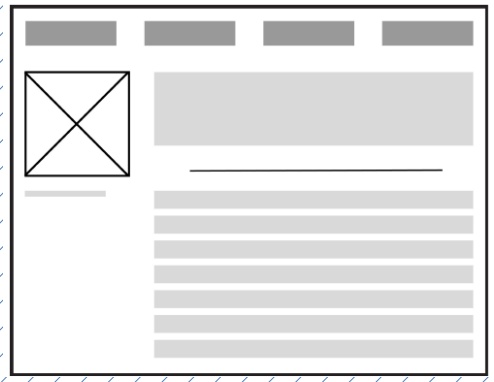
- ✓ Focuses on what the system should do
- ✓ **Key Concept:** High-level architecture without technical details.
- ✓ Involves understanding the problem key objects, classes, and their relationships.
- ➢ DUMS Example:
- ✓ **Entities/Objects Identified:**
  - ➢ Applicant, HousingUnit, GarbageZone, TrafficSignal.
- ✓ **Relationship:**
  - ➢ A Citizen submits a Complaint linked to a GarbageZone.
  - ➢ Housing Allocation System matches Applicant to HousingUnit.





OOAD Main Activities : 1. Conceptual Design

- ✓ The conceptual design recognizes appropriate **components, connections, and responsibilities** of the software product.
- ✓ Conceptual designs are expressed or communicated through **conceptual mock-ups**.
- ✓ A visual notations that provide initial thoughts for how requirements will be satisfied.
- ✓ Mock-ups for software involving user interfaces are often presented as **wireframes**, which are a kind of blueprint or basic visual representation of the product.



OOAD Main Activities : 1. Conceptual Design

- ✓ Mock-ups show **major components and their relations.**
- ✓ They help identify missing or problematic parts, requiring clarification or further conceptual design.
- ✓ Each component has **specific responsibilities**.
- ✓ Mock-ups focus on structure, not technical details.

```
+-------------------------------------+
| DHAKA URBAN MANAGEMENT SYSTEM (DUMS)|
+-------------------------------------+
| [Housing] [Garbage] [Traffic]       |  ← Tabs
|-------------------------------------|
| WELCOME, USER!                      |
|                                     |
| [Apply for Housing] ← Button        |  → Links to `Applicant` class
| [Report Garbage]    ← Button        |  → Creates `Complaint` object
| [Report Traffic Jam] ← Button       |  → Triggers `TrafficReport`
|                                     |
| NOTIFICATIONS:                      |
| - Your housing app #123 is pending  |  ← From `HousingAllocationSystem
```





OOAD Main Activities : 1. Conceptual Design

Purpose and **Role of Conceptual Mock-ups**

- ✓ **Visualize OO Concepts**
  - ✓ *Example*: UML class diagrams for an Order-Product relationship.
- ✓ **Validate Design Early**
  - ✓ *Example*: Wireframes showing how Cart and Checkout classes interact.
- ✓ **Bridge Client-Developer Gaps**
  - ✓ *Example*: A mock-up of a Dashboard class with widgets (client thinks "features", dev thinks "composition").
- ✓ **Tools**:
  - ➢ UML Diagrams (Class, Sequence)
  - ➢ Wireframing tools (Figma, Balsamiq)





Conceptual Mock-ups illustrate in the Software Design process

- ✓ Conceptual mock-ups are **visual representations** of how the system will look and behave before implementation. They illustrate:

1.User Interface (UI) Flow – how users navigate screens.

2.System Behavior – expected actions when users interact with components.

3.Requirements Validation – clients can confirm if the design matches their needs.

4.Communication Tool – bridges the gap between designers, developers, and stakeholders.

- ✓ **For example, a** conceptual mock-up of a **banking app** may show:
  - ➢ Login screen → Dashboard → Transfer Money form → Confirmation screen.
  - ➢ This allows the client to approve the design before coding begins.
- ✓ Conceptual mock-ups illustrate the **look-and-feel, functionality, and usability** of the software, reducing misunderstandings between clients and developers.





OOAD Main Activities : 2.Technical Design

Key Concept: Low-level implementation plans (algorithms, data structures).

- ✓ Builds on conceptual design to define the technical details of the solution.
- ✓ Conceptual design = outlines major components, connections, and responsibilities.
- ✓ Technical design = explains how responsibilities are implemented.
- ✓ Components are refined into smaller, detailed parts until each has clear technical specifications.
- ✓ Technical diagrams are used to:
  - ➢ Visualize structure and behavior of components.
  - ➢ Capture details not shown in conceptual mock-ups.
- ✓ Example:
  - ➢ Kitchen = a component of a house.
  - ➢ Technical design refines it further (e.g., flooring material must be easy to clean if cooking is frequent).





OOAD Main Activities : 2.Technical Design

Key Concept: Low-level implementation plans (algorithms, data structures).

DUMS Example:

- ✓ **Algorithms:**
  - Housing allocation uses a priority queue (income-based ranking).
  - Traffic control uses Dijkstra's algorithm for emergency vehicle routing.
- ✓ **Database Schema:**
  - Applicants table with fields: income, family_size.





OOAD Main Activities : 2.Technical Design

Aspect	Conceptual Design	Technical Design
Purpose	Defines *what* the system should do (major components and their responsibilities).	Defines *how* the system will work (detailed implementation of each component).
Detail Level	High-level overview, not very specific.	Low-level detail, refined until every part is well-defined.
Focus	Identifies major components and their interactions.	Specifies internal structure, materials, methods, and technical details.
Tools	Mock-ups, conceptual diagrams.	Technical diagrams (UML, architecture diagrams, flowcharts, etc.).
House Example	Kitchen is a component of the house.	Kitchen requires flooring; flooring must be easy to clean for frequent cooking use.
Software Example	User Management Module exists in the system.	User Management Module requires login with encryption, password reset via email.

Compromise in Requirements and Design

- **Compromises may be needed** to create an acceptable solution.
- **Communication and feedback** with the client are essential.
- Example: Client wants an open kitchen, but structural support (post/beam)
- may require adjustment → compromise needed.
- **Designs may need rework** if:
  - Conceptual design cannot be achieved in technical design.
  - Requirements are not fully met.
- **Early validation is critical**:
  - Easier to revise during planning than after coding starts.
- **Larger systems** = more design time, more components, more refinements.
- **Final agreed design** → technical diagrams form the blueprint for construction/implementation.





Design for Quality Attributes

- **Broader view in design**: Designing a software system so that it not only performs its required functions but also achieves important **non-functional requirements (quality attributes)** such as performance, convenience, security, reliability, usability, and maintainabilit
- **Trade-offs are common:**
  - **Functional Requirements define *what* the system should do**.
  - **Quality attributes** define *how well* it should work.
- **Key quality attributes**: Making architectural and design decisions that help the software achieve these desired qualities
- **Competing attributes**: improving one may weaken another.
- **Software architects' role**:
  - Continuously balance attributes.
  - Decide which attribute is more critical in a given context.
- **Deadlines matter**: time constraints affect feasibility of quality improvements.
- **Example (House Door)**:
  - Too many locks = high security but low convenience.
  - Good design balances security, convenience, and performance.

Example: Online Shopping syste

Functional Requirements

The system should allow users to:

- Register
- Log in
- Search products
- Add items to the cart
- Pay online
- These describe **what** the system does.

Quality Attribute	**Design Decision**
Performance	Cache product pages to reduce load time
Security	Encrypt passwords and use HTTPS
Reliability	Maintain database backups and redundant servers
Usability	Provide a simple and intuitive interface
Maintainability	Organize the application into separate modules
Scalability	Use cloud infrastructure to support more users

Example: ATM System

Functional Requirements

- Withdraw cash
- Deposit money
- Check balance

Quality Attribute	**Design Decision**
Security	PIN verification and encrypted communication
Performance	Complete transactions within a few seconds
Reliability	Ensure operation even during network interruptions
Availability	Keep ATMs operational 24/7
Maintainability	Allow software updates without replacing the hardware





Context and Consequences in Design

- **Context matters**:
  - Different environments = different requirements.
  - Example: Public-facing system needs stronger security than internal corporate system.
- **Consequences of design choices**:
  - Some decisions may cause problems later.
  - Example: Solution works for small data but fails with large data.
- **Good practices**:
  - Seek feedback from other developers/stakeholders (design reviews).
  - Prototype alternative ideas to compare approaches.
  - Test thoroughly before full implementation.
  - Use tests with different scenarios (e.g., small vs large data) to find limitations.





Satisfying Qualities in Software Design

- **Qualities come from requirements**:
  - **Functional requirements** = what the system should do.
  - **Non-functional requirements** = how well the system should do it.
- **Functional requirements**:
  - Ensure **correctness**.
  - Example: Music app must download and play songs.
- **Non-functional requirements**:
  - Focus on performance, efficiency, resource usage, maintainability, flexibility, reusability.
  - Example: Music app can only download songs within a memory limit.
- **Requirements are often incomplete** → clarified through client/end-user interactions.
- **Trade-offs and compromises** may be needed due to constraints.
  - Example: All cars provide transportation (functional), but differences in acceleration, fuel economy, and handling (non-functional) distinguish minivan vs. sports car.
- **Verification methods**:
  - Reviews and tests check if qualities are satisfied.
  - End-user feedback validates real-world qualities.

Compromise in Software Design

- **Multiple perspectives matter**:
  - Users care about performance, usability.
  - Developers care about reusability, maintainability.
- **Software structure affects both sides**.
- **Common trade-offs**:
  - **Performance vs. Maintainability**
    - High-performance code may be harder to understand/maintain.
    - Backward compatibility may reduce both performance and maintainability.
  - **Performance vs. Security**
    - Stronger security adds overhead, lowering performance.
- **Balance qualities**:
  - Prioritize what is most important for the system.
  - Ask: *Can one quality be reduced slightly to improve another?*

Importance of Design and Object-Oriented Thinking

Key Concept: OOAD promotes modularity, reusability, and maintainability.

- **Encapsulation:** **Group data and behavior into a single unit (class).**
- **Abstraction:** **Focus only on essential features, ignoring details.**
- **Inheritance:** **Reuse common features by extending classes.**
- **Polymorphism:** **Same action behaves differently depending on the object.**

These principles make the design more reusable, flexible, and easier to maintain.





Importance of Design and Object-Oriented Thinking

- DUMS Example:
  - **Modularity:** Each module (housing, garbage, traffic) is independent.
  - **Reusability:** User class inherited by Citizen and Government Official.
  - **Maintainability:** Changing garbage collection logic doesn't break traffic module.



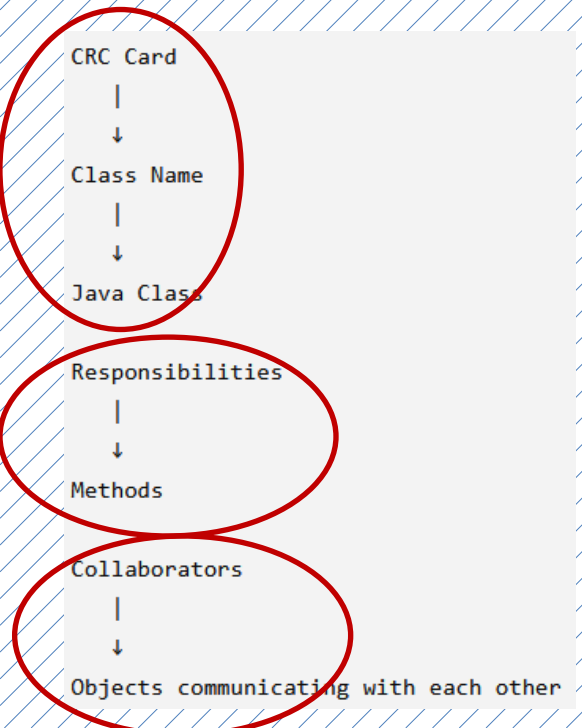


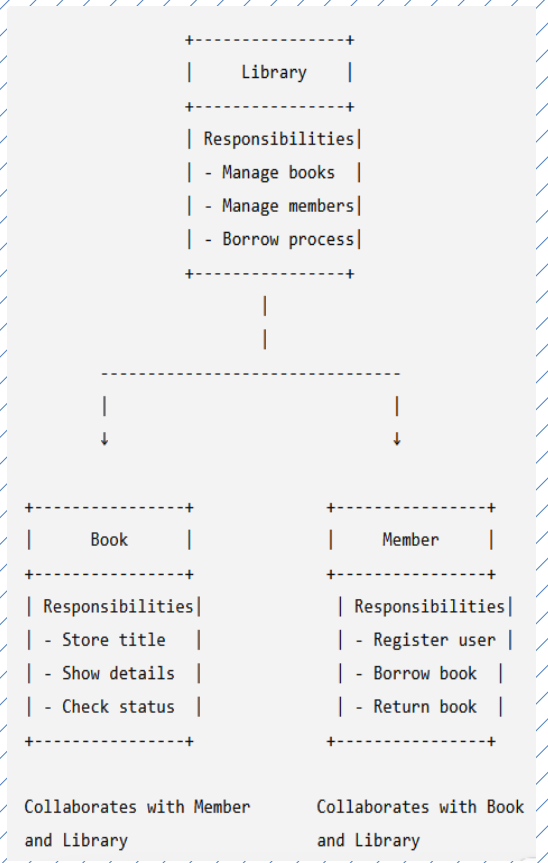
Class Responsibility Collaborator (CRC) Cards

- **Purpose**:
  - Represent **components, responsibilities, and connections** at a high level.
  - A brainstorming tool for object-oriented design, help refine conceptual design.
- **Benefits**:
  - Organize system components into **classes**.
  - Identify **responsibilities** of each class.
  - Define how classes **collaborate** with others.
  - Simple, visual tool to support brainstorming and design.
- **Structure of a CRC Card**:
  - **Top** → Class name.
  - **Left side** → Responsibilities (tasks of the class).
  - **Right side** → Collaborators (other classes it interacts with).
- **Analogy**:
  - Like index cards used for organizing talking points.
  - Each card = one class in the system.
- **Each card represents a class:**
  - **Class Name (e.g., Book)**
  - **Responsibilities (what it does, e.g., "Store title, author")**
  - Collaborators (other classes it interacts with, e.g., Member)

Class Responsibility Collaborator (CRC) Cards

Class	Responsibilities	Collaborators
Book	- Store book information- Display book details- Check availability	Member, Library
Member	- Register member- Borrow book- Return book	Book, Library
Library	- Manage books- Manage members- Process borrowing	Book, Member





CRC (Class–Responsibility–Collaborator) Cards

A simple, powerful tool for high-level object-oriented design.

Purpose

- Represent components, responsibilities, and connections at a high level.
- A brainstorming tool for object-oriented design, help refine conceptual design.

Benefits

- Organize system components into classes.
- Identify responsibilities of each class.
- Define how classes collaborate with others.
- Simple, visual tool to support brainstorming and design.

Structure of a CRC Card

Top → Class name.

Left side → Responsibilities (tasks of thie class).

Right side → Collaborators (other classes it interacts with).

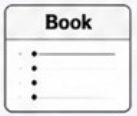
Analogy



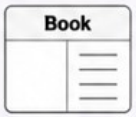
Like index cards used for organizing talking points.



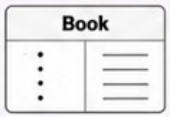
Each card = one class in the system.



Left side = Responsibilities (what it does)



Right side = Collaborators (who it works with)



Together = complete high-level view of the class.

Example: One CRC Card

Book

Responsibilities	Collaborators
• Store title, author, ISBN	• Member
• Maintain availability status	• Loan
• Update book details	• Librarian
• Calculate fine (if returned late)	• Catalog
• Provide book information	

What the class does

Other classes it interacts with

Example: CRC Cards for a Library System

Member

Responsibilities	Collaborators
• Store member details	• Book
• Borrow book	• Loan
• Return book	• Librarian
• View loan history	

Book

Responsibilities	Collaborators
• Store title, author, ISBN	• Member
• Maintain availability status	• Loan
• Update book details	• Librarian
• Calculate fine	• Catalog
• Provide book information	

Loan

Responsibilities	Collaborators
• Record issue date	• Member
• Record return date	• Book
• Calculate fine	• Librarian
• Update loan status	

Librarian

Responsibilities	Collaborators
• Manage members	• Member
• Manage books	• Book
• Issue/return books	• Loan
• Handle fines	• Catalog

Catalog

Responsibilities	Collaborators
• Store catalog data	• Book
• Search books	• Librarian
• Update catalog	

Key Takeaway

CRC cards help you think about classes, where each class is responsible for, and which other classes it needs to work with—before writing any code.

CRC (Class–Responsibility–Collaborator) Cards

A simple, visual tool for high-level object-oriented design.

Purpose

- Represent components, responsibilities, and connections at a high level.
- A brainstorming tool for object-oriented design, help refine conceptual design.

Benefits

- Organize system components into classes.
- Identify responsibilities of each class.
- Define how classes collaborate with others.
- Simple, visual tool to support brainstorming and design.

Structure of a CRC Card

Top → Class name.

Left side → Responsibilities (tasks of thie class).

Right side → Collaborators (other classes it interacts with).

Example: One CRC Card

Citizen	
Responsibilities	**Collaborators**
• Register and login	• Service Request
• Submit service request	• Notification
• Track request status	• Department
• Provide feedback	• Payment
• View notifications	

What the class does

Other classes it interacts with

Example: CRC Cards for Dhaka Urban Management System

Citizen

Responsibilities	Collaborators
• Register and login	• Service Request
• Submit service request	• Notification
• Track request status	• Department
• Provide feedback	• Payment
• View notifications	

Service Request

Responsibilities	Collaborators
• Create and store request	• Citizen
• Assign to department	• Department
• Update request status	• Staff
• Maintain request history	• Notification

Department

Responsibilities	Collaborators
• Manage departments	• Service Request
• Allocate resources	• Staff
• Monitor performance	• Report
• Generate reports	

Staff

Responsibilities	Collaborators
• Receive assigned	• Service Request
• Update task status	• Department
• Add remarks	• Notification
• Escalate issues	

Notification

Responsibilities	Collaborators
• Create notifications	• Citizen
• Send to users	• Staff
• Mark as read	• Service Request
• Maintain records	

Payment

Responsibilities	Collaborators
• Process payments	• Citizen
• Maintain transaction records	• Service Request
• Generate receipts	
• Refund (if needed)	

Key Takeaway

CRC cards help you think about classes, what each class is responsible for, and which other classes it needs to work with—before writing any code.

System Overview (Dhaka Urban Management System)

Citizens submit requests → Departments assign them to staff → Staff process requests → Notifications and payments are handled → Reports are generated for decision making.

CRC Cards Technique

- ✓ **Key Concept:** A brainstorming tool to design classes.
- ✓ **DUMS Example for Housing AllocationSystem:**

Class	Responsibility	Collaborator
HousingAllocationSystem	Match applicants to housing units	Applicant, HousingUnit
Applicant	Provide personal/financial data	HousingAllocationSystem

- ✓ How it Helps:
  - ✓ Clarifies that Housing Allocation System needs Applicant data to function.
  - ✓ Exposes missing links (e.g., "Should Applicant interact with Garbage Zone? No!").





Scenario: CRC Example

❖ **Problem:** Consider a **Dhaka Urban Management System** scenario: a city resident logs into the smart city portal using their ID and password to verify their identity with the municipal database. The user is then provided options to **report waste collection issues**, **pay utility bills**, **request road maintenance**, or **track service status**. Explain how **CRC (Class–Responsibility–Collaborator)** cards can be utilized to isolate and define the system components, including their attributes, functionalities, and interactions — for example, classes such as *Citizen*, *MunicipalService*, *Complaint*, and *Payment* can be defined along with their responsibilities and collaborating relationships.

✓ **Using CRC Cards in the Dhaka Urban Management System**

CRC (Class–Responsibility–Collaborator) cards help break down the Dhaka Urban Management System into manageable components by defining what each class represents, what it is responsible for, and which other classes it interacts with. This method helps designers isolate key system features and understand how objects collaborate to complete tasks such as logging in, submitting complaints, or paying bills.

For example:

❖ **Citizen Class**

Responsibilities: Authenticate user credentials, manage user profile, initiate service requests.

Collaborators: MunicipalService (to request services), Complaint (to report issues), Payment (to pay ls).

Scenario: CRC Example

- **MunicipalService Class**

  Responsibilities: Provide access to available city services such as waste management, road repair, or bill inquiries.

  Collaborators: Citizen (service requester), Complaint (for issue reporting), Payment (for bill processing).

- **Complaint Class**

  Responsibilities: Record waste collection issues, road maintenance requests, and other public service complaints.

  Collaborators: Citizen (complaint initiator), MunicipalService (service handling), ServiceStatus (to track progress).

- **Payment Class**

  Responsibilities: Process utility bill payments, verify transaction details, and update user records.

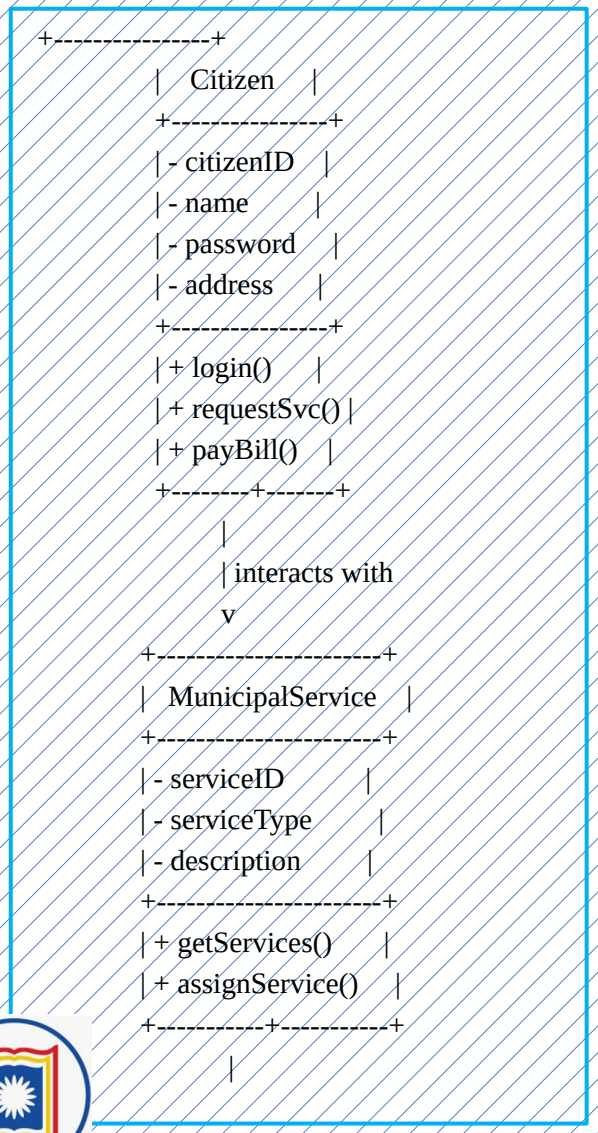
  Collaborators: Citizen (payer), MunicipalService (billing information), TransactionGateway (processing).

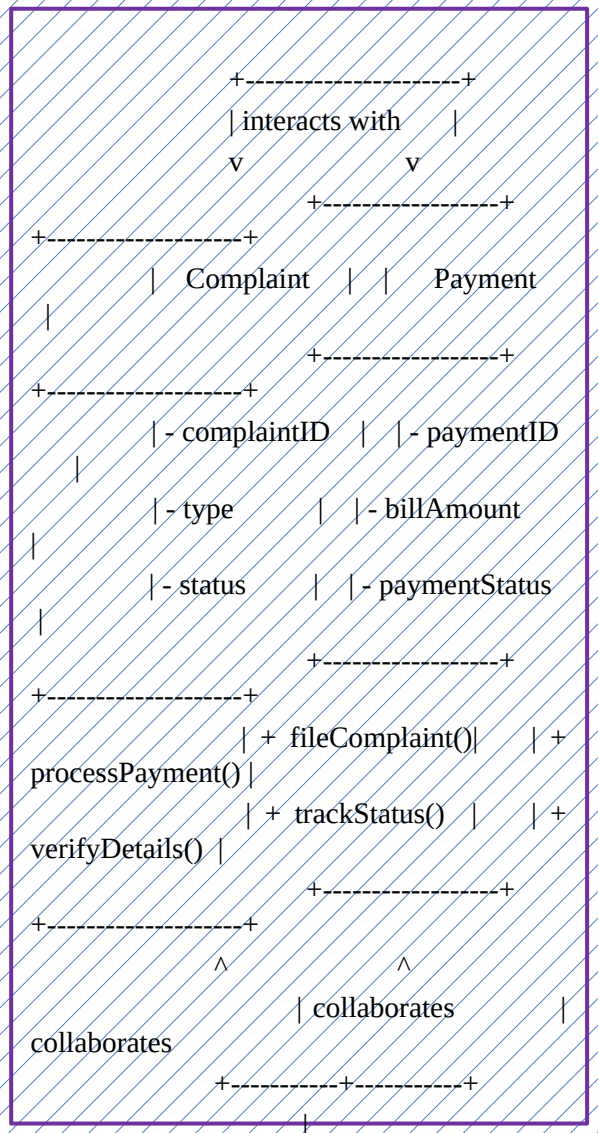
  ✓ Using CRC cards makes it easier to identify each class's attributes (e.g., CitizenID, ServiceType, ComplaintStatus), define their behaviors (e.g., login(), submitComplaint(), processPayment()), and outline how they interact within the system. This approach ensures better system design, promotes modularity, and simplifies implementation and future maintenance.

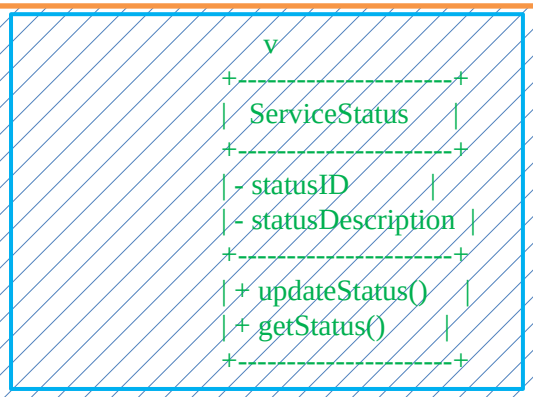




Scenario: CRC Example







- ✓ **Diagram Shows**
- ✓ **Citizen** interacts with:
  - ✓ **MunicipalService** to request services
  - ✓ **Complaint** to report issues
  - ✓ **Payment** to pay bills
- ✓ **MunicipalService** collaborates with:
  - ✓ **Complaint** for issue handling
  - ✓ **Payment** for bill-related operations
  - ✓ **ServiceStatus** to update and trackpro
- ✓ **Complaint** and **Payment** both interact with **ServiceStatus** for tracking results or progress.

Scenario: CRC Example (Automated Agricultural System)

- **Problem:** Consider an Automated Agricultural System scenario: a farmer accesses the automated control system through a mobile app, verifies their identity using a registered farm ID, and is then given options to monitor soil moisture, control irrigation pumps, check crop growth data, or analyze weather forecasts. Explain how CRC (Class–Responsibility–Collaborator) cards can be used to isolate and define the system components, including their attributes, functionalities, and interactions — for instance, classes like Farmer, SensorModule, IrrigationController, and WeatherService can be defined to represent various responsibilities and their collaborations in the automated process.





CRC (Class–Responsibility–Collaborator) Cards

Automated Agriculture System



Scenario: A farmer accesses the automated control system through a mobile app, verifies identity using a registered farm ID, and then gets options to monitor soil moisture, control irrigation pumps, check crop growth data, or analyze weather forecasts.

1. Farmer

Attributes	Responsibilities
• farmID • name • phoneNumber • password	• Login to system • View dashboard • Monitor soil data • Control irrigation • View crop growth • View weather forecast

Collaborators

- MobileApp, AuthService, SensorModule, IrrigationController, CropDataService, WeatherService

2. MobileApp

Attributes	Responsibilities
• appID • version • deviceInfo	• Provide login interface • Show dashboard • Send user requests • Display results/alerts • Manage user session

Collaborators

- Farmer, AuthService, SensorModule, IrrigationController, CropDataService, WeatherService, NotificationService

3. AuthService

Attributes	Responsibilities
• registeredFarmIDs • encryptedData • token	• Verify farm ID • Authenticate user • Generate session token • Manage user access • Logout user

Collaborators

- MobileApp, Farmer

4. SensorModule

Attributes	Responsibilities
• sensorID • location • type • lastReading • soilMoisture	• Collect soil data • Send data to system • Monitor sensor status

Collaborators

- MobileApp, IrrigationController, CropDataService

5. IrrigationController

Attributes	Responsibilities
• pumpID • location • status • mode (Auto/Manual) • lastRunTime	• Receive irrigation command • Control pump (ON/OFF) • Schedule irrigation • Monitor pump status • Send status updates

Collaborators

- MobileApp, SensorService, NotificationService

6. CropDataService

Attributes	Responsibilities
• cropID • cropType • plantingDate • growthStage • healthIndex	• Collect crop growth data • Analyze growth (ON/OFF) • Schedule irrigation • Monitor pump status • Send status updates

Collaborators

- MobileApp, SensorModule

7. WeatherService

Attributes	Responsibilities
• serviceID • location • apiKey • lastUpdate	• Collect crop growth data • Forecast weather • Detect crop issues • Send updates

Collaborators

- MobileApp, SensorModuleService

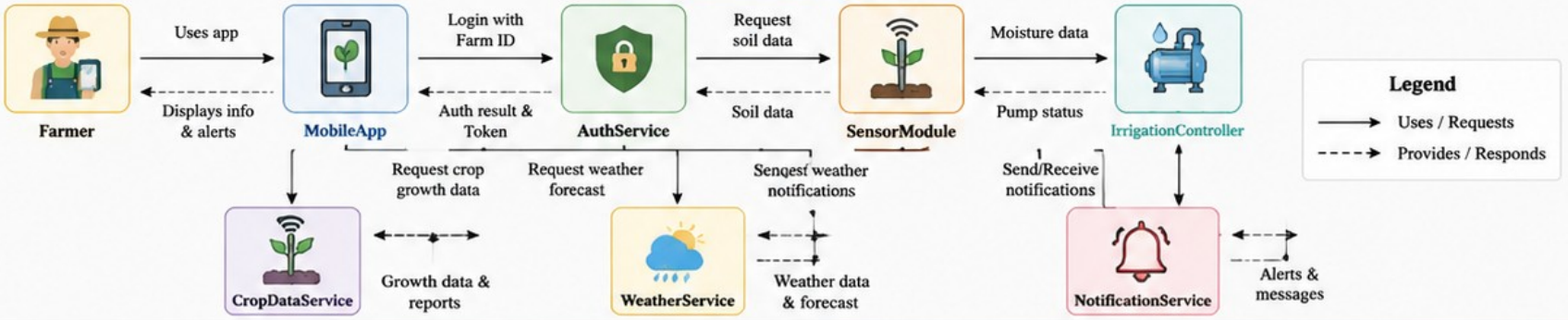
8. WotificationService

Attributes	Responsibilities
• serviceID • channels • messageQueue	• Send alerts • Send reminders • Send weather updates • Send updates

Collaborators

- MobileApp, Farmer, IrrigationController, WeatherService, CropDataService

Class Collaboration Diagram



Is IT possible to fail project work?

✓ Poor Requirements Analysis

- If requirements are not captured correctly in the object model (use cases, class diagrams), the system may not meet the client's expectations.
  Example: Missing a key user role in a DUMS leads to incomplete functionality.
- Building an e-commerce system without understanding payment gateway integration needs.

✓ Incorrect Abstraction and Encapsulations

- Using too many low-level details in classes makes the design rigid and hard to extend.
  Example: Putting database logic inside UI classes instead of separating layers. Or Creating a monolithic User class instead of separate Customer, Admin, and Vendor subclasses.
- Weak Encapsulations: Exposing database connection details in a public class, leading to security risks.

✓ Overuse of Inheritance

- Deep inheritance hierarchies make systems complex and fragile.
  Example: A vehicle class hierarchy with too many subclasses becomes unmanageable.



Is IT possible to fail project work?

- ✓ Over-Engineering
  - ➢ Example: Implementing a complex design pattern (e.g., Observer) for a simple notification system.
- ✓ Poor Reusability and Maintainability
- ✓ Mismatch Between Design and Implementation



Trade-offs between the client's need and developer's scope

- In software design, **trade-offs** occur because clients usually want maximum functionality, low cost, and quick delivery, while developers must work within technical feasibility, resources, and time constraints. Key Trade-offs:
- **Functionality vs. Cost/Time**
  - Clients may demand advanced features, but developers must balance feasibility with budget and deadlines.
- **Performance vs. Flexibility**
  - Clients may want very high performance, while developers may prefer flexible, maintainable code.
- **Customization vs. Standardization**
  - Clients may request customized features, but developers may use standardized frameworks to reduce complexity.



Trade-offs between the client's need and developer's scope

- ✓ In developing an **E-commerce system**,
  - The **client** wants one-click checkout, real-time delivery tracking, and AI-based recommendations.
  - The **developer** explains that implementing all at once would increase cost and delay delivery.
  - **Trade-off:** The system is delivered with checkout and tracking first; AI recommendations are planned for a later release.
- ✓ Trade-offs ensure the project is practical and deliverable while still meeting the essential client requirements.

Client Needs	Developer Constraints	Compromise
"I want all features in 2 months"	Realistic timeline is 6 months	**Prioritize MVP** (Minimum Viable Product)
"The app must work on	High testing overhead	**Phase-wise rollout** (Mobile-first

Thank You (Q/A)





Part A: Object-Oriented Design

MODULE 2: OBJECT-ORIENTED MODELLING



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ **Creating Models in Design**
- ✓ **Major Design Principle**

CREATING MODELS IN DESIGN

Design well before you code well

1. DESIGN FIRST, CODE LATER

- Don't jump directly into coding → first understand requirements & design properly
- Design step bridges requirements → product
- Iterative process → deals with problem space & solution space
- Design must be understandable for both users & developers (common terms)
- Object-Oriented Approach helps describe concepts as objects (understood by both users & developers)
- Popular for solving complex problems

Design Process: From Requirements to Product

Requirements → Design → Product

Problem Space

Understand the problem, needs, constraints

Iterative Process

Solution Space

Explore possible solutions and alternatives

2. TWO LEVELS OF DESIGN

1 Conceptual Design

- Identify key objects
- Break the problem into manageable parts

2 Technical Design

- Refine object details
- Define attributes (data) and behaviors (operations) for implementation

3. TYPES OF OBJECTS IN DESIGN MODELS

Entity Objects

- Represent things or information in the problem domain
- Focus on the problem space

Control Objects

- Handle events
- Coordinate, sequence and execute actions between other objects
- Control the flow

Boundary Objects

- Interface between the system and external actors or services
- Manage input/output with the outside world

4. ROLE OF MODELS IN SOFTWARE DESIGN

- Help organize & understand the design process
- Simplify & break down complex problems
- Ensure key qualities: reusability, flexibility, maintainability
- Serve as design documentation & can map to skeletal code (e.g., Java)

5. UNIFIED MODELLING LANGUAGE (UML)

UML is a visual notation used to create software models.

Different diagrams focus on different issues.

UML Diagram Categories

Structural Models

- Describe the static structure of the system
- Focus on object roles, classes, relationships

Behavioral Models

- Describe how objects interact and behave
- Focus on flow of control, events, use cases

6. ANALOGY IN ARCHITECTURE



Just as architects use scale models to visualize and communicate building designs, **UML models help us visualize, analyse, and communicate software designs.** Models are **abstract** representations that guide effective construction.

Creating Models in Design

- ✓ Don’t jump directly into coding → first understand requirements & design properly
- ✓ Design step bridges **requirements → product**
- ✓ Iterative process → deals with **problem space** & **solution space**
- ✓ Design must be understandable for both **users** & **developers** (common terms)
- ✓ **Object-Oriented Approach helps** describe concepts as **objects** (understood by both users & developers)
- ✓ Popular for solving **complex problems**
- ✓ Two levels of design:
  - ✓ **Conceptual Design** → identify key objects, break problem into parts
  - ✓ **Technical Design** → refine object details (attributes & behaviors) for implementation





Creating Models in Design

- ✓ **Types of Objects in Design Models**
  - **Entity Objects** → focus on problem space
  - **Control Objects** → handle events & coordinate actions
  - **Boundary Objects** → connect system to outside services
- ✓ **Role of Models in Software Design**
  - Help **organize & understand** design process
  - Simplify & break down complex problems
  - Ensure qualities: **reusability, flexibility, maintainability**
  - Serve as **design documentation** & can map to skeletal code (e.g., Java)
- ✓ **Unified Modelling Language (UML)**
  - Visual notation for software models
  - Different UML diagrams focus on different issues:
    - **Structural Models** → describe object roles & relationships
    - Analogy: like a **scale model in architecture**

Dhaka Urban Management System (DUMS)

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Core data objects from the problem space)

Represent real-world things or information that the system manages.

Citizen



- citizenId
- name
- address
- phone
- email

Road



- roadId
- name
- length
- width
- location

Waste Bin



- binid
- location
- capacity
- status
- lastCollectedTime

Building



- buildingId
- type
- address
- floorCount
- ownerId

Traffic Signal



- signalId
- location
- currentState
- cycleTime

2 CONTROL OBJECTS

(Coordinators, logic that controls the flow of events)

Handle requests, make decisions and coordinate between entity objects.

Complaint Controller



- receiveComplaint()
- validateComplaint()
- assignToDepartment()
- updateStatus()

Waste Collection Controller



- scheduleCollection()
- assignWorker()
- updateBinStatus()
- generateReport()

Traffic Management Controller



- monitorTraffic()
- adjustSignalTiming()
- detectCongestion()
- alertAuthority()

Permit Management Controller



- receiveApplication()
- verifyDocuments()
- approveOrReject()
- issuePermit()

3 BOUNDARY OBJECTS

(Interfaces connecting the system with external actors/services)

Manage communication between the system and the outside world.

Mobile App Interface



- sendRequest()
- displayResponse()
- userLogin()

→ Citizens

Web Portal Interface



- submitApplication()
- viewStatus()
- downloadReport()

→ Businesses & Organizations

Sensor / IoT Interface



- receiveSensorData()
- sendCommands()
- deviceStatus()

→ Sensors & IoT Devices

Payment Gateway Interface



- processPayment()
- validateTransaction()
- sendReceipt()

→ Banks / Payment Services

External Agency Interface



- exchangeData()
- receiveUpdates()
- sendReports()

→ Govt. Agencies (Police, DCC, etc.)

How They Work Together

Boundary Objects — Receive requests/data from external actors

→ **Control Objects** — Process requests, make decisions, coordinate

→ **Entity Objects** — Create, update, or retrieve real-world data

→ **Boundary Objects** — Send responses or results back to external actors

Example Scenario

A citizen reports a garbage bin overflow via mobile app (boundary) → Complaint Controller processes it (control) → Waste Bin entity is updated (entity) → Status is sent back to citizen (boundary).

Creating Models in Design: Example

1. Entity Objects (core data objects from the problem space) DUMS

These represent **things in the real world** that your system needs to keep track of.

- ✓ **House Management** → *House*, *Resident*, *Water Bill*, *Electricity Bill*, *Maintenance Request*
- ✓ **Garbage Management** → *GarbageBin*, *WasteTruck*, *CollectionSchedule*, *DumpingSite*
- ✓ **Traffic Management** → *TrafficLight*, *Vehicle*, *RoadSegment*, *AccidentReport*, *CongestionLevel*

2. Control Objects (coordinators, logic that controls the flow of events)

These objects **process requests, make decisions, and coordinate between entity objects**.

- ☐ ***Example:*** House(id, address, owner, status) or GarbageBin(id, location, capacity, lastCollectedTime)
- ✓ **House Management** → *BillCalculator*, *MaintenanceManager*,*HouseAllocationController*
- ✓ **Garbage Management** → *CollectionScheduler*, *RouteOptimizer*, *GarbageMonitoringControlle*
- ✓ **Traffic Management** → *SignalController*, *TrafficFlowManager*, *EmergencyResponseCoordina*
- ☐ ***Example:*** CollectionScheduler checks all GarbageBin fill levels and sends a task to a WasteTruc

3. Boundary Objects (interfaces connecting the system with external actors/services)

These are **interaction points** with users, devices, or external services.

- ✓ **House Management** → *ResidentAppUI*, *BillingPortal*, *SMSNotificationService*
- ✓ **Garbage Management** → *TruckGPSInterface*, *IoT Sensor in GarbageBin*, *CitizenComplaintPortal*
- ✓ **Traffic Management** → *DriverMobileApp*, *CCTV Feed Interface*, *PoliceDashboard*, *TrafficL.H*

💡 *Example:* A *CitizenComplaintPortal* lets residents report uncollected garbage, which is then sen to the CollectionScheduler.

Creating Models in Design: Example

Iterative & Continuous Design

- ✓ **First iteration:** define *entity objects* like House, GarbageBin, TrafficLight.
- ✓ **Next iteration:** design *control objects* like BillCalculator, RouteOptimizer, SignalController to manage them.
- ✓ **Then:** refine *boundary objects* like mobile apps, web portals, and IoT sensors that connect the outside world to the system.
- ✓ This cycle continues — as Dhaka grows, new **entities** (e.g., metro stations), new **control logic** (e.g., AI-based traffic prediction), and new **boundary services** (e.g., chatbot for citizens) can be added.

1. Entity Objects (real-world data and resources in the farm) (AAS)

These represent **things being managed** in the farm:

- ✓ **Crop** (type, growth stage, health status)
- ✓ **Soil** (moisture level, pH, nutrient level)
- ✓ **IrrigationPump** (status, capacity)
- ✓ **WeatherData** (temperature, humidity, rainfall)
- ✓ **FertilizerTank** (amount, type)
- ✓ **PestData** (detection, severity)

💡 *Example*: SoilSensor(id, location, moisture, phLevel)

Agricultural Automation System (AAS) – Design Models

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Real-world data and resources in the farm)

These represent things being managed in the farm.

Example:
SoilSensor(id, location, moisture, pHLevel)

Crop

- Type
- Growth stage
- Health status

Soil

- Moisture level
- pH level
- Nutrient level

IrrigationPump

- Status
- Capacity
- Location

WeatherData

- Temperature
- Humidity
- Rainfall

FertilizerTank

- Amount
- Type
- Location

PestData

- Detection
- Severity
- Location

2 CONTROL OBJECTS

(Decision-making and process logic)

These coordinate farming tasks based on rules, data, or AI models.

Example:
IrrigationController reads SoilSensor data and activates IrrigationPump if moisture < threshold.

Irrigation Controller

- Decides when & how much to irrigate
- Uses soil moisture, weather & crop data

Fertilizer Scheduler

- Calculatos fertilizer needs
- Based on crop & soil data

Greenhouse Climate Controller

- Controls fan, heater, ventilation
- Maintains temp., humidity

Pest Detection Manager

- Detects pest from sensor/drone data
- Triggers pesticide spraying

Harvest Planner

- Determines best harvest time
- Uses crop maturity & weather forecast

3 BOUNDARY OBJECTS

(Interfaces with users and external services)

These allow interaction between system, farmers, and external tools.

Example:
A FarmerMobileApp shows soil moisture levels and allows manual override of IrrigationController.

Farmer Mobile App

- Monitor crops, soil data, irrigation status
- Manual control & overide

→ Farmers

Weather API Interface


- Get weather forecast
- Temperature, rain, humidity, wind

→ Weather Services

Drone Camera Interface

- Capture crop images
- Detect stress, pests, nutrient deficiency

→ Drones / Image Analysis Service

IoT Sensor Gateway

- Collect data from soil, water, crop sensors
- Send to system

→ Sensors & Field Devices

SMS / Alert Service


- Send alerts about irrigation, pests, low fertilizer, etc.

→ Farmers / Operators

ITERATIVE & CONTINUOUS DESIGN PROCESS

Iteration 1
Model Entity Objects
Identify and model key real-world objects (e.g., Crop, SoilSensor, IrrigationPump)

→

Iteration 2
Design Control Objects
Create logic and rules to manage entities (e.g., IrrigationController, PestDetectionManager)

→

Iteration 3
Design Boundary Objects
Develop interfaces for users and external systems (e.g., FarmerApp, WeatherAPI, DroneInterface)

→

Iteration n
Refine & Optimize
Evaluate, test, and improve models continuously based on feedback and new data

Repeat and Refine for Better Farm Automation

Dhaka Urban Management System (DUMS)

Entity Objects, Control Objects & Boundary Objects

1 ENTITY OBJECTS

(Core data objects from the problem space)

Represent real-world things or information that the system manages.

Citizen
- citizenId
- name
- address
- phone
- email

Road
- roadId
- name
- length
- width
- location

Waste Bin
- binId
- location
- capacity
- status
- lastCollectedTime

Building
- buildingId
- type
- address
- floorCount
- ownerId

Traffic Signal
- signalId
- location
- currentState
- cycleTime

2 CONTROL OBJECTS

(Coordinators, logic that controls the flow of events)

Handle requests, make decisions and coordinate between entity objects.

Complaint Controller
- receiveComplaint()
- validateComplaint()
- assignToDepartment()
- updateStatus()

Waste Collection Controller
- scheduleCollection()
- assignWorker()
- updateBinStatus()
- generateReport()

Traffic Management Controller
- monitorTraffic()
- adjustSignalTiming()
- detectCongestion()
- alertAuthority()

Permit Management Controller
- receiveApplication()
- verifyDocuments()
- approveOrReject()
- issuePermit()

3 BOUNDARY OBJECTS

(Interfaces connecting the system with external actors/services)

Manage communication between the system and the outside world.

Mobile App Interface
- sendRequest()
- displayResponse()
- userLogin()

→ Citizens

Web Portal Interface
- submitApplication()
- viewStatus()
- downloadReport()

→ Businesses & Organizations

Sensor / IoT Interface
- receiveSensorData()
- sendCommands()
- deviceStatus()

→ Sensors & IoT Devices

Payment Gateway Interface
- processPayment()
- validateTransaction()
- sendReceipt()

→ Banks / Payment Services

External Agency Interface
- exchangeData()
- receiveUpdates()
- sendReports()

→ Govt. Agencies (Police, DCC, etc.)

HOW THEY WORK TOGETHER

Boundary Objects — Receive requests/data from external actors

→ **Control Objects** — Process requests, make decisions, coordinate

→ **Entity Objects** — Create, update, or retrieve real-world data

→ **Boundary Objects** — Send responses or results back to external actors

EXAMPLE SCENARIO

A citizen reports a garbage bin overflow via mobile app (boundary) → Complaint Controller processes it (control) → Waste Bin entity is updated (entity) → Status is sent back to citizen (boundary).

4 ITERATIVE & CONTINUOUS DESIGN

Iteration 1: Define Entity Objects

Identify and define core real-world objects.

House, GarbageBin, TrafficLight, ...

Iteration 2: Design Control Objects

Design the logic and controllers to manage them.

BillCalculator, RouteOptimizer, SignalController, ...

Iteration 3: Refine Boundary Objects

Develop interfaces that connect external users and devices to the system.

Mobile App, Web Portal, IoT Sensor, ...

Continuous Evolution

This cycle continues — as Dhaka grows, new entities (e.g., metro stations), new control logic (e.g., AI-based traffic prediction), and new boundary services (e.g., chatbot for citizens) can be added.

Metro Station, AI Traffic Prediction, Citizen Chatbot, ...

Creating Models in Design: Example

2. Control Objects (decision-making and process logic)

These **coordinate farming tasks** based on rules, data, or AI models:

- ✓ **IrrigationController** → decides when and how much to irrigate crops
- ✓ **FertilizerScheduler** → calculates fertilizer requirements based on crop and soil data
- ✓ **GreenhouseClimateController** → controls fan, heater, and humidity
- ✓ **PestDetectionManager** → triggers pesticide spraying if pest threshold is crossed
- ✓ **HarvestPlanner** → determines best harvesting time based on crop maturity & weather forecast

💡 *Example*: IrrigationController reads SoilSensor data and activates IrrigationPump if moisture < threshold.

3. Boundary Objects (interfaces with users and external services)

These allow **interaction between system, farmers, and external tools**:

- ✓ **FarmerMobileApp** → lets farmer monitor crops, soil data, and irrigation status
- ✓ **WeatherAPIInterface** → gets weather forecasts from an external service
- ✓ **DroneCameraInterface** → captures crop health images
- ✓ **IoTSensorGateway** → collects data from soil, water, and crop sensors
- ✓ **SMS/AlertService** → sends alerts about irrigation, pest outbreak, or low fertilizer

💡 *Example*: A *FarmerMobileApp* shows soil moisture levels and allows the farmer to manually override the IrrigationController.

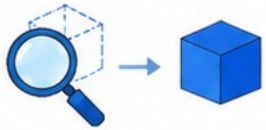
(Iterative & Continuous Design)

- ✓ **First iteration:** model *entities* (Crop, SoilSensor, IrrigationPump)
- ✓ **Next iteration:** design *control objects* (IrrigationController, PestDetectionManager) to manage
- ✓ **Then:** refine *boundary objects* (FarmerApp, DroneInterface, WeatherAPI) for user/system inter

Four Key Principles of OOP

1 Abstraction

Show essential features, hide unnecessary details



Focus on what matters, ignore what doesn't.

2 Encpasulation

Bundle data and methods together, hide internal state



Controlled access through methods

3 Decomposition

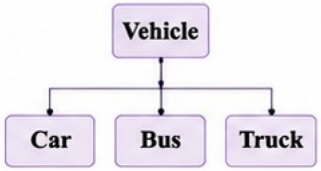
Break a complex system into smaller objects/modules



Easier to design, build and maintain

4 Generalization

Capture common features and create hierarchies



Reuse common structure and behavior

Abstraction: Focus on Essentials, Hide Complexity

What is Abstraction?

- Hides complex implementation details and shows only the essential features of an object.
- Simplifies concepts by focusing on what is important.
- Ignores irrelevant characteristics → follows rule of least astonishment.

Example: Lion



Essential Attributes
- Age
- Species
- Weight

Essential Rehaviors
- Hunting
- Eating
- Sleeping

Benefits of Abstraction

- Simplifies class design
- Makes code more focused, understandable & maintainable
- Adaptable when purpose or problems change
- Simplifies complex concepts to essentials within a context

How Abstraction is Implemented

- Implemented via classes; objects are instances of a class.
- Context matters: abstraction depends on software purpose (e.g., gamer vs. athlete).
- Applied at design level using UML class diagrams.
- CRC cards capture components, later refined into classes/functions.

Abstraction Example 1: Dhaka Urban Management System (DUMS)

Step 1: Identify Real-World Concepts

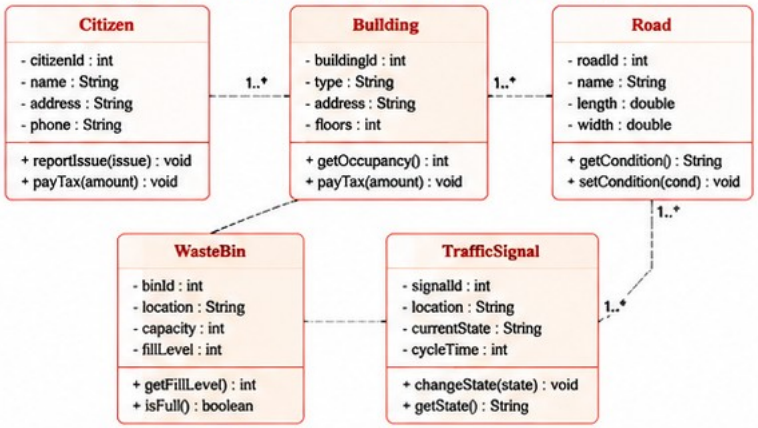
Citizen	People living in Dhaka city
Building	Residential / Commercial building
Road	City roads and streets
Waste Bin	Garbage bins placed around the city
Traffic Signal	Signals controlling traffic flow

Step 2: Capture with CRC Cards

Citizen	Building	Road
Responsbilities Report issue Pay tax Follow rules	**Responsbilities** Provide space Consume utilities Pay tax	**Responsbilities** Enable movement Connect areas Require maintenance
Collaborators Authority, Service	**Collaborators** Citizen, UtilityDept	**Collaborators** TransportDept, Citizen

Waste Bin	Traffic Signal
Responsbilities - Collect waste Indicate fill level	**Responsbilities** - Control traffic Ensure safety
Collaborators WasteDept	**Collaborators** TrafficDept, Road

Step 3: Refine into Classes (UML Class Diagram)



Key Takeaway

Abstraction helps us model real-world complexity by focusing on essential attributes and behaviors of objects.

Identify real-world concepts → Capture using CRC cards → Refine into classes → Relate classes in UML → Implement in code

Evolution of Programming Languages

- Language = A system for communicating thoughts and ideas with each other.
- ✓ System for **communicating ideas** (speaking, writing, gestures, etc.)
- ✓ Languages must **evolve** to stay alive and useful
- ✓ Programming languages evolve for:
  - Solving new problems
  - Offering more effective solutions
  - Supporting new data structures
- ✓ Evolution of ideas → shifts in **programming paradigms**
- ✓ **Top-Down Programming (Example)**
  - **Design strategy** for solving **data-processing problems**
  - Maps processes → routines (starting from the "top")
  - Expressed as a **tree of routines**
  - Implemented in languages supporting **subroutines**





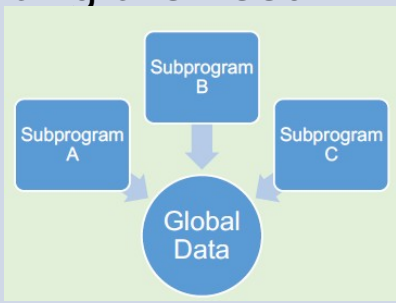
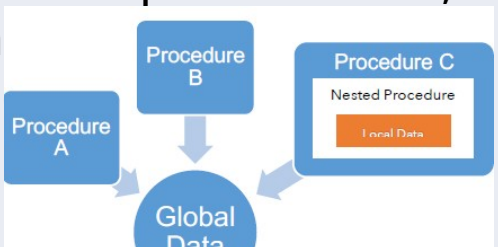
Evolution of Programming Languages

- **Why Study the History of Programming Paradigms?**
  - Older systems still use **legacy languages & paradigms**
  - Object-oriented programming ≠ best solution for all problems
  - New languages may:
    - Expand on existing structures
    - Modify older paradigms
  - Understanding history helps:
    - Recognize “old in new” paradigms
    - Apply the most **efficient paradigm** for each problem

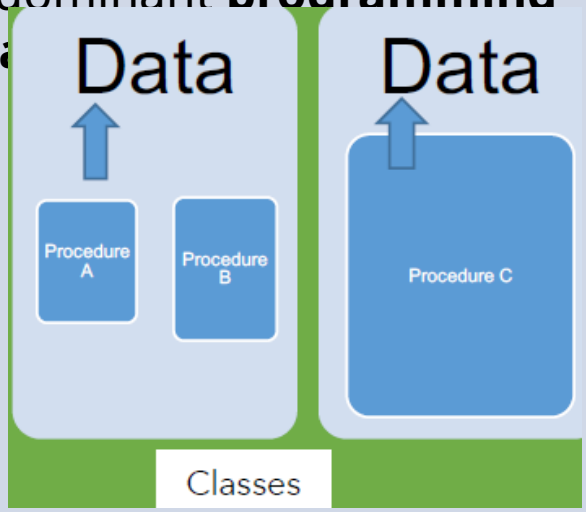




Evolution of Programming Languages

Programming Language and Time period	Solutions afforded by Programming Language	**Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data management.**
COBOL **Fortran** 1960s	COBOL and Fortran employed a centralized global data model that stored information in a single shared memory location, reducing the overhead of passing individual copies through subroutines and enabling faster, more direct access on costly hardware.	Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data 
Algol 68 **Pascal** Early 1970s	**Data Management in the 1960s** Global data widely used → changes caused issues for subroutines Solution: **Scopes & local variables** → each subroutine/procedure had its own data. **Abstract Data Types**	By the mid-1970s, cheaper computer processing and expensive human labor led to larger, more complex software, making sin maintenan 

Evolution of Programming Languages

Programming Language and Time period	Solutions afforded by Programming Language	**Uncontrolled global data changes cause unpredictable errors, highlighting the need for better data management.**
Object-Oriented Programming (Java, C++, C#, etc.) 1980s to present	**Rise of Object-Oriented Design** Abstract data types improved program management, but **no inheritance existed** Object-oriented design emerged to solve this limitation **Objectives of Object-Oriented Design:** Simplify writing abstract data types Structure systems around **classes** (abstract data types) Enable **inheritance** → one data type can extend another	Object oriented programming is the predominant **programming para** 

Evolution of Programming Languages

1. COBOL (Business-oriented, verbose English-like, 1960s)

Designed for **business data processing**. DUMS Code looks like English.

```
IDENTIFICATION DIVISION.
PROGRAM-ID. GARBAGE-MANAGEMENT.
DATA DIVISION.
WORKING-STORAGE SECTION.
01 BIN1 PIC 9(4) VALUE 120.
01 BIN2 PIC 9(4) VALUE 150.
01 BIN3 PIC 9(4) VALUE 100.
01 TOTAL PIC 9(5).

PROCEDURE DIVISION.
    ADD BIN1 TO TOTAL.
    ADD BIN2 TO TOTAL.
    ADD BIN3 TO TOTAL.
    DISPLAY "TOTAL GARBAGE COLLECTED: " TOTAL.
    STOP RUN.
```

```
01 BIN1 PIC 9(4) VALUE 120.   *> can store numbers like 45, 9999, 0
01 BIN2 PIC 9(2) VALUE 12.    *> can store up to 2 digits (00–99)
01 BIN3 PIC X(10) VALUE "Dhaka". *> X means alphanumeric, up to 10 chars
```





Evolution of Programming Languages

2. ALGOL 68 (Algorithmic, structured programming).

```
begin
int bin1 := 120, bin2 := 150, bin3 := 100;
 int total := bin1 + bin2 + bin3;
print(("TOTAL GARBAGE COLLECTED = ", total))

end
```

3. FORTRAN (Scientific/Engineering, 1950s–1960s)

```
PROGRAM GARBAGE
    INTEGER BIN1, BIN2, BIN3, TOTAL
    BIN1 = 120
    BIN2 = 150
    BIN3 = 100
    TOTAL = BIN1 + BIN2 + BIN3
    PRINT *, 'TOTAL GARBAGE
COLLECTED = ', TOTAL
    END
```

4. Pascal (Teaching/Stru Programming, early 1970s)

```
program GarbageManagement;
var
   bin1, bin2, bin3, total: integer;
begin
   bin1 := 120;
   bin2 := 150;
   bin3 := 100;
   total := bin1 + bin2 + bin3;
   writeln('TOTAL GARBAGE COLLECTE
', total);
end.
```

Evolution of Programming Languages

3. JAVA

```
class GarbageBin {
    int weight;
    GarbageBin(int w) { weight = w; }
}

public class DhakaUrbanSystem {
    public static void main(String[] args) {
        GarbageBin b1 = new GarbageBin(120);
        GarbageBin b2 = new GarbageBin(150);
        GarbageBin b3 = new GarbageBin(100);

        int total = b1.weight + b2.weight + b3.weight;
        System.out.println("TOTAL GARBAGE COLLECTED = " + total);
    }
}
```





Evolution of Programming Languages

- ✓ **COBOL** → relies heavily on **global data areas** (like *WORKING-STORAGE SECTION*), which makes large projects difficult to manage (hard to avoid conflicts).
- ✓ **ALGOL 68** → introduced **block structure with scopes** (local vs global variables).
- ✓ **Java / OOP** → solves the "global data" problem by **encapsulation**:
  - Data is stored **inside objects (classes)**
  - Only accessible through **methods** (controlled access)
  - Prevents unwanted global data sharing

- ✓ COBOL → everything is global (messy in big systems).
- ✓ ALGOL 68 → introduced **local scopes**.
- ✓ Java/OOP → goes further by **encapsulating data inside classes** and **restricting access** with modifiers





Evolution of Programming Languages

- **Example: Sum of 10 Numbers in Java (OOP Encapsulation)**

```
class NumberSummer{
        private int[] numbers;   // data is private, not global
        private int total;    // Constructor initializes numbers
        public NumberSummer(int[] nums) {
                numbers = nums;
                 total = 0;    }    // Method to calculate sum

 public int calculateSum() {
                for (int n : numbers) {
                total += n;        }
                 return total;    }}

public class Main {
public static void main(String[] args) {
            int[] nums = {1,2,3,4,5,6,7,8,9,10};
 NumberSummer summer = new NumberSummer(nums);
System.out.println("Sum = " + summer.calculateSum());
}}
```

Four Major Design Principles in OOP

- ✓ Object-oriented programming models objects in a system
- ✓ Four key principles: **Abstraction, Encapsulation, Decomposition, Generalization**



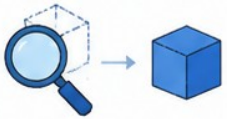


Object-Oriented Programming: Modeling Objects in a System

Four Key Principles of OOP

1 Abstraction

Show essential features, hide unnecessary details



Focus on what matters, ignore what doesn't.

2 Encapsulation

Bundle data and methods together, hide internal state



Controlled access through methods

3 Decomposition

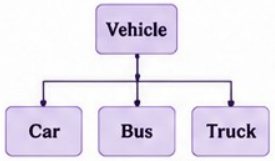
Break a complex system into smaller objects/modules



Easier to design, build and maintain

4 Generalization

Capture common features and create hierarchies



Reuse common structure and behavior

Abstraction: Focus on Essentials, Hide Complexity

What is Abstraction?

- Hides complex implementation details and shows only the essential features of an object.
- Simplifies concepts by focusing on what is important.
- Ignores irrelevant characteristics → follows rule of least astonishment.

Example: Lion



Essential Attributes
- Age
- Species
- Weight

Essential Behaviors
- Hunting
- Eating
- Sleeping

Benefits of Abstraction

- Simplifies class design
- Makes code more focused, understandable & maintainable
- Adaptable when purpose or problems change
- Simplifies complex concepts to essentials within a context

How Abstraction is Implemented

- Implemented via classes; objects are instances of a class.
- Context matters: abstraction depends on software purpose (e.g., gamer vs. athlete).
- Applied at design level using UML class diagrams.
- CRC cards capture components, later refined into classes/functions.

Abstraction Example 1: Dhaka Urban Management System (DUMS)

Step 1: Identify Real-World Concepts

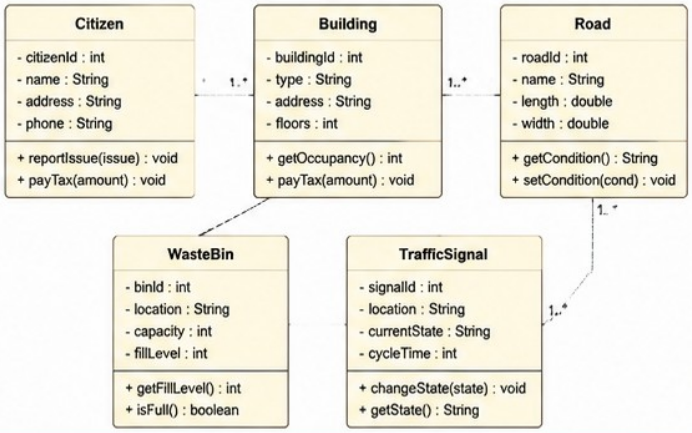
Citizen	People living in Dhaka city
Building	Residential / Commercial building
Road	City roads and streets
Waste Bin	Garbage bins placed around the city
Traffic Signal	Signals controlling traffic flow

Step 2: Capture with CRC Cards

Citizen	Building	Road
Responsbilities Report issue Pay tax Follow rules	Responsbilities Provide space Consume utilities Pay tax	Responsbilities Enable movement Connect areas Require maintenance
Collaborators Authority, Service	Collaborators Citizen, UtilityDept	Collaborators TransportDept, Citizen

Waste Bin	Traffic Signal
Responsbilities Collect waste Indicate fill level	Responsbilities Control traffic Ensure safety
Collaborators WasteDept	Collaborators TrafficDept, Road

Step 3: Refine into Classes (UML Class Diagram)



Key Takeaway

Abstraction helps us model real-world complexity by focusing on essential attributes and behaviors of objects.

Identify real-world concepts → Capture using CRC cards → Refine into classes → Relate classes in UML → Implement in code

Abstraction

- Abstraction is the process of hiding complex implementation details and showing only the essential features of an object.
- Simplifies complex concepts by focusing on **essential details**
- Ignores irrelevant characteristics → follows **rule of least astonishment**
- Implemented via **classes**; objects are instances of a class
- **Context matters**: abstraction depends on software purpose (e.g., gamer vs. athlete)
- Captures two main aspects:
  - **Basic attributes** (e.g., age of a lion)
  - **Basic behaviors/responsibilities** (e.g., hunting, eating, sleeping)
- Benefits:
  - Simplifies class design
  - Makes code more **focused, understandable, and maintainable**
  - Adaptable when system purpose or problem changes





Abstraction

- ✓ Hiding complex reality while exposing only essentials

```
// Abstract class example
abstract class Vehicle { abstract void startEngine(); // Abstract method (no implementation)
 public void honk() { // Concrete method
System.out.println("Beep beep!"); } }

class Car extends Vehicle { @Override void startEngine()
 { System.out.println("Car engine started with key"); } }

 class ElectricCar extends Vehicle { @Override void startEngine()
{ System.out.println("Electric car started with button"); } }
```





Abstraction

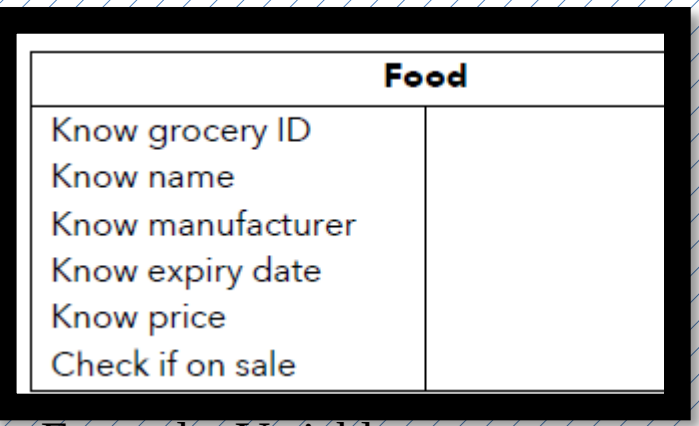
- Simplifies complex concepts to essentials within a context
- Applied at the design level using **UML class diagrams**
- CRC cards capture components, later refined into classes or functions
- Each concept/class represented as a **box with three sections**:
  - **Class Name** – same as Java class name
  - **Properties** – equivalent to Java's member variables
    - Format: <variable name> : <variable type>
    - Types can be classes or primitives
  - **Operations** – equivalent to Java's methods
    - Format: <method>(<parameter list>) : <return type>

- **Mapping**:
  - Class Diagram → Java Class
  - Properties → Member Variables
  - Operations → Methods

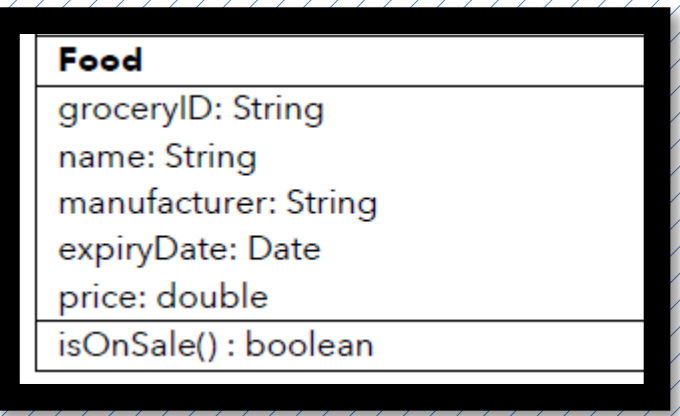




Abstraction



Example: Variables



Example: Class Diagrams

- ✓ **Example: Food Item (Grocery Store Context)**
- ✓ **Class Name**: Food
- ✓ **Properties**: groceryID, name, manufacturer, expiryDate, price
- ✓ **Operation**: isOnSale(date: Date): Boolean
  - ✓ Returns **true/false** depending on sale status
- ✓ **CRC vs. Class Diagram**
  - ➢ CRC: lists responsibilities together
  - ➢ Class Diagram: separates **properties vs. operations**
  - ➢ Easier translation into code for programmers

```
public class Food {
   public String groceryID;
   public String name;
   public String manufacturer;
   public Date expiryDate;
   public double price;
   public boolean isOnSale(Date date) {
      // Implementation
   }}
```

Abstraction

Abstraction Example 1: Dhaka Urban Management System (DUMS)

- ✓ Suppose we want to model **Garbage Bins** in DUMS: We don't need to care about *sensor chip design* or *truck engine details* → we just need **essential attributes and behaviors**.

```
abstract class GarbageBin {
    // Essential attributes
    protected String location;
    protected int capacity;
    protected int currentLevel;

    // Essential behaviors
    abstract void collectGarbage();
    abstract boolean isFull();
}
```

```
// Concrete class hides details of how collection works
class SmartGarbageBin extends GarbageBin {
    @Override
    void collectGarbage() {
        currentLevel = 0; // reset after collection
        System.out.println("Garbage collected at " +
location);
    }
    @Override
    boolean isFull() {
        return currentLevel >= capacity;
    }}
```

- **Abstraction here:**
  - ✓ Shows *location*, *capacity*, and *basic behavior (collectGarbage, isFull)*.
  - ✓ Hides *how sensors detect fullness, how trucks are dispatched, how waste is processed*.

Abstraction

Abstraction Example 2: Agricultural Automation System (AAS)

✓ Suppose we want to model an **Irrigation System**.

```
// Abstract class for irrigation
abstract class IrrigationSystem {
    protected String fieldId;
    protected int waterLevel;

    // Essential behaviors
    abstract void irrigate();
    abstract boolean
needsIrrigation();
}
```

```
// Implementation hides details of pump operation
class SmartIrrigationSystem extends IrrigationSystem
{
    @Override
    void irrigate() {
        waterLevel = 100; // restore soil moisture
        System.out.println("Field " + fieldId + "
irrigated.");
    }
    @Override
    boolean needsIrrigation() {
        return waterLevel < 40; // below threshold
    }
}
```

Abstraction here:

✓ Shows fieldId, waterLevel, irrigate, needsIrrigation.Hides how soil sensors measure moisture, how water is pumped, how scheduling is optimized..

Abstraction

- **Benefits in Both Systems**
  - **Simplifies design** → Developers think only about key features.
  - **Maintainability** → If the *garbage collection* logic or *pump mechanism* changes, we don't touch the abstraction.
  - **Adaptability** → Can extend the system (e.g., add AI-based optimization) without changing high-level abstractions.
- In **DUMS**, abstraction means thinking of *GarbageBin* with only essential features (location, capacity, collect).
- In **AAS**, abstraction means thinking of *IrrigationSystem* with only essential features (field, water level, irrigate).

Abstraction Table: DUMS vs AAS

System	Object	Essential Attributes (What it *is*)	Essential Behaviors (What it *does*)
DUMS (Dhaka Urban Management System)	**House**	Address, Owner, Occupants, UtilityStatus	PayBill(), RequestMaintenance(), ReportIssue()
	GarbageBin	Location, Capacity, CurrentLevel	collectGarbage(), isFull(), sendAlert()
	TrafficLight	Location, SignalState (Red/Green/Yellow), Timer	changeSignal(), detectTrafficFlow(), alertEmergency()
AAS (Agricultural Automation System)	**Soil**	Location, MoistureLevel, pH, Nutrients	measureMoisture(), checkFertility()
	Crop	Type, GrowthStage, HealthStatus	grow(), harvest(), detectDisease()
	IrrigationSystem	FieldId, WaterLevel, Status	irrigate(), needsIrrigation(), scheduleWatering()

Encapsulation in Object-Oriented Programming

Encapsulation is the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit (a class).

1. WHAT IS ENCAPSULATION?

- Encapsulation = bundling data (attributes) and methods (behaviors) in a class.
- Controls access to data:
  - Expose what is needed via interfaces
  - Restrict internal data/methods to the object itself
- Ensures each object has its own data values and behaviors.

Class (Blueprint)

Data (Attributes)

gpa, degreeProgram, studentId, name, ...

Methods (Behaviors)

getGPA(), setGPA(), calculateCGPA(), updateProgram(), ...

Objects (Instances)

student1

gpa = 3.80
program = CSE
id = 101 ...

student2

gpa = 3.40
program = EEE
id = 102 ...

The class bundles data and methods together.
Each object has its own copy of data.

2. BENEFITS OF ENCAPSULATION

Integrity & Security
Protects sensitive data and prevents unintended or illegal changes.

Changeable Implementation
Internal code can change without affecting the interface.

Black Box Concept
Users interact with the class through methods, without knowing internal details.

Modularity & Reusability
Simplifies code management and promotes modular design.

Bundles data + functions into a self-contained object, allows controlled access, and restricts direct manipulation of internal data.

3. ENCAPSULATION IN UML (EXAMPLE: STUDENT CLASS)

Student

Attributes (Data)
- gpa : double
- degreeProgram : String
- studentId : int
- name : String

Methods (Operations)
+ getGPA() : double
+ setGPA(gpa : double) : void
+ getProgram() : String
+ setProgram(p : String) : void

Visibility Indicators
- **+ (public)** → accessible from outside the class
- **- (private)** → accessible only within the class

Key Points
- Attributes are private (-) so they cannot be accessed directly from outside.
- Methods are public (+) to provide controlled access to the data.

The private attributes (-) are hidden from outside. Only public methods (+) can access or modify them.

4. GETTER & SETTER METHODS

Getter Methods (retrieve private data)

Format:
get<AttributeName>()

Example:
getGPA() : double

Setter Methods (update private data safely)

Format:
set<AttributeName>(value)

Example:
setGPA(double gpa) : void

Getter & Setter methods ensure approved access and modification of data, preserving integrity and security.

SUMMARY: HOW ENCAPSULATION WORKS

Data is hidden (private) inside the class. → Public methods provide controlled access. → Users interact through methods (interface) only. → Data integrity is maintained and system is safe. → Code becomes modular, maintainable and reusable.

Encapsulation in Object-Oriented Programming

- Encapsulation is the bundling of data (attributes) and the methods (functions) that operate on that data into a single unit (a class).
- Encapsulation = bundling **data (attributes)** and **methods (behaviors)** in a class
- Controls access to data:
  - **Expose** what is needed via interfaces
  - **Restrict** internal data/methods to the object itself
- Ensures **each object has its own data values and behaviors**
- **Benefits of Encapsulation**
  - **Integrity & Security** protects sensitive data, prevents unintended changes
  - **Changeable Implementation**: internal code can change without affecting the interface
  - **Black Box Concept**: users interact with the class through methods, without knowing internal details
  - **Modularity & Reusability** simplifies code management, promotes modular design





Encapsulation in Object-Oriented Programming UML

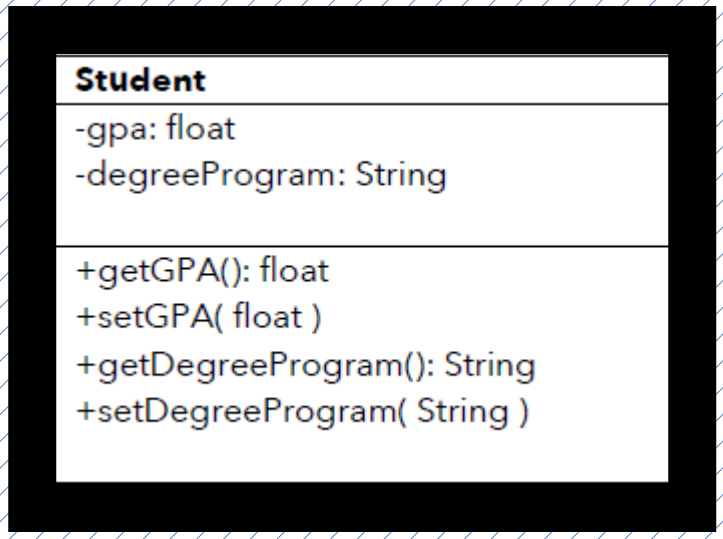
- Bundles **data + functions** into a self-contained object
- Allows **controlled access** to data and functions
- Restricts direct manipulation of internal data
- Expressed in **UML class diagrams** through attributes + methods
- **Encapsulation in UML:**
  - Class diagram **bundles attributes & operations**
  - Visibility indicators:
    - **+ (public)** → accessible from outside the class
    - **- (private)** → accessible only within the class
  - Example: Student class
    - Attributes: - gpa, - degreeProgram (private)
    - Methods: + getGPA(), + setGPA() (public)
- **Benefits of Encapsulation**
  - Controls **how data is accessed/modified**
  - Prevents **unexpected direct changes**
  - Preserves **data integrity**





Encapsulation in Object-Oriented Programming UML

- **Getter & Setter Methods**
  - ✓ **Getter methods**: retrieve private data
    - ✓ Format: get<AttributeName>()
    - ✓ Example: getGPA()
  - ✓ **Setter methods**: safely update private data
    - ✓ Format: set<AttributeName>(value)
    - ✓ Example: setGPA(double gpa)
  - ✓ Ensure **approved access & modification**



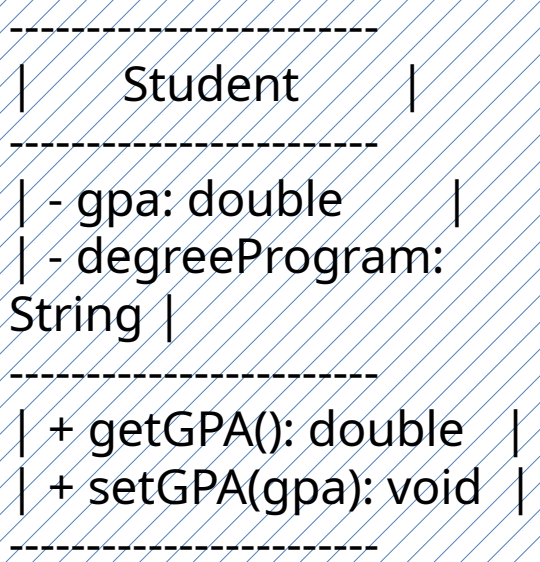
Example: UML Class Diagram for Studen

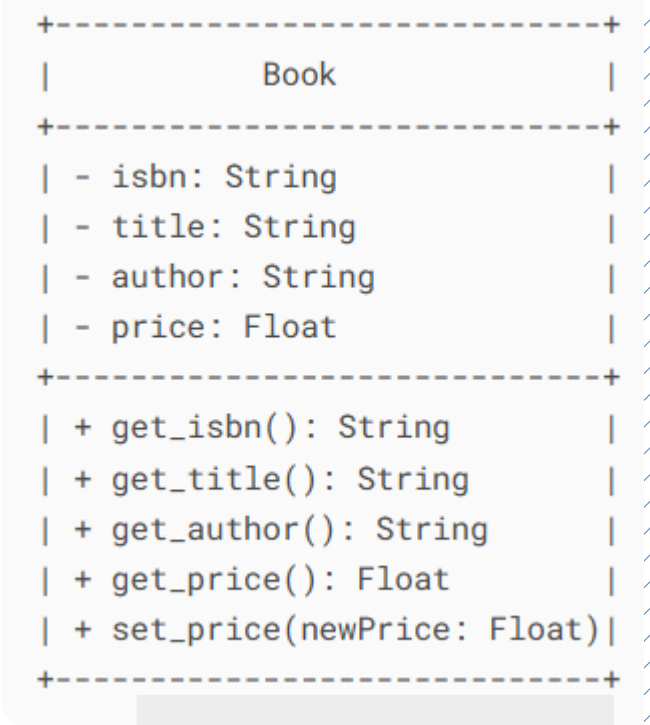
```
public class Student {
    // Private attributes (hidden data)
    private double gpa;
    private String degreeProgram;

    // Public methods (controlled
access)
    public double getGPA() {
        return gpa;     }
    public void setGPA(double gpa) {
        if (gpa >= 0 && gpa <= 4.0)
    {  // validation
            this.gpa = gpa;
        }
    }
}
```

Encapsulation

UML Visualization:
The full class diagram shows methods and their visibility.





The - denotes private (encapsulated) attributes, and + denotes public methods that form the interface.

Encapsulation: DUMS-Example 1: Garbage Collection

```
public class GarbageBin {
    // Private attributes (hidden data)
    private double fillLevel;     // percentage (0–100)
    private String location;

    // Public methods (controlled access)
    public double getFillLevel() {
        return fillLevel;
    }

    public void updateFillLevel(double level) {
        if (level >= 0 && level <= 100) {
            this.fillLevel = level;
        }
    }

    public String getLocation() {
        return location;
    }
}
```

- ✓ **Encapsulation role:** Prevents outsiders from setting fillLevel to invalid values (e.g., >100%). Only controlled updates are allowed.





Encapsulation: DUMS-Example 2: Traffic Signal Management

```
public class TrafficSignal {
    private String currentColor;    // "Red", "Yellow", "Green"
    private int timer;              // seconds remaining

    public String getCurrentColor() {
        return currentColor;
    }

    public void changeSignal(String color) {
        if (color.equals("Red") || color.equals("Yellow") || color.equals("Green"))
        {
            this.currentColor = color;
        }
    }
}
```

- ✓ **Encapsulation role:** Restricts invalid signals (e.g., "Blue"). Ensures system integrity in city traffic management.





Encapsulation: AAS-Example 1: Soil Sensor

```
public class SoilSensor {
    private double moistureLevel;  // %
    private double pH;             // soil pH value

    public double getMoistureLevel() {
        return moistureLevel;
    }

    public void setMoistureLevel(double level) {
        if (level >= 0 && level <= 100) {
            this.moistureLevel = level;
        }
    }

    public double getPH() {
        return pH;
    }

    public void setPH(double pH) {
        if (pH >= 0 && pH <= 14) {
            this.pH = pH;
        }
    }
}
```

- ✓ **Encapsulation role:** Protects against invalid sensor values (e.g., negative moisture, impossible pH > 14).

Encapsulation: AAS-Example-2 Irrigation Pump

```
public class IrrigationPump {
    private boolean isOn;
    private int flowRate;   // liters/minute

    public void startPump() {
        isOn = true;
    }

    public void stopPump() {
        isOn = false;
    }

    public void setFlowRate(int rate) {
        if (rate >= 0 && rate <= 100) {
            this.flowRate = rate;
        }
    }

    public boolean getPumpStatus() {
        return isOn;
    }
}
```

- ✓ **Encapsulation role:** Prevents unsafe flow rates and provides safe on/off control.
- ✓ In **DUMS**, encapsulation ensures **public safety & system consistency** (e.g., garbage bins, traffic signals).
- ✓ In **AAS**, encapsulation ensures **data integrity & safe automation** (e.g., soil readings, irrigation controls).

Decomposition in Object-Oriented Design

- ✓ **Definition:** Dividing a whole into smaller parts or combining parts to form a whole
- ✓ **Purpose:** Break problems into manageable pieces, each with a specific responsibility
- ✓ **Relation to Classes:** Each part can be represented by a **class**; the whole object relates to its part classes
- ✓ **Similarity to Abstraction:** Focuses on essential characteristics of each part





Definition: Dividing a whole into smaller parts or combining parts to form a whole.
Purpose: Break problems into manageable pieces, each with a specific responsibility.

1. NATURE OF PARTS

- **Fixed vs. Dynamic Parts**

Fixed: constant number of parts.
(e.g., oven with 4 burners)

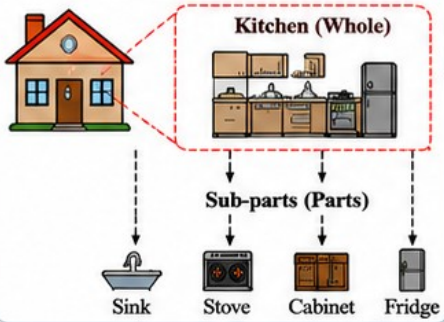


Dynamic: number of parts can change over time.
(e.g., food items in a fridge)



2. PARTS AS WHOLES

A part can itself be a whole with sub-parts.
Example: Kitchen in a House



3. LIFETIMES OF PARTS

Share lifetime with the whole
(e.g., fridge gauge)

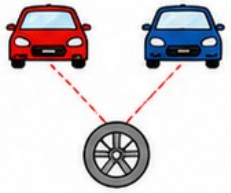


Have independent lifetime
(e.g., food items)



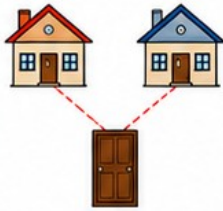
4. SHARED PARTS

Some parts may be shared between wholes.



Wheel (shared)

Some parts cannot be shared.



Door (not shared)

5. RELATION TO CLASSES

Each part can be represented by a class; the whole object relates to its part classes.

5. FOCUS: SIMILARITY TO ABSTRACTION

Focuses on essential characteristics of each part.

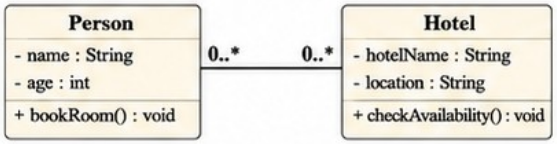
BENEFITS OF DECOMPOSITION

- ✓ Divides a whole into parts (or combines parts into a whole)
- ✓ Helps organize system functionality
- ✓ Improves reusability, maintainability
- ✓ and easier understanding

THREE RELATIONSHIP TYPES IN UML

1. ASSOCIATION (LOOSE RELATIONSHIP)

- Objects interact but are independent.
- If one is destroyed, the other still exists.
- Example: Person ↔ Hotel



A person can interact with many hotels.
A hotel can interact with many people.



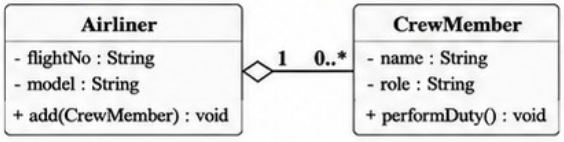
Java Example

```
class Person {
   private String name;
   public void bookRoom(Hotel h) { /* ... */ }
}

class Hotel {
   private String hotelName;
}
```

2. AGGREGATION (HAS-A RELATIONSHIP, WEAK)

- Whole has parts, but parts can exist independently.
- Example: Airliner ↔ Crew Members





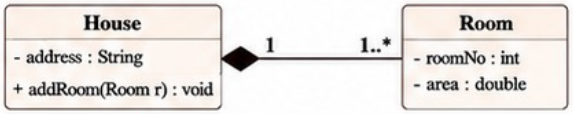
Airliner has a crew.
Crew members exist even if not in airliner.

Java Example

```
public class Airliner {
   private ArrayList<CrewMember> crew = new ArrayList<>();
   public void add(CrewMember crewMember) {
      crew.add(crewMember);
   }
```

3. COMPOSITION (HAS-A RELATIONSHIP, STRONG)

- Whole owns its parts; parts cannot exist without the whole.
- If the whole is destroyed, its parts are also destroyed.
- Example: House ↔ Room





Rooms are part of the house and cannot exist without it.

Java Example

```
public class House {
   private ArrayList<Room> rooms = new ArrayList<>();
   public void addRoom(Room room) {
      rooms.add(room);
   }
```

Key Takeaway: Decomposition breaks a system into meaningful parts, each modeled as a class. Relationships

Decomposition in Object-Oriented Design

- **Nature of Parts**
- **Fixed vs. Dynamic Parts:**
  - Fixed: constant number of parts (e.g., oven with 4 burners)
  - Dynamic: number of parts can change over time (e.g., food items in a fridge)
- **Parts as Wholes:** A part can itself be a whole with sub-parts (e.g., kitchen in a house)
- **Lifetimes:**
  - Some parts share lifetime with the whole (e.g., fridge gauge)
  - Some parts have independent lifetimes (e.g., food items)
- **Shared Parts:** Some parts may be shared between wholes; others cannot





Decomposition in Design

- ✓ Divides a whole into parts (or combines parts into a whole)
- ✓ Helps organize system functionality
- ✓ Three relationship types in UML:
  - ➢ Association
  - ➢ Aggregation
  - ➢ Composition

- ◆ **Association (Loose Relationship)**
- ✓ Objects interact but are **independent**
- ✓ If one is destroyed, the other still exists
- ✓ Example: **Person ↔ Hotel**
  - ✓ A person can interact with many hotels
  - ✓ A hotel can interact with many people
- ✓ UML: straight line with **0..*** multiplicity
- ✓ Java Example:

```
public class Student {
    public void play(Sport sport) {
        execute.play(sport);
    }
}
```

Example: Student and Sport: separate loosely connected



Decomposition in Design

- **Aggregation (Has-a Relationship, Weak)**
  - ✓ Whole has parts, but **parts can exist independently**
  - ✓ Example: **Airliner ↔ Crew Members**
    - ✓ Airliner has a crew
    - ✓ Crew members exist even if not in airliner
  - ✓ UML: empty diamond (◊) at the whole
  - ✓ Java Example:

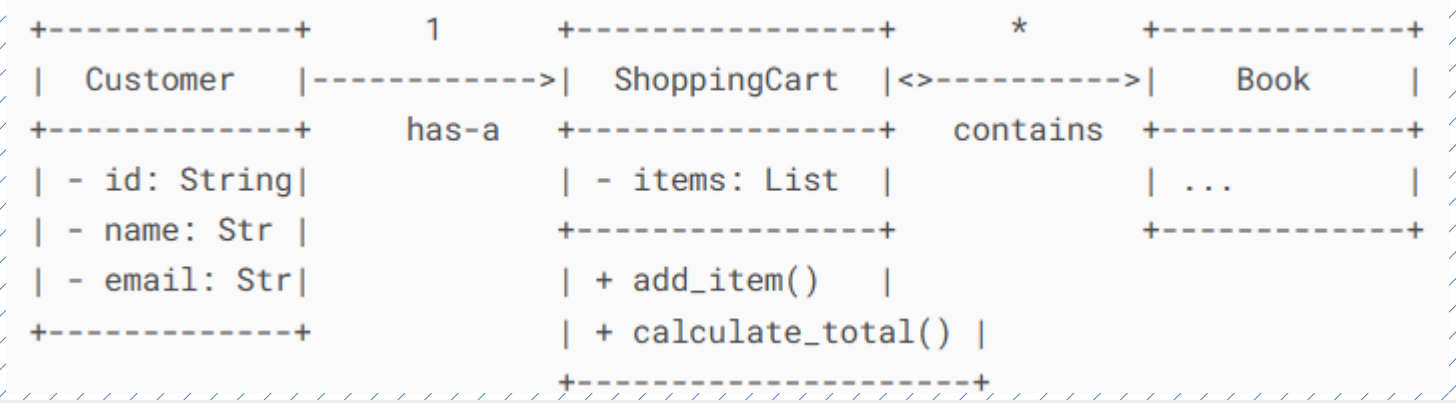
```
public class Airliner {
    private ArrayList<CrewMember> crew = new
ArrayList<>();
    public void add(CrewMember crewMember) {
        crew.add(crewMember);
    }
}
```

Example: Airliner can have zero or more crew members



Decomposition

- ✓ **UML Visualization (Class Diagram with Relationships):**
  This shows the decomposed classes and their relationships.



*The diagram uses association (has-a), aggregation (<>), and multiplicities (1, *) to show how the decomposed parts relate.*

Composition (Strong Has-a Relationship)

- **Composition (Strong Has-a Relationship)**
  - **Exclusive containment** of parts (strongest form of decomposition)
  - Whole **cannot exist without its parts**
  - If whole is destroyed → parts are destroyed too
  - Parts are **tightly bound** and accessed only through the whole

- **Example: House ↔ Room**
  - A **House** is made up of **Rooms**
  - If the House is removed, Rooms no longer exist
  - UML: **filled diamond (◆)** at the whole (House)
  - Multiplicity: 1..* (House must have one or more Rooms)

Example:

```
public class House {
    private Room room;

    public House() {
        room = new Room();
    }
}
```



- **Room object** created when House is created
- Room cannot exist independently → **tightly coupled**

Example: Composition can be represented with a filled-in diamond using UML class diagrams.

Generalization (Inheritance)

- ✓ **Purpose:** Reduces redundancy by capturing shared characteristics in a **parent class** (superclass)
- ✓ **Mechanism:**
  - ➢ Inheritance allows **child classes** (subclasses) to reuse attributes and behaviors from parent classes
  - ➢ Child classes can add specialized attributes or behaviors
- ✓ **Benefits:**
  - ➢ Saves time and prevents errors
  - ➢ Makes software more **flexible, maintainable, and reusable**
  - ➢ Changes in common code only need to be made in the superclass
  - ➢ Supports easy expansion of software with new subclasses

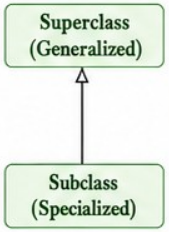




Purpose: Reduces redundancy by capturing shared characteristics in a parent class (superclass).

1. WHAT IS GENERALIZATION?

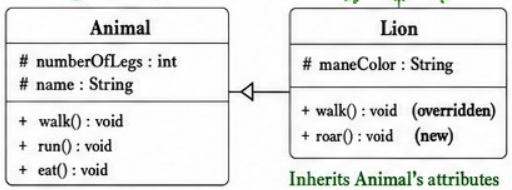
- Factors out shared characteristics into superclass.
- Supports code reuse and avoids redundancy.
- Subclasses inherit superclass' attributes & methods.
- Represented in UML with solid line + arrow (↑).
- Superclass at arrowhead (top), subclass at tail (bottom).



2. EXAMPLE: ANIMAL ↔ LION

Superclass: Animal (parent) (generalized)

Subclass: Lion (specialized)



Declared abstract → cannot be instantiated.

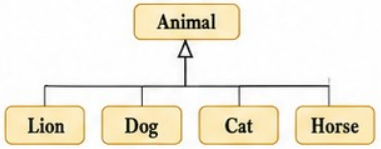
Inherits Animal's attributes & behaviours and adds specialized behaviour.

3. BENEFITS OF GENERALIZATION

- Saves time and prevents errors through code reuse.
- Makes software more flexible, maintainable, and reusable.
- Changes in common code only need to be made in the superclass.
- Supports easy expansion of software with new subclasses.

4. UML VISUALIZATION (INHERITANCE HIERARCHY)

UML uses an arrow with a solid line to represent inheritance (generalization).



Animal is the superclass.
Lion, Dog, Cat, Horse are subclasses.

5. CONSTRUCTORS IN INHERITANCE

Implicit Constructor (default)

Auto-assigned default values (to 0, false, null, etc.)

```
class Animal {
    int numberOfLegs;   // 0
    String name;        // null
}
```

Explicit Constructor (initialization)

Initialize attributes during object creation.

```
class Animal {
    int numberOfLegs;
    String name;

    Animal(int legs, String n) {
        numberOfLegs = legs;
        name = n;
    } }
```

Subclass must call superclass constructor using super().

```
class Lion extends Animal {
    String maneColor;

    Lion(int legs, String n,
         String color) {
        super(legs, n);   // call
        maneColor = color;
    }
}
```

6. METHOD OVERRIDING

Subclass can redefine (superclass) methods.

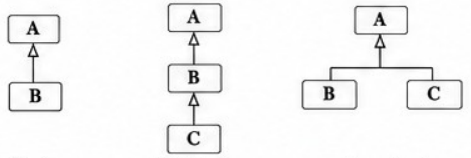
```
class Animal {
    void walk() {
        System.out.println
          ("I walk");
    }
}
```

```
class Lion extends Animal {
    @Override
    void walk() {
        System.out.println
          ("I'd rather nap");
        roar(legs, n);   // call
    }
```

Allows specialized behaviour while keeping inheritance structure.

7. TYPES OF INHERITANCE IN JAVA

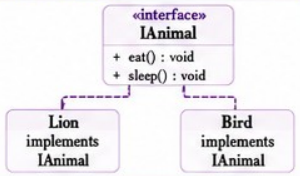
- Java supports single implementation inheritance (using extends).



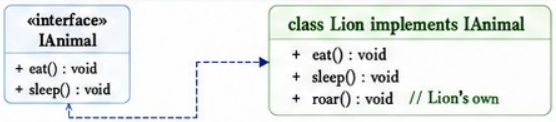
- A superclass can have multiple subclasses.
- Subclasses can also be superclasses (multi-level inheritance).

8. INTERFACE INHERITANCE

- Java alternative to multiple inheritance.
- Declares method signatures only (no attributes or method bodies).
- Implemented using implements keyword.
- UML: interface shown with «interface» and dotted line + hollow arrow (↑).



9. EXAMPLE: LION IMPLEMENTS IANIMAL



- ✓ Lion behaves like a Lion and like an IAnimal.
- ✓ Multiple interfaces can be implemented → no ambiguity.

10. ADVANTAGES OF INTERFACES

- Enable polymorphism (same behaviour, different implementation).
- Allow multiple inheritance of type without ambiguity.
- Promote flexibility & reuse in system design.

11. DESIGN PROCESS IN OOP

Conceptual Design

Prototyping & high-level simulation

Focus on what the system should do.

Technical Design

Detailed guidance for developers

Focus on how the system will be built.

Iterative Process: Refine and Improve

12. DESIGN TOOLS

Conceptual Tool: CRC Cards

- Easy communication with clients.
- Focus on design without writing code.

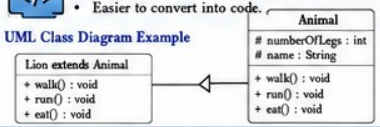
CRC Card Example

Class Name	Responsibilities	Collaborators
Lion	Roar, Eat, Sleep	Animal, Zookeeper

Technical Tool: UML Class Diagrams

- More detailed than CRC cards.
- Easier to convert into code.

UML Class Diagram Example



13. INTEGRATION WITH OOP PRINCIPLES

Abstraction Focus on essential features of objects.

Encapsulation Bundle data & methods, control access.

Decomposition Break system into manageable parts.

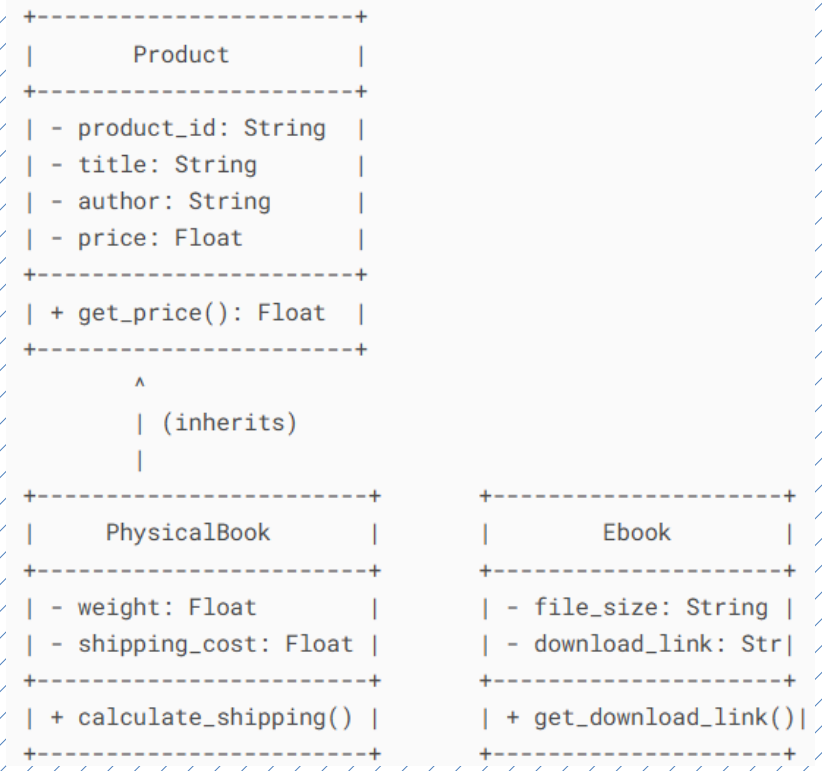
Generalization (Shtyping) Reuse & extend existing classes.



Key Takeaway: Generalization (Inheritance) promotes code reuse and extensibility by modeling "is-a" relationships between classes. It works together with Abstraction, Encapsulation, and Decomposition to build robust, maintainable, and scalable systems.

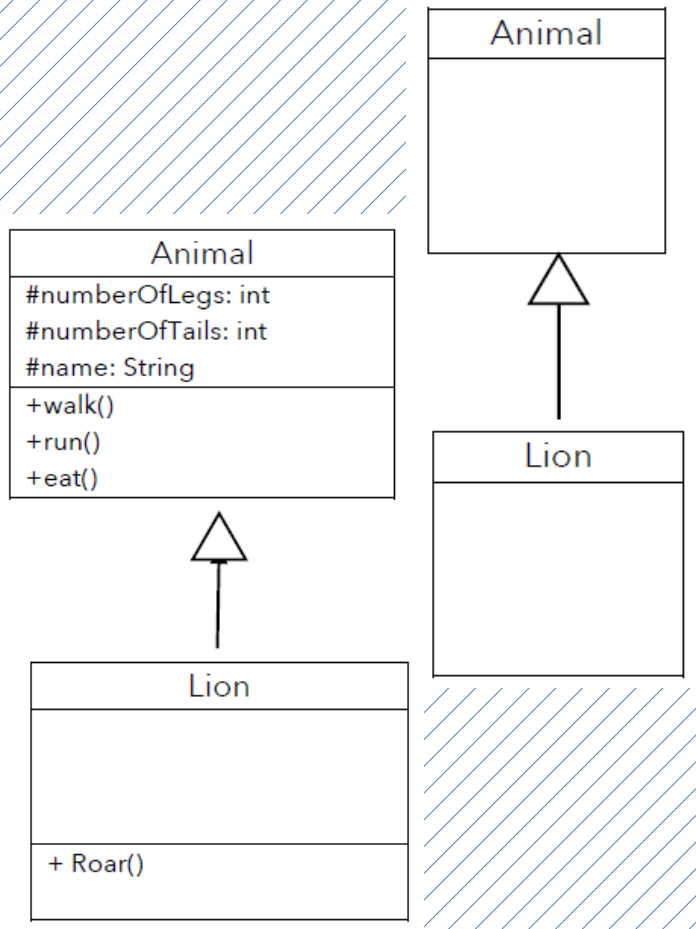
Generalization (Inheritance)

- **UML Visualization (Inheritance Hierarchy):**
  UML uses an arrow with a solid line to represent inheritance (generalization).



Generalization (Inheritance)

- ✓ Factors out **shared characteristics** into a superclass
- ✓ Supports **code reuse** and avoids redundancy
- ✓ Subclasses inherit superclass' attributes & methods
- ✓ Represented in UML with **solid line + arrow (↑)**
  - ➢ **Superclass** at arrowhead (top)
  - ➢ **Subclass** at tail (bottom)
- ✓ **Example: Animal ↔ Lion**
- ✓ **Superclass**: Animal (generalized)
  - ➢ Attributes: #numberOfLegs, #name, etc.
  - ➢ Methods: walk(), run(), eat()
  - ➢ Declared **abstract** → cannot be instantiated
- ✓ **Subclass**: Lion (specialized)
  - ➢ Inherits Animal's attributes & behaviours
  - ➢ Adds specialized behaviour → roar()



Generalization (Inheritance)

- ✓ **Constructors in Inheritance**
  - **Implicit constructor**: auto-assigned defaults (0, null)
  - **Explicit constructor**: initialize attributes during object creation
  - Subclass must call superclass constructor using **super()**
- ✓ **Method Overriding**
  - Subclass can **redefine superclass methods**

  Example:
  public void walk() {
  System.out.println("I'd rather nap"); } Allows specialized behaviour while keeping inheritance structure

- ✓ **Types of Inheritance in Java**
  - **Single implementation inheritance** only (using extends)
  - A superclass can have multiple subclasses
  - Subclasses can also be superclasses (multi-level inheritance)





Generalization (Inheritance)

- **Interface Inheritance**
  - Java alternative to multiple inheritance
  - Declares **method signatures only** (no attributes or bodies)
  - Implemented using **implements keyword**
  - UML: interface shown with «interface» + **dotted arrow (↑)**

- Lion behaves like a Lion **and** like an IAnimal
- Multiple interfaces can be implemented → no ambiguity
- **Advantages of Interfaces**
  - Enable **polymorphism** (same behaviour, different implementation)
  - Allow **multiple inheritance of type** without ambiguity
  - Promote **flexibility & reuse** in system design

```
public interface IAnimal {
    void move();
    void speak();
    void eat();
}
public class Lion implements IAnimal {
    public void move() { … }
    public void speak()
{ System.out.println("Roar!"); }
    public void eat() { … }
}
```

Example: Animal Inheritance

Design Structure in Java and UML Class Diagrams

- **Design Process:**
  - Conceptual design → prototyping & high-level simulation
  - Technical design → detailed guidance for developers
- **Conceptual Tools:**
  - CRC cards → easy communication with clients, focus on design without code
- **Technical Tools:**
  - UML Class Diagrams → more detailed than CRC cards, easier conversion to code
- **Integration with OOP Principles:**
  - Abstraction, Encapsulation, Decomposition, Generalization applied in Java and UML





Example

```
// Demonstrate method overloading.
class OverloadDemo {
  void test() {
    System.out.println("No parameters");
  }

  // Overload test for one integer parameter.
  void test(int a) {
    System.out.println("a: " + a);
  }

  // Overload test for two integer parameters.
  void test(int a, int b) {
    System.out.println("a and b: " + a + " " + b);
  }

  // overload test for a double parameter
  double test(double a) {
    System.out.println("double a: " + a);
    return a*a;
  }
}
```

```
class Overload {
  public static void main(String args[]) {
    OverloadDemo ob = new OverloadDemo();
    double result;

    // call all versions of test()
    ob.test();
    ob.test(10);
    ob.test(10, 20);
    result = ob.test(123.25);
    System.out.println("Result of ob.test(123.25): " + result);
  }
}
```

Access Control

- Java's access specifiers are **public**, **private**, and **protected**
- Java

Access Levels

Modifier	Class	Package	Subclass	World
`public`	Y	Y	Y	Y
`protected`	Y	Y	Y	N
no modifier	Y	Y	N	N
`private`	Y	N	N	N

Thank You Q/A

Part A: Object-Oriented Design

Design Principles



Abu Saleh Musa Miah (PhD)

Presentation Outline

- ✓ **Design Principle**
- ✓ Evaluating Design Complexity

Design Principles

- ✓ Provides **guidelines for evaluating software structure**
- ✓ Ensures software is **flexible, reusable, and maintainable**
- ✓ Covers **behavior modeling** using:
  - ➢ **UML State Diagrams**
  - ➢ **UML Sequence Diagrams**

◆ Evaluating Design Complexity

- ✓ **Keep modules simple** to avoid unnecessary complexity
- ✓ Excessive complexity → **harder to understand** → more **bugs**
- ✓ **Evaluation of design complexity** is essential for quality assurance
- ✓ Complexity applies to:
  - ➢ **Classes**
  - ➢ **Methods** within classes
- ✓ In this module, **"module" = program unit containing classes and methods**





Design Principles

1. WHAT ARE DESIGN PRINCIPLES?

- ✓ Provides guidelines for evaluating software structure
- ✓ Ensures software is flexible, reusable, and maintainable
- ✓ Covers behavior modeling using:
  - ➢ UML State Diagrams
  - ➢ UML Sequence Diagrams



2. EVALUATING DESIGN COMPLEXITY

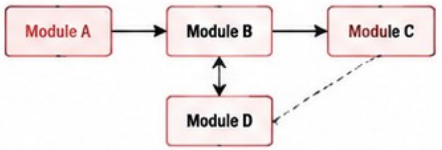
- ✓ Keep modules simple to avoid unnecessary complexity
- ✓ Excessive complexity → harder to understand → more bugs
- ✓ Evaluation of design complexity is essential for quality assurance
- ✓ Complexity applies to:
  - ➢ Classes
  - ➢ Methods within classes
- ✓ In this module, "module" = program unit containing classes and methods
- ✓ System = combination of modules



3. GOOD DESIGN vs BAD DESIGN (MODULE CONNECTIONS)

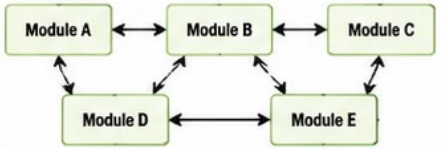
Bad Design: ✗

- Modules can connect only to specific modules
- Limits flexibility and reuse



Good Design: ✓

- Modules are compatible with each other
- Can be easily connected and re-used



4. COUPLING (Interdependence Between Modules)

Definition: Measures complexity between a module and other modules

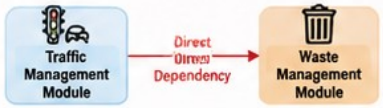
Types of Coupling:

- ➢ **Tight Coupling** (Bad Design) – module heavily depends on others
- ➢ **Loose Coupling** (Good Design) – module connects via well-defined interfaces

4A. TIGHT vs LOOSE COUPLING EXAMPLE (SMART CITY)

Tight Coupling (Bad Design / High Coupling)

Traffic Management Module directly depends on Waste Management Module to update road availability.



- If the waste system fails, the traffic system also fails.
- This makes the system rigid and hard to maintain.

VS

Loose Coupling (Good Design / Low Coupling)

Traffic Management Module reads from a shared Service (e.g., City Resource Service) for road availability.



- If waste management fails, traffic still works with last data.
- Modules can evolve independently and be reused in other cities.

4B. METRICS TO EVALUATE COUPLING

Degree – Number of connections
- Fewer connections → looser coupling

Ease – Clarity of connections
- Should not require understanding internal details of other modules

Flexibility – Interchangeability
- Modules should be easily replaceable

4C. SIGNS OF TIGHT COUPLING (BAD DESIGN)

- ⚠ Too many parameters or interfaces in connections
- ⚠ Difficulty finding corresponding modules
- ⚠ Module can only connect to specific modules (not interchangeable)

4C. COHESION (How Well a Module Works Internally)

Definition: Focuses on complexity within a module

Types of Cohesion:

- ➢ High Cohesion (Good Design) – module has a clear, single purpose
- ➢ Low Cohesion (Bad Design) – module has multiple or unclear responsibilities

5A. HIGH COHESION (GOOD DESIGN)



Water Supply Module
- Pipeline Monitoring
- Supply Scheduling
- Water Quality Checking
- Billing

- ✓ Preforms one task only (Water related)
- ✓ Purpose is clear and easy to understand
- ✓ Easy to maintain and extend
- ✓ (e.g., adding smart water sensors)

5B. LOW COHESION (BAD DESIGN)



Water Supply Module
- Water Distribution
- Traffic Control
- Garbage Collection

- ✗ Multiple unrelated tasks in one module
- ✗ Purpose is unclear → harder to maintain
- ✗ Encapsulation must be broken to understand

6. BALANCING COHESION & COUPLING

Goal: Low Coupling + High Cohesion

Trade-offs in Complex Systems:

- ➢ Increasing cohesion may increase coupling
- ➢ Reducing coupling may reduce cohesion



7. EXAMPLE SUMMARY

Bad Design (Low Cohesion & Tight Coupling)



- ✗ Modules depend on each other
- ✗ Mixed responsibilities
- ✗ Hard to test, maintain, and reuse
- ✗ System becomes rigid

Good Design (High Cohesion & Loose Coupling)



- ✓ Modules are independent
- ✓ Single responsibility
- ✓ Easy to maintain and extend
- ✓ System is flexible and reusable

Key Takeaway: Design good software by keeping modules highly cohesive and loosely coupled. This leads to maintainable, flexible, reusable, and reliable systems.

System Design & Module Interaction

- **System = combination of modules**
- **Bad design**:
  - Modules can connect only to specific modules
  - Limits flexibility and reuse
- **Good design**:
  - Modules are **compatible** with each other
  - Can be **easily connected** and **re-used**
- **Evaluating Design Complexity**
  - **Coupling** (Interdependence between modules)– degree of interdependence between modules
  - **Cohesion** (How well a single module works internally)– degree to which elements within a module work together

Coupling in Software Design

- **Definition**: Measures complexity between a module and other modules
- **Types of Coupling**:
  - **Tight Coupling (Bad Design)** – module heavily depends on others
  - **Loose Coupling (Good Design)** – module connects via well-defined interfaces
- **Metrics to Evaluate Coupling**
- **Degree** – number of connections
  - Fewer connections → **looser coupling**
- **Ease** – clarity of connections
  - Should not require understanding internal details of other modules
- **Flexibility** – interchangeability
  - Modules should be **easily replaceable**
- **Signs of Tight Coupling (Bad Design)**
- Too many parameters or interfaces in connections
- Difficulty finding corresponding modules
- Module can only connect to **specific modules** (not interchangeable)

Coupling (Interdependence between modules)

- **Tight Coupling (Bad Design/ High Coupling):**
  Suppose the **Traffic Management Module** directly depends on the **Waste Management Module** to update road availability. For example, traffic routes can only be updated if the waste collection module first finishes its update.
  - If the waste system fails, the traffic system also fails.
  - This makes the system rigid and hard to maintain.

- **Loose Coupling (Good Design/Low Coupling):**
  The **Traffic Management Module** just reads from a **shared database or service** (e.g., a central "City Resource Service") that contains road availability. The Waste Management Module also updates the same database, but they don't directly depend on each other.
  - If waste management fails, traffic still works with the last available data.
  - Modules can evolve independently and be reused in other cities.

Cohesion in Software Design

- **Definition**: Focuses on complexity **within a module**
- **Types of Cohesion**:
  - **High Cohesion (Good Design)** – module has a **clear, single purpose**
  - **Low Cohesion (Bad Design)** – module has **multiple or unclear responsibilities**
- **Characteristics**
  - **High Cohesion**:
    - Module performs **one task only**
    - Purpose is **clear and easy to understand**
  - **Low Cohesion**:
    - Module performs **multiple unrelated tasks**
    - Purpose is unclear → harder to maintain
    - Encapsulation must be broken to understand

Cohesion in Software Design

- **Balancing Cohesion & Coupling**
  - **Goal**: **Low Coupling + High Cohesion**
  - **Trade-offs in Complex Systems**:
    - Increasing **cohesion** may increase **coupling**
    - Reducing **coupling** may reduce **cohesion**

Cohesion in Software Design

- **Bad Design (Low Cohesion):**
  The **Water Supply Module** not only distributes water but also contains code for traffic control and garbage collection.
  - Responsibilities are mixed.
  - Hard to test, maintain, or reuse.
- **Good Design (High Cohesion):**
  The **Water Supply Module** only handles tasks related to water: pipeline monitoring, supply scheduling, water quality checking, and billing.
  - Single responsibility.
  - Easy to maintain and extend (e.g., adding smart water sensors).

Coupling Cohesion Example-1

Scenario-1: Suppose we have a class called FarmController in the Automated Agricultural System that has two purposes: controlling irrigation and fertilizer dispensing. Here we have a controlSystem() method that takes a zero (0) flag if you want to activate the irrigation system, and takes the one (1) flag if you want to activate the fertilizer system. The class diagram for the FarmController and implementation of its controlSystem() method is as below:

FarmController
+ controlSystem(controlFlag: int)
	public void controlSystem(int controlFlag) {
	switch (controlFlag) {
	case 0:
	return this.startIrrigation();
	case 1:
	return this.startFertilizerDispense();
	default:
	throw new
	UnknownControlFlagException();
	}}

Problem a): Evaluate the above design in terms of **coupling** and **cohesion**. Give justification in each case.

Problem b): Suggest a **better design** for the above class that meets acceptable **coupling** and **cohesion** requirements.

Coupling Cohesion Example-1

a) Evaluation of the Current Design

- **Cohesion: Low Cohesion (Poor Design)-** The FarmController class is responsible for **two unrelated tasks**:
  1. **starting irrigation**, and
  2. **starting fertilizer dispensing**.
- Both actions belong to different subsystems of the agricultural system, yet they are handled in a single method (controlSystem()), which switches behavior via flags. This leads to **low cohesion** because:
  - ✓ The class performs **multiple independent functions**, violating the *Single Responsibility Principle (SRP)*.
  - ✓ The controlSystem() method does **more than one job**, decided by a control flag (0 or 1).
  - ✓ If future systems (e.g., pest-control, greenhouse ventilation) are added, the class must be modified repeatedly, further reducing cohesion.
  - ✓ Thus, the class is **not focused on one responsibility**, making it difficult to maintain and extend.
- **Coupling: Higher Than Necessary (Flag-Based Coupling)-** While the class itself may not depend on many external components, the **internal coupling** is unnecessarily strong:
  - ✓ The class **knows the internal details** of both irrigation and fertilizer systems.
  - ✓ Changing the logic of either subsystem would force modifications inside FarmController.
  - ✓ Using numeric flags (0 and 1) introduces **implicit, error-prone coupling**—only the sender of the flag knows what the values mean.
  - ✓ External components that call controlSystem(flag) must also depend on the meaning of those numbers, creating a hidden dependency.

This design leads to **tighter coupling** and reduces flexibility.

Coupling Cohesion Example-1

b) Suggested Improved Design (Better Cohesion & Lower Coupling)

To achieve a clean design, the responsibilities should be **separated** into their own classes, and FarmController should act only as a **coordinator**.

- **Better Design Structure**

1. IrrigationController Class

- ✓ Handles all irrigation-related operations.
- ✓ Highly cohesive: one responsibility.

2. FertilizerController Class

- ✓ Responsible only for fertilizer dispensing.
- ✓ Highly cohesive.

3 FarmController Class

- ✓ Coordinates actions by calling the appropriate controller.
- ✓ Low coupling: interacts with controllers through clean method calls—not numeric flags.

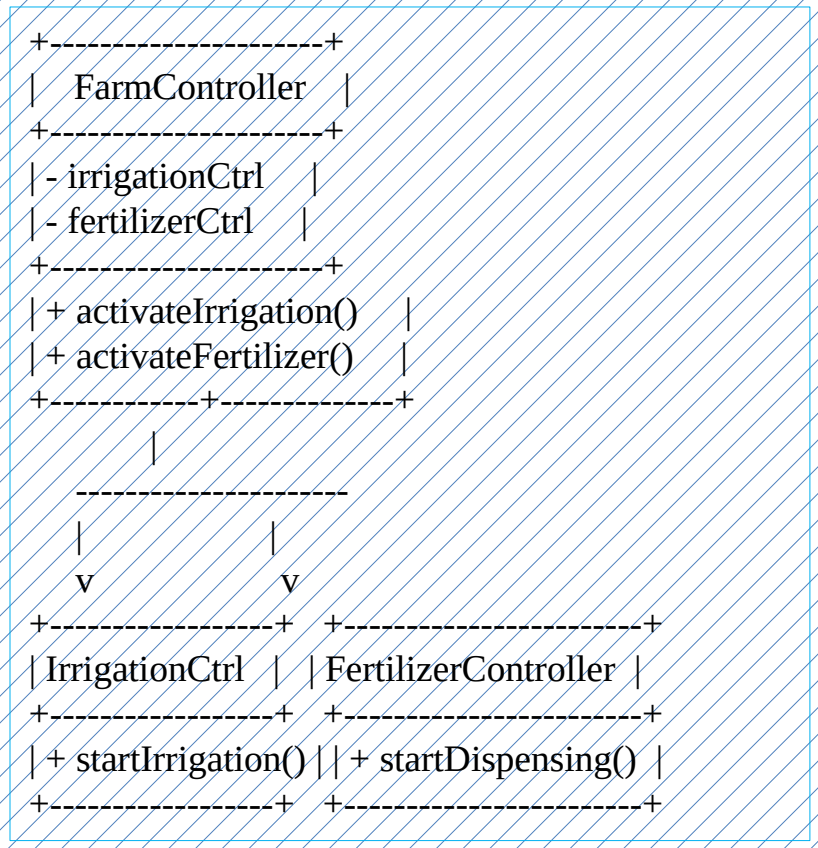


Figure: Improved UML-Style Diagram

Coupling Cohesion Example-1

Improved Code Example

```
public class FarmController {

    private IrrigationController irrigationCtrl;
    private FertilizerController fertilizerCtrl;

    public FarmController() {
        this.irrigationCtrl = new
IrrigationController();
        this.fertilizerCtrl = new
FertilizerController();
    }

    public void activateIrrigation() {
        irrigationCtrl.startIrrigation();
    }

    public void activateFertilizer() {
        fertilizerCtrl.startDispensing();
    }
}
```

Figure: Improved Code

Why This Design Is Better

- **High Cohesion**
  - ✓ Each class has **one dedicated purpose**.
  - ✓ Easier to update and test.
- **Low Coupling**
  - ✓ FarmController does not know internal details of each subsystem.
  - ✓ New systems can be added (e.g., pestControl) by creating new classes without changing existing ones.
- **Readable and Maintainable**
  - ✓ Method names clearly describe actions.
  - ✓ Eliminates confusing flag-based logic.

Advantage of the code: No flags, No switch statements., Clear responsibilities

Coupling Cohesion Example-2

Scenario-2: Suppose we have a class called ServiceManager in the Dhaka Urban Management System that has two purposes: handling waste collection requests and road maintenance requests.

Here we have a processRequest() method that takes a zero (0) flag if we want to process a waste collection request, and takes the one (1) flag if we want to process a road maintenance request. The class diagram for the ServiceManager and implementation of its processRequest() method is as below:

ServiceManager
+ processRequest(controlFlag: int)
	public void processRequest(int controlFlag) {
	switch (controlFlag) {
	case 0:
	return this.handleWasteCollection();
	case 1: return this.handleRoadMaintenance();
	default:
	throw new UnknownControlFlagException(); }}

a) Evaluate the above design in terms of **coupling** and **cohesion**. Give justification in each case.

b) Suggest a **better design** for the above class that meets acceptable **coupling** and **cohesion** requirements.

Separation of Concerns (SoC)

- ✓ One of the design principles examined in the previous module was that of decomposition.
- ✓ Decomposition divides a whole into different parts. To understand why decomposition is necessary in design, the principle of **separation of concerns** must be examined.

◆ **Separation of Concerns (SoC)**

- ✓ **Definition**: Dividing a system into distinct parts, each addressing a separate concern
- ✓ **Concern**: Anything that matters in solving a problem (e.g., data, logic, presentation)
- ✓ **Goal**: Keep concerns **separate and organized** in design and implementation

- ✓ A **concern** is a very general notion: it is anything that matters in providing a solution to a problem. Separation of concerns is about keeping the different concerns in your design separate.
- ✓ When software is designed, different concerns should be addressed in different portions of the software.

Separation of Concerns (SoC)

- One of the design principles examined in the previous module was that of **decomposition**.
- Decomposition divides a whole into different parts. To understand why decomposition is necessary in design, the principle of **separation of concerns** must be examined.

What is Separation of Concerns (SoC)?

Definition: Dividing a system into distinct parts, each addressing a **separate concern**.

Concern: Anything that matters in solving a problem (e.g., **data, logic, presentation**).

Goal: Keep concerns separate and organized in design and **implementation**.

Why is Separation of Concerns Important?

- A **concern** is a very general notion: it is anything that matters in providing a solution to a problem. Separation of concerns is about keeping the different concerns in your design separate.
- When software is designed, different concerns should be addressed in different portions of the software.
- Separation of concerns provides many advantages. They allow us to develop and update sections of the software **independently**.
- Using separation of concerns also means that we **do not need to know** how all sections of code work in order to update a section.
- Finally, separation of concerns allows changes to be made to one component **without requiring a change** in another.
- When addressing concerns separately, **more cohesive classes** are created and the design principles of **abstraction, encapsulation, decomposition**, and **generalization** are enforced.

SoC in Action: Without vs With Separation of Concerns

Without SoC (Tangled Design)

All concerns are mixed in one module.
Hard to understand, maintain, and extend.

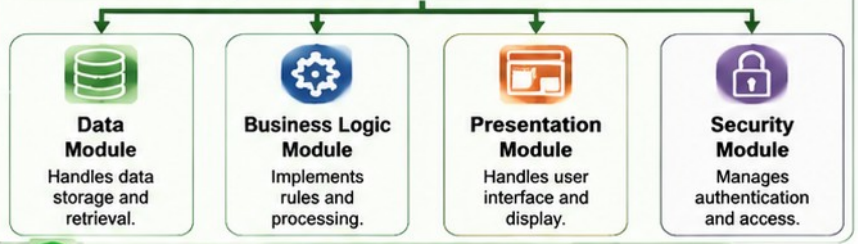


Any change affects the whole system.

VS

With SoC (Modular Design)

Each concern is separated into its own module.
Easy to understand, maintain, and extend.



Changes in one module do not affect others.

Advantages of Separation of Concerns

- Independent development and updates
- No need to know entire system to update one section
- Changes in one component do not force changes in others
- Improves maintainability, modularity, and reusability

How SoC Supports Key Design Principles

Abstraction
Each concept is separated with its own relevant attributes and behaviors.

Encapsulation
Attributes and behaviors are bundled in a class. Implementation can change without affecting the interface.

Decomposition
A whole class can be broken down into multiple smaller classes.

Generalization
Commonalities are identified and moved into a superclass.

SoC leads to clean design, fewer dependencies, easier maintenance, and better software quality.

Separation of Concerns (SoC)

- ✓ **Separation of concerns provides many advantages**. They allow us to develop and update sections of the software independently.
- ✓ Using separation of concerns also means that **we do not need to know how all sections of code work in order to update a section.**
- ✓ Finally, separation of concerns allows changes to be made to one component without requiring a change in another.
- ✓ When addressing concerns separately, more cohesive classes are created and the design principles of abstraction, encapsulation, decomposition, and generalization are enforced

Advantages of SoC

- ✓ **Abstraction occurs as each concept in the problem** space is separated with its own relevant attributes and behaviors.
- ✓ **Encapsulation occurs as the attributes** and behaviors are gathered into their own section of code called a class. Access to the class from the rest of the system and its implementation are separated, so details of implementation can change while the view through an interface can stay the same.

Separation of Concerns (SoC)

Advantages of SoC

- ✓ **Decomposition occurs** as a whole class can be separated into multiple classes.
- ✓ **Generalization occurs** as commonalities are recognized and subsequently separated and generalized into a superclass.
- ✓ Independent **development and updates** of software sections
- ✓ No need-to-know entire system to update one section
- ✓ **Changes in one component** do not force changes in others
- ✓ Improves **maintainability, modularity, and reusability**

Separation of Concerns Example

- **Consider a smartphone capable of many behaviors:**
  - taking photos,
  - scheduling meetings,
  - sending and receiving email,
  - browsing the Internet,
  - sending texts, and
  - making phone calls.
- This example will only focus on two functions **camera and traditional phone** functions.

Separation of Concerns Example

- This example will only focus on two functions camera and traditional phone functions.

```
public class SmartPhone {
private byte camera;
private byte phone;
public SmartPhone() { … }
public void takePhoto() { … }
public void savePhoto() { … }
public void cameraFlash() { … }
public void makePhoneCall() { … }
public void encryptOutgoingSound() { … }
public void decipherIncomingSound() { … } }
```

- This code has a **SmartPhone class with attributes called camera and phone**, and associated behaviors.
- This system has **low cohesion, as there are behaviors that are not related to each other**.
- The camera behaviors do not need to be encapsulated with the behaviors of the phone in order for the camera to do its job.
- The components also do not offer modularity.

Separation of Concerns Example

- For example, it is not possible to access the camera or the phone separately if another system is built that requires only one or the other.
- The camera also would not be able to be replaced with a different camera, or even a different object, without removing the code for the camera completely from this class.
- The SmartPhone class needs to be more cohesive, and each component of the smartphone should have distinctive functionality.
- **Using the separation of concerns, we can identify that the SmartPhone class has two concerns:**
  - To act as a traditional telephone.
  - To take pictures using the built-in camera.
- With the concerns identified, it is possible to separate them into their own more cohesive classes and encapsulate all the details about each concern into functionally distinct and independent classes.

Separation of Concerns Example

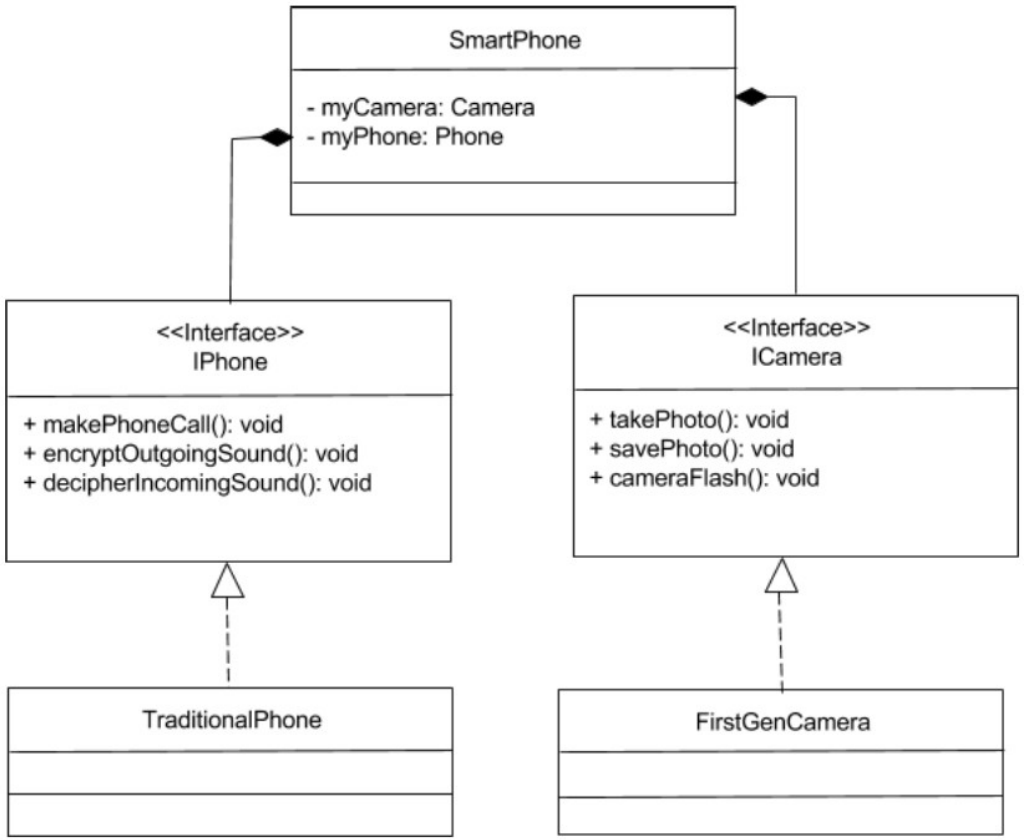
- **Problem: Low Cohesion in SmartPhone Class**
  - SmartPhone class mixes **camera** and **phone** functions
  - **Low cohesion**: unrelated behaviors grouped together
  - **No modularity**: cannot reuse/replace camera or phone independently

- **Solution: Separation of Concerns**
  - Identify two distinct concerns:
    - **Phone functionality**
    - **Camera functionality**
  - Create **separate cohesive classes**: Phone and Camera
  - SmartPhone acts as a **coordinator**, referencing both
  - Improves:
    - **Cohesion** (focused responsibilities)
    - **Modularity** (components can be reused/replaced)

Separation of Concerns Example

- The SmartPhone class will reference instances of the new classes so that the smartphone can act as a coordinator of the camera and the phone.
- This will let our smartphone provide access to all the behaviours of the camera and the phone, without having to know how each component behaves.
- Using a UML Class diagram, this is how the new design for the SmartPhone system might look:

Example: Smartphone System



The code would translate as below:

```
public interface ICamera {
public void takePhoto();
public void savePhoto();
public void cameraFlash();
}
public interface IPhone {
public void makePhoneCall();
public void encryptOutgoingSound();
public void deciphereIncomingSound();
}
public class FirstGenCamera implements
ICamera {
/* Abstracted camera attributes */
public class TraditionalPhone implement
IPhone {
/* Abstracted phone attributes */
}
```

- ✓ The attributes and behaviors for the phone and camera have been separated into two distinct interfaces. These are each implemented with a corresponding class. The code would translate as below:

Example: Smartphone System

The SmartPhone class also need to be redesigned for two separate classes:

```
public class SmartPhone {
private ICamera myCamera;
private IPhone myPhone;
public SmartPhone( ICamera aCamera, IPhone
                        aPhone ) {
                        this.myCamera = aCamera;
                        this.myPhone = aPhone;
                        }
public void useCamera() {
return this.myCamera.takePhoto();
}
public void usePhone() {
return this.myPhone.makePhoneCall();
}
}
```

Example: Smartphone System

- **Redesigned SmartPhone System**
  - SmartPhone aggregates **Phone** and **Camera** classes
  - **Constructor** takes IPhone and ICamera objects as parameters
  - SmartPhone forwards responsibilities to these objects
  - **Who creates the Phone/Camera objects** → separate responsibility
- **Benefits of Redesign**
  - **Modular design** – phone/camera can be swapped without changing SmartPhone code
  - **High cohesion** – Phone and Camera have clear, independent functionality
  - **Flexibility** – supports future upgrades (e.g., new camera type)
- **Trade-off**
  - Increased **coupling**: SmartPhone depends on IPhone and ICamera interfaces
- **Use of Separation of Concerns**
  - **Generalization** – define IPhone and ICamera interfaces
  - **Abstraction & Encapsulation** – implement separate Phone and Camera classes
  - **Decomposition** – split smartphone into distinct, reusable parts

Example: Smartphone System

- **SmartPhone Class Redesign**
  - **Constructor**Integrates **camera** and **phone** functionalities.
  - Keeps **Camera** and **Phone** classes separated (hidden from each other).
  - Both are aggregated under the **SmartPhone** class.
- **Design**
  - SmartPhone constructor accepts a **Camera** and a **Phone** as parameters.
  - New SmartPhone instances can be created by passing in objects implementing **ICamera** and **IPhone** interfaces.
- **Responsibility Separation**
  - Creation of appropriate **Camera** and **Phone** objects is handled outside the SmartPhone class.
  - SmartPhone does not need to know *how* these objects are created.
- **Method Delegation**
  - SmartPhone methods **forward responsibilities** to the aggregated camera and phone objects.

Example: Smartphone System

- **Modular Design Benefits**
  - Camera or Phone classes can be **swapped** without modifying SmartPhone code.
  - Only the instantiation of SmartPhone and its parts needs to change.
- **Cohesion & Coupling**
  - SmartPhone class is now **more cohesive**.
  - However, coupling increases since SmartPhone depends on **Camera** and **Phone** interfaces (indirect dependency).
- **Separation of Concerns Applied**
  - **Generalization** → Separate notions of Camera and Phone via **interfaces**.
  - **Abstraction & Encapsulation** → Define first-gen Camera and traditional Phone as **implementing classes**.
  - **Decomposition** → Break down SmartPhone into **constituent parts** (camera + phone) separated from the whole.

Information Hiding in Software Design

- **Definition:** Modules only access the information needed to do their job
- **Purpose:** Hide details that may change (algorithms, data representations)
- Information hiding is commonly associated with sensitive data —the more sensitive the data, the more likely it should have limited access.
- Assumptions: Exposed via APIs & interfaces
- **Benefits of Information Hiding**
  - Developers can **work independently** on modules
  - Reduces dependencies on **implementation details**
  - **Flexibility**: implementation can change without affecting others
  - **Security**: sensitive data protected
  - **Reliability**: prevents unwanted changes to critical attributes
- **Relation to Encapsulation**
  - **Encapsulation**: bundles attributes & behaviors into a class
  - Provides access via **interfaces**
  - Hides **implementation details** behind method signatures
  - Allows internal changes without breaking external expectations

Information Hiding in Software Design

Definition: Modules only access the information needed to do their job.

Purpose: Hide details that may change (algorithms, data representations).

Information hiding is commonly associated with sensitive data — the more sensitive the data, the more likely it should have limited access.

Assumptions: Exposed via APIs & interfaces

Benefits of Information Hiding

- Developers can work **independently** on modules.
- Reduces dependencies on **implementation details**.
- **Flexibility:** implementation can change without affecting others.
- **Security:** sensitive data protected.
- **Reliability:** prevents unwanted changes to critical attributes.

Relation to Encapsulation

- **Encapsulation:** bundles attributes & behaviors into a class.
- Provides access via **interfaces**.
- Hides **implementation details** behind method signatures.
- Allows internal **changes** without breaking external expectations.

How Information Hiding Works

Other Modules (Clients)

They only know what they can do, not **how** it is done.

Interact through Interface (API)

Knows only method signatures

Interface (API)

- Method A()
- Method B()
- Method C()

Defines **what** can be done.

Hides Implementation Details

Module (Implementation)

- Algorithms
- Data Structures
- Internal Logic

Defines **how** it is done.

Key Ideas & Benefits

- **Purpose:** Conceals changeable details (e.g., algorithms, data structures) to reduce dependency.
- **Assumptions:** Not hidden—explicitly expressed in APIs and interfaces.
- **Key Idea:**
  - Hide what may change → implementation details.
  - Reveal what remains stable → assumptions via interfaces.
- **Benefits:**
  - Enables independent module development.
  - Promotes modularity and maintainability.
- **Relation to Encapsulation:**
  - Encapsulation groups atributes and behaviors into classes.
  - Interfaces provide controlled access, hiding internal implementations.
  - Other modules depend on method signatures, not underlying logic.

Encapsulation & Interfaces

- Implementation is hidden behind an interface.
- Other classes rely on method signatures, not internal logic.
- Behaviour can change without altering expected outcomes.

Why It Matters

- Expectations for a behaviour are fulfilled without exposing *how* it works.
- Enhances flexibility, maintainability, and modularity.
- Protects system from unintended interference.

Hiding Attributes

- Critical attributes should not be modified directly by external classes.
- Protects class integrity and prevents unintended side effects.

Access Modifiers (Control Visibility in Java)

Modifier	Where Accessible	Attributes (Fields)	Methods	Use / Notes
public	Any class in the system	Can be read / modified	Can be invoked	Widest access. Encapsulation still applies.
protected	Same package & subclasses (in any package)	Can be read / modified	Can be invoked	Useful for inheritance and related groups.
default (no modifier)	Same package only	Can be read / modified	Can be invoked	Also called package-private.
private	Within the class only	Cannot be accessed directly	Cannot be invoked directly	Strongest restriction. Access via getters/setters if needed.

Example (Java)

```
public class Person {
    String name; // default access
}
```

- Person class is public
- **name** has default access (no modifier)
- Could also be declared **private** or protected for stricter control

Information Hiding leads to secure, flexible, and maintainable software by exposing only what is necessary.

Information Hiding in Software Design

- **Purpose**: Conceals changeable details (e.g., algorithms, data structures) to reduce dependency.
- **Assumptions**: Not hidden—explicitly expressed in APIs and interfaces.
- **Key Idea**:
  - Hide what may change → implementation details.
  - Reveal what remains stable → assumptions via interfaces.
- **Benefits**:
  - Enables independent module development.
  - Promotes modularity and maintainability.
- **Relation to Encapsulation**:
  - Encapsulation groups attributes and behaviors into classes.
  - Interfaces provide controlled access, hiding internal implementations.
  - Other modules depend on method signatures, not underlying logic.

Information Hiding in Software Design

- **Encapsulation & Interfaces**
  - Implementation is hidden behind an interface.
  - Other classes rely on method signatures, not internal logic.
  - Behaviour can change without altering expected outcomes.
- **Why It Matters**
  - Expectations for a behaviour are fulfilled without exposing *how* it works.
  - Enhances flexibility, maintainability, and modularity.
- **Hiding Attributes**
  - Critical attributes should not be modified directly by external classes.
  - Protects class integrity and prevents unintended side effects.
- **Access Modifiers**
  - Control visibility of attributes and behaviours.
  - Define what subclasses and other classes can access.
  - Examples: public, private, protected.

Attributes & Access Modifiers (Java)

- Attributes can be **hidden** to prevent direct external modification
- **Access levels**:
  - **Public** – Accessible everywhere
  - **Protected** – Accessible within package & subclasses
  - **Default** – Accessible within package only
  - **Private** – Accessible within the class only
- **Public**
  - Accessible by **any class** in the system
  - **Attributes:** Can be read/modified by other classes
  - **Methods:** Can be **invoked** by other classes
  - **Encapsulation still applies** → Implementation hidden
- **Protected**
  - Accessible by:
    - The class itself
    - **Subclasses**
    - **Same package** classes
  - Useful for inheritance and related groups of classes

Attributes & Access Modifiers (Java)

- **Default (No Modifier)**
  - Also called **package-private**
  - Accessible only within the **same package**
  - Not visible to subclasses outside the package
- **Private**
  - Accessible **only within the class itself**
  - Attributes cannot be accessed directly by other classes
  - Methods cannot be invoked by external classes

```
public class Person {
    String name; // default access }
```

- Person class is public
- name has default access (no modifier)
- Could also be declared private or protected for stricter control

Conceptual Integrity in Software Design

- ✓ **Ensures a software system feels consistent and cohesive, as if guided by a single mind.**
- ✓ **Maintains uniformity in design and implementation, even with multiple contributors.**
- ✓ **How It's Achieved**:
  - Clear design principles and conventions.
  - Effective communication among team members.
  - Regular code reviews for consistency.
  - Unified architecture and design foundation.
  - Adoption of common programming constructs.
  - A small core group approving commits to ensure coherence.
- ✓ **Benefit**:
  - Produces software that is predictable, maintainable, and easier to understand.

Conceptual Integrity in Software Design

A software system should feel consistent and cohesive, as if guided by a single mind.

1. WHAT IS CONCEPTUAL INTEGRITY?

- ✓ Ensures software is **consistent** and **cohesive**.
- ✓ Makes the system feel like it was built by **one guiding mind**.
- ✓ Achieved through **shared design principles & conventions**.
- ✓ Not about ignoring **team opinions** → ideas must **align with agreed principles**.
- ✓ Promotes **uniform design** and implementation across thie system.

Benefit:
Produces software that is predictable, maintainable, and easier to understand.

2. WAYS TO ACHIEVE CONCEPTUAL INTEGRITY

- Effective **communication** among team members (Daily stand-ups, sprint retrospectives)
- **Code reviews** for consistency
- Use of **standard design principles** & constructs (Java Interfaces, Design Patterns)
- Adoption of a well-defined architecture (Design for single process, Architecture for multi-process)
- **Unifying concepts** across modules (Find commonality, reduce special cases)
- A small **core review group** to approve commits and ensure coherence

3. WHY IT MATTERS (EXAMPLES)

Dhaka Urban Management System

- **Without conceptual integrity:** traffic control, waste management, and safety systems might be built by different teams with different standards → leading to incompatibility and confusion.
- **With conceptual integrity:** all subsystems follow the same design principles, communication standards, and interfaces → citizens experience **one cohesive urban management platform**.

Agricultural Automation System

- **Without conceptual integrity:** irrigation, drones, and greenhouses might work as separate tools → farmers must learn and manage each independently.
- **With conceptual integrity:** tools share a common architecture (e.g., one dashboard, unified data standards) → system feels **seamless and farmer-friendly**.

4. KEY PRACTICES THAT PROMOTE CONCEPTUAL INTEGRITY

4.1 Code Reviews

- Systematic review of written code (like academic peer review).
- Detects mistakes and improves consistency among developers.
- Ensures adherence to agreed design principles.

4.2 Design Principles & Constructs



- **Java Interfaces** enforce consistent behaviors across classes.
- **Design Patterns** provide conventional, reusable solutions.
- Promote uniformity and conceptual integrity.

4.3 Well-Defined Design & Architecture



- **Design** guides internal structure of single-process software.
- **Architecture** describes multi-process interactions and relationships.
- Both provide consistent structure and logic.

4.4 Unifying Concepts



- Finding commonality across different resources/features.
- Example: Unix philosophy – "treat everything as a file".
- Simplifies usage and reduces special cases → more consistency.

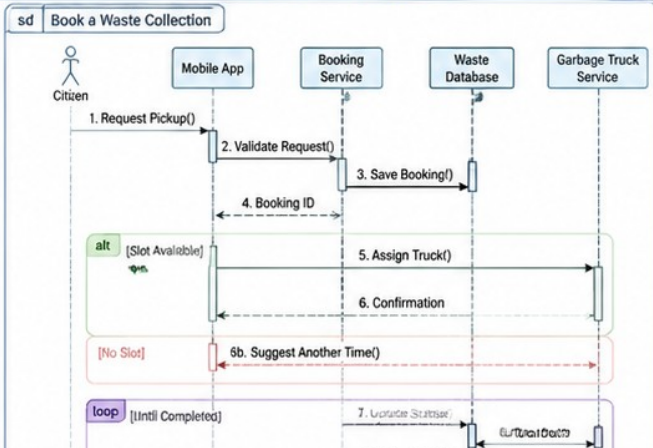
4.5 Small Core Review Group



- A trusted **core team** accepts commits.
- Ensures alignment with architecture & design rules.
- **Prevents** unorganized or "informal" coding practices.

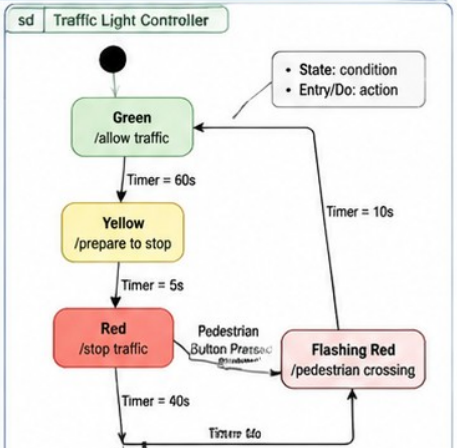
5. UML SEQUENCE DIAGRAM – HOW OBJECTS INTERACT OVER TIME

- **Objects / Roles** (Boxes at top)
- **Lifelines** (Vertical dashed lines)
- **Messages** (Solid arrows)
- **Return Messages** (Dashed arrows)
- **Activation Bars** (Thin rectangles)
- **Actors** (Stick figures)
- **Frame (Diagram box)** Title at top-left
- **Advanced Elements** alt (Alternative)



6. UML STATE DIAGRAM – HOW AN OBJECT BEHAVES OVER TIME

- **States** (Rounded rectangles) Condition of an object at a point in time.
- **Transistions (Arrows)** Movement from one state to another.
- **Events (Triggers)** Cause a transition between states.
- **Actions (Outputs)** What the system does during transition or inside a state.
- **Initial State** (Black filled circle) Starting point.
- **Final State**



Conceptual Integrity in Software Design

- ✓ Ensures software is **consistent and cohesive**
- ✓ Makes the system feel like it was built by **one guiding mind**
- ✓ Achieved through **shared design principles & conventions**
- ✓ Not about ignoring team opinions → ideas must align with agreed principles
- ✓ Promotes **uniform design and implementation** across the system

- ✓ **Ways to Achieve Conceptual Integrity**
  - ➢ **Effective communication** among team members
  - ➢ **Code reviews** for consistency
  - ➢ Use of **standard design principles & constructs**
  - ➢ Adoption of a **well-defined architecture**
  - ➢ **Unifying concepts** across modules
  - ➢ A **core review group** to approve commits

- ✓ Best Practice
  - ➢ Agile practices: **daily stand-ups** & **sprint retrospectives**
  - ➢ Agree on **libraries, frameworks, and methods** to ensure uniformity

Conceptual Integrity in Software Design

- ✓ **Dhaka Urban Management System & Conceptual Integrity**
  - If traffic control, waste management, safety, and citizen services are developed by different teams, the system could feel fragmented.
  - **Without conceptual integrity: traffic control, waste management, and safety systems might be built by different teams with different standards** → leading to incompatibility and confusion.
  - **With conceptual integrity: all subsystems follow the same design principles, communication standards, and interfaces** → citizens experience one **cohesive urban management platform**.
- ✓ **Agricultural Automation System & Conceptual Integrity**
  - Different modules (irrigation sensors, drones, greenhouses, AI models) are often developed separately.
  - **Without conceptual integrity: irrigation, drones, and greenhouses might work as separate tools** → farmers must learn and manage each independently.
  - **With conceptual integrity: these tools share a common architecture (e.g., one dashboard, unified data standards), making the system feel seamless** and farmer-friendly.
  - **Conceptual integrity ensures that complex systems—whether in urban management or agriculture—work as a unified whole, not just a collection of disconnected parts.**

Conceptual Integrity – Key Practices

- **Code Reviews**
  - Systematic review of written code (like academic peer review)
  - Detects mistakes and improves consistency among developers
  - Ensures adherence to agreed design principles
- **Design Principles & Constructs**
  - **Java Interfaces**: enforce consistent behaviors across classes
  - **Design Patterns**: conventional, reusable solutions to design problems
  - Promote uniformity and conceptual integrity
- **Well-Defined Design & Architecture**
  - **Design**: guides internal structure of single-process software
  - **Architecture**: describes multi-process interactions and relationships
  - Both provide consistent structure and logic
- **Unifying Concepts**
  - Finding commonality across different resources or features
  - Example: **Unix philosophy** – treat everything as a file
  - Simplifies usage and reduces special cases → more consistency

Conceptual Integrity – Key Practices On DUMS

1. Code Reviews

- ✓ Developers working on **traffic monitoring** and **waste collection modules** review each other's code.
- ✓ Detects logic errors (e.g., wrong sensor data handling).
- ✓ Ensures both modules follow the same **naming conventions, error handling, and security standards**.

2. Design Principles & Constructs

- ✓ **Java Interfaces**: A standard SensorData interface defines how all sensors (traffic, pollution, waste bins) send data.
- ✓ **Design Patterns**: Observer pattern used so multiple subsystems (traffic control, pollution alerts) can react to real-time sensor updates in the same way.
- ✓ Promotes **uniformity** across modules.

3. Well-Defined Design & Architecture

- ✓ **Design (internal)**: Traffic light control software has a clear internal structure separating data collection, decision logic, and output commands.
- ✓ **Architecture (multi-process)**: Traffic control, waste management, and emergency response services communicate through a **central IoT hub**.
- ✓ Provides a **consistent structure** for scaling and integration.

Conceptual Integrity – Key Practices On DUMS

4. Unifying Concepts

- ✓ **Single Dashboard**: Citizens and administrators use one portal for traffic updates, garbage collection schedules, and safety alerts.
- ✓ Like the **Unix philosophy**: "treat everything as a file," here → "treat all services as a data stream."
- ✓ Reduces special cases → improves **user experience and system consistency**.

- ✓ **Small Core Review Group**
  - ➢ A trusted core team accepts commits
  - ➢ Ensures alignment with architecture & design rules
  - ➢ Prevents unorganized or "informal" coding practices

- ✓ So, **conceptual integrity in Dhaka Urban Management System** ensures all modules—built by different teams—work as one cohesive, predictable, and maintainable system.

Specialized UML class diagrams

- Beyond standard UML class diagrams, this section explores two specialized types and their application in technical design. UML **sequence diagrams, UML State Diagram.**
- UML **sequence diagrams** describe how objects interact with each other over time to complete a task. They are widely used in **planning and technical design**, before coding begins.
- Think of a sequence diagram as a **conversation map**: it shows the messages exchanged between objects (or actors) to achieve a goal.

Key Components of a Sequence Diagram

- **Objects / Roles (Boxes at the top)**
  - Represent roles played by objects (often named after their class).
- **Lifelines (Vertical dashed lines)**
  - Show the passage of time for an object.
- **Messages (Solid arrows)**
  - Represent communication from one object to another.
  - Label above arrow describes the message/action.

Specialized UML class diagrams

- **Return messages (Dashed arrows)**
  - Show data/control returning to the sender.
- **Activation bars (Small rectangles on lifeline)**
  - Indicate when an object is active (sending/receiving/waiting).
- **Actors (Stick figures)**
  - Represent people interacting with the system.
- **Frame (Large box)**
  - Encloses the diagram; title placed at the top-left.
- **Advanced elements**
  - **alt** (Alternative): shows conditional flows.
  - **loop**: repeats actions while a condition holds true.

Example: Changing TV Channel with a Remote

- **Simple Sequence:**
  - Actor (**Viewer**) sends input (press button) → **Remote**.
  - **Remote** forwards command → **TV**.
  - **TV** changes channel → displays to Viewer.
- **Extended with Alternatives & Loops:**
  - If Viewer is **not satisfied** with the channel → loop repeats.
  - Viewer presses "up/down" → Remote → TV → New channel.
  - Continues until Viewer is satisfied.

UML State Diagrams

- A **UML State Diagram** describes how a system or an object **behaves and responds** to events over time. It shows the **different states** an object can be in, and the **transitions** between those states when events occur.
- A **state** represents the condition of an object at a specific moment, defined by its **attributes and behavior**.

Key Elements of a State Diagram

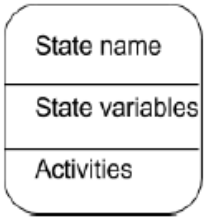
- **States (Rounded rectangles)**
  - Show the condition of an object at a point in time.
  - Example: *Park, Reverse, Neutral, Drive* in a car.
- **Transitions (Arrows)**
  - Show the movement from one state to another when an event occurs.
- **Events (Triggers)**
  - Cause a transition between states.
  - Example: *Shift gear, Press start, Turn off*.
- **Actions (Outputs/Behaviors)**
  - What the system does during a transition or inside a state.
- **Initial State (Black filled circle)**
  - The starting point of the system/object.
- **Final State (Circle with dot inside):** Where the system/object ends.

UML State Diagrams

- A filled circle indicates the starting state of the object. Every state diagram begins with a filled circle.

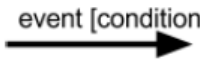


- Rounded rectangles indicate other states. These rectangles have three sections: a state name, state variables, and **activities**.



  - State names should be short, meaningful titles for the state of the object. Each state should have at least a state name.

- Arrows indicate **transitions** from one state to another. Transitions are typically triggered by an event. This event is generally described above the arrow.



  Each transition arrow will always have an event, and it may even have a guard condition and an action. The transition and action happens from a given state if the event occurs and the condition is true.
- A circle with a filled circle inside indicates termination. Termination represents an object being destroyed or the process being completed. Not all diagrams have a termination—some may run continuously.

  - State variables are data relevant to the state of the object.
  - **Activities** are actions that are performed when in a certain state. There are three types of activities



present for each state: Entry, Exit, and Do activities. **Entry activities** are actions that occur when the state is just entered from another state. **Exit activities** are actions that occur when the state is exited and moves on to another state. **Do activities** are actions that occur while the object is in a certain state.

Example: Car with Automatic Transmission

Example: Car with Automatic Transmission

✓ A car can exist in **different states**:

- **Park** → Car is stationary and locked.
- **Reverse** → Car moves backward.
- **Neutral** → Car wheels are disengaged.
- **Drive** → Car moves forward.

Transition Example:

- If the car is in **Reverse**, it can only move backward.
- To move forward, the driver triggers a **gear shift event**, changing the state to **Drive**.

✅ **Why Use State Diagrams?**

- ✓ **Clarify behavior**: Easier to see how an object responds to different inputs than reading source code.
- ✓ **Reveal missing cases**: Helps identify unplanned conditions or transitions.
- ✓ **Support testing**: Knowing all possible states ensures **complete test coverage**.
- ✓ **Better communication**: Provides a shared visual language for designers, developers, and testers.

Model Checking

- ✓ Model checking is a **formal verification technique** used to systematically check whether a software system behaves correctly in **all its possible states**.
  - ➢ It explores every possible state transition of the system.
  - ➢ It helps identify errors that might not be caught by traditional testing methods (unit tests, beta testing, simulations, etc.).
- **How It Works**

1.Generate a State Model

1. From your software (e.g., code or specification), the model checker builds a **state model**.
2. A state model is like an abstract state machine that represents the system's states and transitions.

2.Check Behavioural Properties

1. The model checker verifies whether the state model satisfies specific properties, such as:
   1. No **deadlock** (system should not freeze with tasks waiting for each other).
   2. No **race conditions** (two processes accessing shared resources unpredictably).
   3. Certain events must always eventually occur.

Model Checking

1. 🔹 **Why It's Useful Explore All States**
   1. The model checker simulates every possible state change caused by different events and inputs.
   2. If a violation is found (e.g., a deadlock is possible), the tool provides a **counterexample trace** showing how it could happen.

✓ **Example**

- Suppose a system has a **rule: "No deadlock should occur."**
- The model checker simulates all possible state transitions.
- If it finds that Process A and Process B could wait forever for the same resource, it reports a **deadlock violation**.

✅ Finds errors not easily detected by testing.
✅ Ensures system reliability, especially in safety-critical systems (banking, aviation, medical devices, etc.).
✅ Complements other testing techniques.

Example: Smartphone System

- ✓ Initial Design:
  - Single SmartPhone class with both camera and phone functions
  - Low cohesion → unrelated behaviors mixed together
  - Poor modularity → cannot reuse or replace components separately
- ✓ Improved Design (with SoC):
  - Identify two concerns:
    - Phone functionality
    - Camera functionality
  - Create separate cohesive classes (Phone, Camera)
  - SmartPhone acts as coordinator, referencing these components
  - Provides access without needing internal details

Thank You (Q/A)

Design Patterns

Introduction to Design Patterns

- ✓ **Practical proven solutions for recurring software design problems.**
- ✓ **Guide software structure and improve flexibility, reusability, and maintainability.**
- ✓ **Not a code library;** a conceptual design solution applied by developers.
- ✓ Similar to tested recipes for solving common design challenges.

Importance and Applications of Design Patterns

- ✓ **Reduce repeated design effort and improve software quality.**
- ✓ Create reusable and maintainable object-oriented systems.
- ✓ **Provide common communication language** among developers.
- ✓ Applications: enterprise systems, web applications, frameworks, large software projects.

Gang of Four (GoF) Pattern Catalogue

- Introduced by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- **Contains 23 classic object-oriented design patterns.**
- **Patterns are grouped based on purpose**: **Creational, Structural, Behavioural**.
- Provides a common reference for solving software design problems.

Pattern Languages

- A collection of related patterns for a specific problem domain.
- Pattern selection **depends on system context** and design goals.
- Different **domains require different combinations** of patterns.

Categories of Design Patterns

- ✓ Creational Patterns: object creation and initialization.
- ✓ Structural Patterns: organization and relationships among objects.
- ✓ Behavioural Patterns: communication and responsibility distribution.

Gang of Four (GoF) Pattern Catalogue and Categories of Design Patterns

- **Gang of Four (GoF):** The book "Design Patterns: Elements of Reusable Object-Oriented Software" was published in 1994 by Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides.
- **Purpose:** To describe 23 proven solutions (patterns) to common object-oriented design problems. These patterns promote reusable, flexible, and maintainable software design.

The Gang of Four (GoF) Authors



Erich Gamma | Richard Helm | Ralph Johnson | John Vlissides

23 Design Patterns in GoF Catalogue

Creational (5)

1. Singleton
2. Factory Method
3. Abstract Factory
4. Builder
5. Prototype

Structural (7)

1. Adapter
2. Bridge
3. Composite
4. Decorator
5. Facade
6. Flyweight
7. Proxy

Behavioural (11)

1. Template Method
2. Chain of Responsibility
3. Command
4. Interpreter
5. Iterator
6. Mediator
7. Memento
8. Observer
9. State
10. Strategy
11. Visitor

Three Categories of Design Patterns

1. Creational Patterns

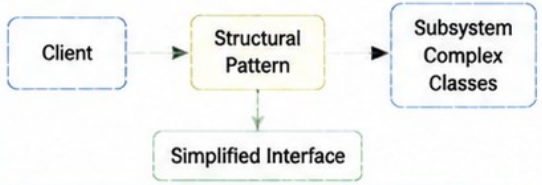
Focus: Object creation mechanisms.
Goal: Make a system independent of how its objects are created, composed, and represented.



Examples: Singleton, Factory Method, Abstract Factory, Builder, Prototype

2. Structural Patterns

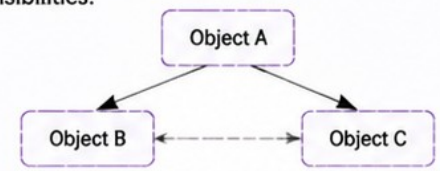
Focus: Class and object composition.
Goal: Build larger structures from simpler components and expose easy-to-use interfaces.



Examples: Adapter, Bridge, Composite, Decorator, Facade, Flyweight, Proxy

3. Behavioural Patterns

Focus: Communication between objects.
Goal: Define how objects interact and distribute responsibilities.



Examples: Template Method, Chain of Responsibility, Command, Iterator, Observer, State, Strategy, Visitor, Mediator, Memento, Interpreter

Design Patterns are time-tested solutions to recurring design problems. They help us write software that is **reusable**, **flexible**, **maintainable**, and easy to understand.

Creational Patterns

- ✓ Focus on flexible object creation mechanisms.
- ✓ Reduce dependency on concrete object classes.
- ✓ **Examples:** Singleton Pattern, Factory Method Pattern.

Singleton Pattern

- Creates only one instance of a class.
- Useful when a single shared object is required.
- **Implementation:** private constructor, unique instance variable, getInstance() method.
- **Examples:** database connection, printer queue, configuration manager.

Singleton Pattern

1. Create private static instance variable.
2. Hide constructor using private access modifier.
3. Create public method to provide instance access.
4. Return existing instance if already created.

Factory Method Pattern

- Separates object creation from object usage.
- Client depends on general interfaces instead of concrete classes.
- Allows new object types without modifying client code.

Factory Method Steps

1. Create Product interface or abstract class.
2. Create concrete product classes.
3. Define factory method in creator class.
4. Subclasses decide which object to create.

Singleton Pattern vs Factory Method Pattern

SINGLETON PATTERN

Concept

Singleton pattern ensures that a class has only one instance and provides a global point of access to it.
It controls object creation and is used when exactly one object is needed to coordinate actions across the system.

When to Use

- When exactly one instance of a class is required.
- To control access to shared resource (e.g., configuration, logger, cache, database connection).

Real-life Examples

- Logger in an application (writes logs to a single file).
- Database connection manager.
- Configuration or settings manager.

Key Benefits

- Controlled access to the single instance.
- Reduced memory usage.
- Global access point.
- Ensures consistency of state.

UML Class Diagram



How it Works



1. Client calls getInstance().
2. If instance is null, create a new one.
3. Store the instance and return it.
4. Next time, return the same instance.

Java Code Example

Singleton Class

```
public class Logger {
   // 1. private static instance
   private static Logger instance;

   // 2. private constructor
   private Logger() {
      System.out.println("Logger instance created");
   }
   // 3. public static method to get the instance
   public static Logger getInstance() {
      if (instance == null) {
         instance = new Logger();
      }
      return instance;
   }
   public void log(String message) {
      System.out.println("LOG:  " + message);
   }
}
```

Client Class

```
public class Client {
   public static void main(String[] args) {
      Logger logger1 = Logger.getInstance();
      logger1.log("First log");

      Logger logger2 = Logger.getInstance();
      logger2.log("Second log");

      System.out.println("Same instance? " +
         (logger1 == logger2));  // true
   }
}
```

Output:

```
Logger instance created
LOG: First log
LOG: Second log
Same instance? true
```

Key Point: The constructor is private and the instance is static. Therefore, only one instance can be created and shared.

FACTORY METHOD PATTERN

Concept

Factory Method pattern defines an interface for creating an object, but lets subclasses decide which class to instantiate.
It delegates object creation to subclasses.

When to Use

- When a class can't anticipate the class of objects it must create.
- When subclasses should decide which object to create.
- To achieve loose coupling.

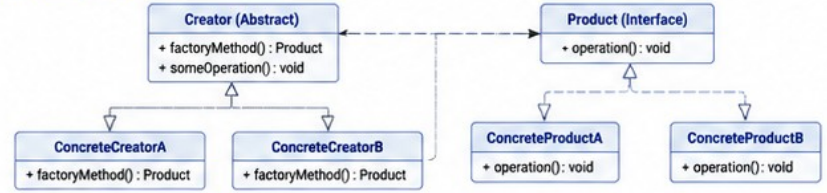
Real-life Examples

- Document creator: Create PDFDocument or WordDocument without client knowing details.
- Notification system: EmailNotification or SMSNotification.
- UI component: Create WindowsButton or MacButton.

Key Benefits

- Promotes loose coupling.
- Easy to add new products without changing client code.
- Follows Open/Closed Principle.

UML Class Diagram



How it Works



1. Client calls someOperation() on Creator.
2. Creator calls factoryMethod().
3. Subclass decides which ConcreteProduct to create.
4. The created product is returned and used by client.

Java Code Example

Product Interface

```
public interface Document {
  void open();
}
```

Concrete Products

```
public class PDFDocument
implements Document {
  public void open() {
    System.out.println("Open PDF Document");
  }
}
public class WordDocument
implements Document {
  public void open() {
    System.out.println("Open Word Document");
  }
}
```

Creator (Abstract Class)

```
public abstract class
DocumentCreator {
  public abstract Document
    factoryMethod();

  public void someOperation() {
    Document doc = factoryMethod();
    doc.open();
  }
}
```

Concrete Creators

```
public class PDFCreator
extends DocumentCreator {
  public Document factoryMethod() {
    return new PDFDocument();
  }
}

public class WordCreator
extends DocumentCreator {
  public Document factoryMethod() {
    return new WordDocument();
  }
}
```

Client Class

```
public class Client {
  public static void main
  (String[] args) {
    DocumentCreator creator1 =
      new PDFCreator();
    creator1.someOperation();
    DocumentCreator creator2 =
      new WordCreator();
    creator2.someOperation();
  }
}
```

Output:

```
Open PDF Document
Open Word Document
```

Key Point: The client depends on the Creator abstract class. Subclasses decide which Product implementation to create.

Singleton Pattern vs Factory Method Pattern

Concept, Example, UML and Code

SINGLETON PATTERN

class has only one instance and provide a global [illegible]

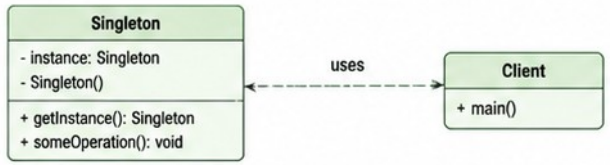
When to Use

- When exactly one instance of a class is needed.
- To control access to shared resources.
- Example: Configuration, Logger, Database Connection.

Key Benefit

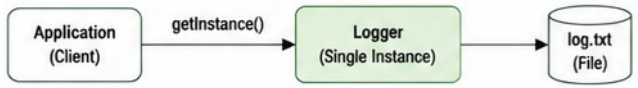
Controlled access to the single instance, reduces memory usage.

UML Class Diagram



Real-life Example

A Logger in an application should have only one instance to write logs to a single file.



Java Code Example

```java
public class Logger {
    private static Logger instance;   // single instance
    private Logger() {
        System.out.println("Logger instance created");
    }

    public static Logger getInstance() {
        if (instance == null) {
            instance = new Logger();
        }
        return instance;
    }
    public void log(String message) {
        System.out.println("LOG: " + message);
    }
}
```

```java
class Client {
    public static void main(String[] args) {
        Logger logger1 = Logger.getInstance();
        logger1.log("First log");

        Logger logger2 = Logger.getInstance();
        logger2.log("Second log");

        System.out.println(logger1 == logger2);
        // Output: true (same instance)
    }
}
```

How it Works



FACTORY METHOD PATTERN

Define an interface for creating an object, but let subcl[illegible] decide which class to instantiate.

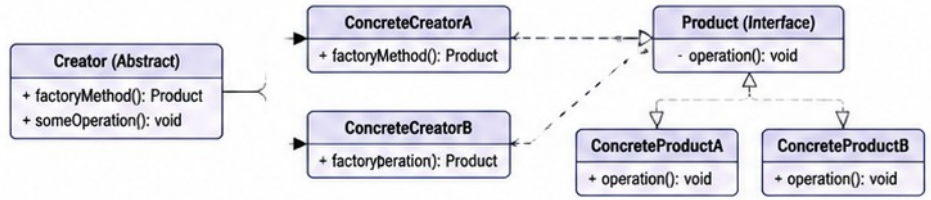
When to Use

- When a class can't anticipate the class of objects it must create.
- When subclasses should decide which object to create.
- Example: Document, Notification, Button in a GUI.

Key Benefit

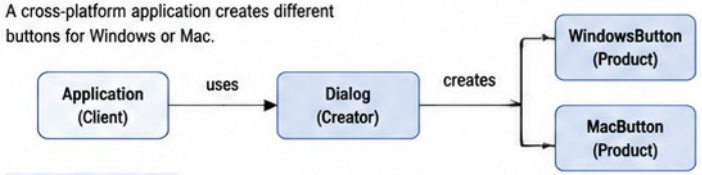
Promotes loose coupling and easy extension (Open/Closed Principle).

UML Class Diagram



Real-life Example

A cross-platform application creates different buttons for Windows or Mac.



Possible Products

- WindowsButton
- MacButton
- LinuxButton
- WebButton

Java Code Example

```java
// Product
interface Button {
  void render();
}

// Concrete Products
class WindowsButton inplements Button {
  public void render() {
    System.out.println("Render Windows Button");
  }
}

class MacButton inplements Button {
  public void render() {
    System.out.println("Render Mac Button");
  }
```

```java
// Creator
abstract class Dialog {
  // Factory Method
  abstract Button createButton();

  public void renderUI() {
    Button b = createButton();
    b.render();
  }
}

// Concrete Creators
class WindowsDialog extends Bialog {
    Button createButton() {
        return new WindowsButton();
    }
```

```java
class MacBialog extends Bialog {
    Button createButton() {
        return new MacButton();
    }
}

// Client
class Client {
    public static void main(String[] args) {
        Bialog dialog;

        dialog = new WindowsDialog();
        dialog.renderUI();   // Render Windows Button

        dialog = new MacDialag();
        dialog.renderUI();   // Render Mac Button
```

How it Works



Abstract Factory Pattern, Builder Pattern & Prototype Pattern

Concept, Example, UML Diagram & Java Code

1. ABSTRACT FACTORY PATTERN

Provides an interface for creating families of related or dependent objects without specifying their concrete classes.

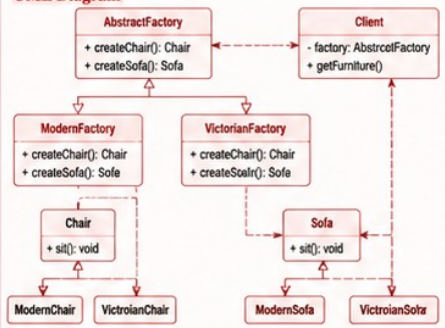
When to Use

- When a system should be independent of how its products are created, composed and represented.
- When the system must be configured with one of multiple families of products.

Example

A furniture store offers modern and victorian style furniture. Client code uses a factory to get matching chair and sofa.

UML Diagram



Java Code Example

```
// Product Interfaces
interface Chair { void sit(); }
interface Sofa { void sit(); }

// Concrete Products
class ModernChair implements Chair {
    public void sit() {
        System.out.println("Sitting on Modern Chair");
    }
}
class ModernSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on Modern Sofa");
    }
}
// Abstract Factory
interface FurnitureFactory {
    Chair createChair();
    Sofa createSofa();
}
```

```
// Concrete Factories
class ModernFactory implements FurnitureFactory {
    public Chair createChair() {
        return new ModernChair();
    }
    public Sofa createSofa() {
        System.out.println("Sitting on Modern Sofa");
    }
}
// Client
class Client {
    private FurnitureFactory factory;
    Client(FurnitureFactory factory) {
        this.factory = factory;
    }
    void getFurniture() {
        Chair chair = factory.createChair();
        Sofa sofa = factory.createSofa();
        chair.sit();
        sofa.sit();
    }
}
```

How it Works



1. Client selects a concrete factory
2. Factory creates related products
3. Products are used together
4. System remains consistent

2. BUILDER PATTERN

Separates the construction of a complex object from its representation so that the same construction process can create different representations.

When to Use

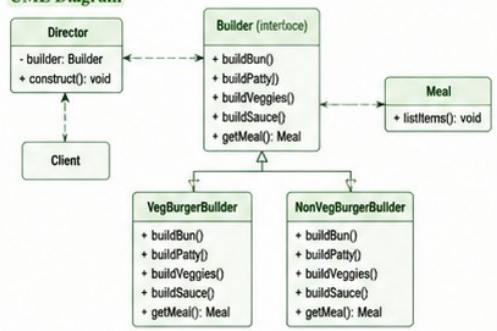
- When the construction process must allow different representations for the object that is constructed.
- When a complex object should be created step by step.

Example



A meal can be a veg or non-veg burger. We build the burger step by step using the same process but with different ingredients.

UML Diagram



Java Code Example

```
// Product
class Meal {
    private List<String> items = new ArrayList<>();
    public void addItem(String item) { items.add(item); }
    public void listItems() {
        System.out.println(items);
    }
}

// Builder
interface Builder {
    void buildBun();
    void buildPatty();
    void buildVeggies();
    void buildSauce();
    Meal getMeal();
}

// Concrete Builder
class VegBurgerBuilder implements Builder {
    private Meal meal = new Meal();
    public void buildBun() {
        meal.addItem("Veg Bun");
    }
    public void buildPatty() {
        meal.addItem("Aloo Patty");
    }
    public void buildVeggies() {
        meal.addItem("Lettuce");
    }
    public void buildSauce() {
        meal.addItem("Tomato Sauce");
    }
    public Meal getMeal() { return meal; }
}

// Director
class Director {
    private Builder builder;
    Director(Builder builder) { this.builder = builder; }
    void construct() {
        builder.buildBun();
        builder.buildPatty();
        builder.buildVeggies();
        builder.buildSauce();
    }
}
```

How it Works



1. Client chooses a builder
2. Director controls construction steps
3. Builder builds the complex object
4. Final object is returned to client

3. PROTOTYPE PATTERN

Specifies the kinds of objects to create using a prototypical instance, and creates new objects by copying this prototype.

When to Use

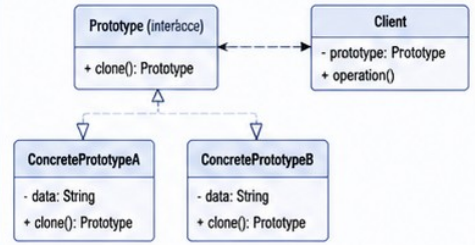
- When the classes to instantiate are specified at runtime.
- To avoid building a class hierarchy of factores that parallels the class hierarchy of products.

Example



A document editor can clone existing documents (letter, report) to create new ones with the same content.

UML Diagram



Java Code Example

```
// Prototype Interface
interface Document extends Cloneable {
    Document clone();
    void showData();
}
// Concrete Prototype
class Letter implements Document {
    private String content;
    public Letter(String content) { this.content = content; }
    public Document clone() {
        return new Letter(this.content);
    }
    public void showData() {
        System.out.println("Letter: " + content);
    }
}
// Client
class Client {
    public static void main(String[] args) {
        Document d1 = new Letter("Hello, this is a letter.");
        Document d2 = d1.clone();  // cloning
        d2.showData();             // Letter: Hello, this is a letter.
    }
}
```

How it Works



1. Client has a prototype object
2. Clone() is called to copy the prototype
3. New object is created
4. Client uses the cloned object

Abstract Factory Pattern, Builder Pattern & Prototype Pattern

Concept, Example, UML Diagram, Java Code, How it Works & Key Benefits

1. ASTRACT FACTORY PATTERN

Concept: Provides an interface for creating familles of related or dependent objects without specifying their concrete classes.

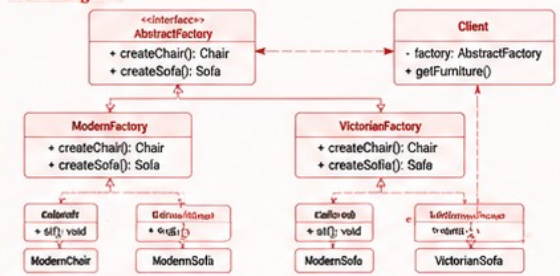
When to Use

- When a system should be independent of how its products are created, composed and represented.
- When the system must be configured with one o multiple families of products.

Example

A furniture store offers modern and victorian style furniture. Client code uses a factory to get matching chair and sofa.

UML Diagram



Java Code Example

```
// Product Interfaces
interface Chair {
    void sit();
}
interface Sofa {
    void sit();
}
// Concrete Products (Modern)
class ModernChair implements Chair {
    public void sit() {
        System.out.println("Sitting on Modern Chair");
    }
}
class ModernSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on Modern Sofa");
    }
}
```

```
// Concrete Products (Victorian)
class VictorianChair implements Chair {
    @Override public void sit() {
    public sofa sit() {
        System.out.println("Sitting on
        Victorian Chair");
    }

class VictorianSofa implements Sofa {
    public void sit() {
        System.out.println("Sitting on
        Victorian Sofa");
    }
}
```

```
// Abstract Factory
interface AbstractFactory {
    Chair createChair();
    Sofa createSofa();
}
// Concrete Factories
class ModernFactory implements AbstractFactory {
    public Chair createChair() { return new ModernChair(); }

    public Sofa createSofa() { return new ModernSofa(); }
}
class VictorianFactory implements AbstractFactory {
    public Chair createChair() { return new VictorianChair(); }
    System.out.createSofa() { return new ModernCouch(); }
}
// Client (
class Client {
    private AbstractFactory factory;
    public Client(AbstractFactory factory) {
        this.factory = factory; } } }
```

Structural Patterns

- ✓ Describe how objects and classes are organized and connected.
- ✓ Support decomposition and flexible relationships.
- ✓ Examples: Facade, Adapter, Composite, Proxy, Decorator.

Facade Pattern

- Provides a simple interface to a complex subsystem.
- Hides internal subsystem details from clients.
- Reduces coupling and simplifies interaction.

Facade Pattern Steps

1. Design subsystem interfaces.
2. Implement subsystem classes.
3. Create facade wrapper class.
4. Client uses facade instead of multiple subsystem classes.

Adapter Pattern

- Allows incompatible interfaces to communicate.
- Works as a bridge between client and existing system.
- Components: Client, Target Interface, Adapter, Adaptee.

Adapter Pattern Steps

1. Design target interface.
2. Implement adapter using target interface.
3. Convert client requests into adaptee format.
4. Client communicates through adapter.

Composite Pattern

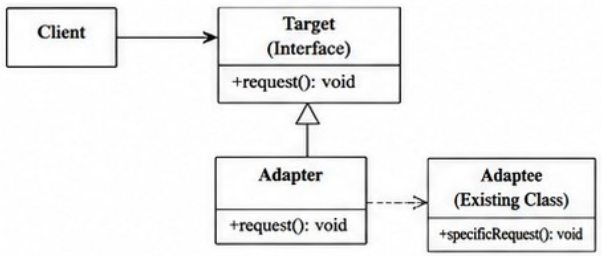
- Creates tree-like object structures.
- Allows individual objects and groups to be treated uniformly.
- Examples: file systems, organization structures.

Structural Design Patterns: Adapter, Bridge, Composite

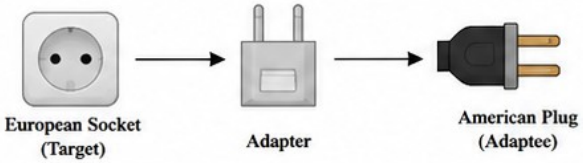
1. ADAPTER PATTERN

Adapter pattern allows incompatible interfaces to work together.

Diagram



Example: Power Plug Adapter



Java Code Example

```
// Target interface
interface EuropeanSocket {
    void providePower();
}

// Adaptee class
class AmericanPlug {
    public void provide110Volt() {
        System.out.println("Providing 110 Volts");
    }
}

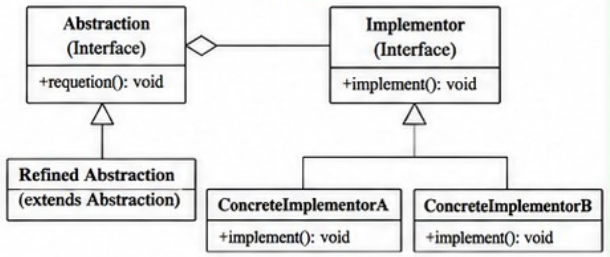
// Adapter class
class SocketAdapter implements EuropeanSocket {
    private AmericanPlug plug;
    public SocketAdapter(AmericanPlug plug) {
        this.plug = plug;
    }
    public void providePower() {
        plug.provide110Volt();
    }
}

// Client
EuropeanSocket socket = new SocketAdapter(new AmericanPlug());
socket.providePower();  // Output: Providing 110 Volts
```

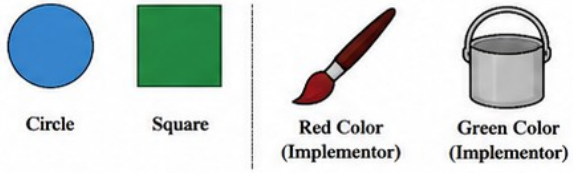
2. BRIDGE PATTERN

Bridge pattern decouples an abstraction from its implementation so that the two can vary independently.

Diagram



Example: Drawing Shapes



Java Code Example

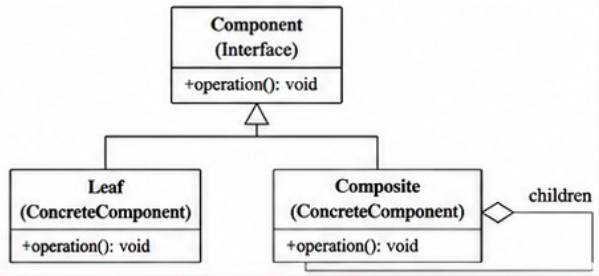
```
// Implementor
interface Color {
    void applyColor();
}
class RedColor implements Color {
    public void applyColor() {
        System.out.println("Applying Red Color");
    }
}
class GreenColor implements Color {
    public void applyColor() {
        System.out.println("Applying Green Color");
    }
}
// Abstraction
abstract class Shape {
    protected Color color;
    public Shape(Color color) { this.color = color; }
    abstract void draw();
}
class Circle extends Shape {
    public Circle(Color color) { super(color); }
    public void draw() {
        System.out.print("Drawing Circle -> ");
        color.applyColor();
    }
}

// Client
Shape s1 = new Circle(new RedColor());  s1.draw();
Shape s2 = new Circle(new GreenColor()); s2.draw();
```

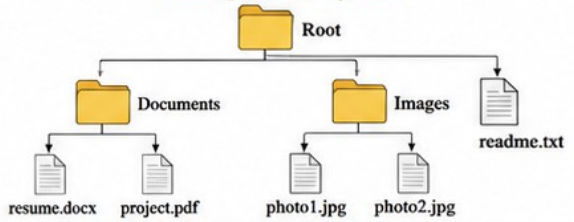
3. COMPOSITE PATTERN

Composite pattern composes objects into tree structures to represent part-whole hierarchies.

Diagram



Example: File System



Java Code Example

```
// Component
interface FileSystemComponent {
    void showDetails();
}
// Leaf
class File implements FileSystemComponent {
    private String name;
    public File(String name) { this.name = name; }
    public void showDetails() {
        System.out.println("File: " + name);
    }
}

// Composite
class Folder implements FileSystemComponent {
    private String name;
    private List<FileSystemComponent> children = new ArrayList<>();
    public Folder(String name) { this.name = name; }
    public void add(FileSystemComponent comp) { children.add(comp); }
    public void showDetails() {
        System.out.println("Folder: " + name);
        for (FileSystemComponent comp : children) {
            comp.showDetails();
        }
    }
}
// Client
Folder root = new Folder("Root");
Folder docs = new Folder("Documents");
docs.add(new File("resume.docx"));
docs.add(new File("project.pdf"));
root.add(docs);
root.add(new Folder("Images"));
root.add(new File("readme.txt"));
root.showDetails();
```

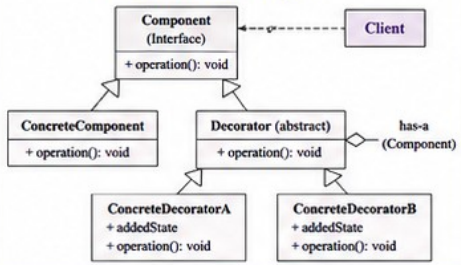
Structural Design Patterns: Decorator, Facade, Flyweight, Proxy (Server)

These patterns organize classes and objects to form larger structures while keeping the system flexible and easier to maintain.

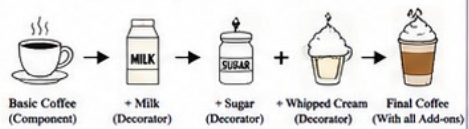
1. DECORATOR PATTERN

Decorator pattern attaches additional responsibilities to an object dynamically. It provides a flexible alternative to subclassing for extending functionality.

Structure / UML Diagram



Example: Coffee Shop



Java Code Example (Simplified)

```
// Component
interface Coffee {
    String cost();
    String description();
}

// Concrete Component
class SimpleCoffee implements Coffee {
    public String cost() { return "50"; }
    public String description() { return "Simple Coffee"; }
}

// Decorator (abstract)
abstract class CoffeeDecorator implements Coffee {
    protected Coffee coffee;
    public CoffeeDecorator(Coffee c) { this.coffee = c; }
}

// Concrete Decorator
class MilkDecorator extends CoffeeDecorator {
    public MilkDecorator(Coffee c) { super(c); }
    public String cost() { return coffee.cost() + " + 10"; }
    public String description() { return coffee.description() + ", Milk"; }
}
```

When to Use

- When you want to add responsibilities to objects dynamically.
- When subclassing leads to too many classes.
- When the extension should be flexible at runtime.

Key Benefits

- More flexible than inheritance.
- Allows responsibilities to be added/removed at runtime.
- Follows Open/Closed Principle.
- Combinations can be created easily.

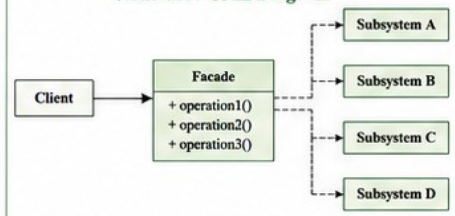
Think & Discuss

What new decorators would you add to the Coffee Shop example? Try adding VanillaDecorator and ChocolateDecorator.

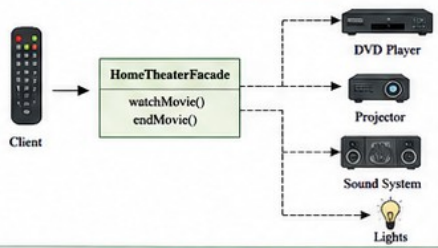
2. FACADE PATTERN

Facade pattern provides a simplified interface to a complex subsystem. It hides the complexities and makes the subsystem easier to use.

Structure / UML Diagram



Example: Home Theater System



Java Code Example (Simplified)

```
class HomeTheaterFacade {
    private DVDPlayer dvd;
    private Projector projector;
    private SoundSystem sound;
    private Lights lights;

    public HomeTheaterFacade(DVDPlayer dvd, Projector projector,
            SoundSystem sound, Lights lights) {
        this.dvd = dvd; this.projector = projector;
        this.sound = sound; this.lights = lights;
    }
    public void watchMovie() {
        lights.dim();
        projector.on();
        dvd.play();
        sound.on();
    }
    public void endMovie() {
        dvd.stop();
        projector.off();
        sound.off();
        lights.on();
    }
}
```

When to Use

- When a complex subsystem is difficult to use.
- When you need a simple entry point to use the system.
- When you want to decouple clients from subsystem classes.

Key Benefits

- Reduces complexity for clients.
- Promotes loose coupling.
- Organizes subsystem classes.
- Easier to understand and use.

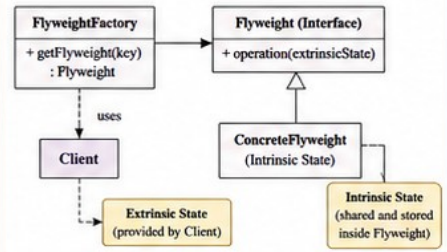
Think & Discuss

Why should the client not directly use the subsystem classes? What happens if the subsystem changes?

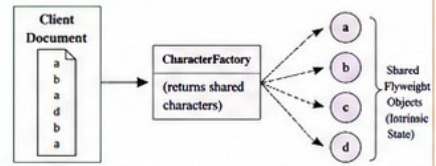
3. FLYWEIGHT PATTERN

Flyweight pattern uses sharing to support large numbers of fine-grained objects efficiently. It reduces memory usage by sharing common state among objects.

Structure / UML Diagram



Example: Text Editor (Characters)



Java Code Example (Simplified)

```
interface Character {
    void display(int position);
}

class ConcreteCharacter implements Character {
    private char symbol; // intrinsic state (shared)
    public ConcreteCharacter(char c) { this.symbol = c; }
    public void display(int position) {
        System.out.println("'" + symbol + "' at " + position);
    }
}

class CharacterFactory {
    private Map<Character, Character> map = new HashMap<>();
    public Character getCharacter(char c) {
        return map.computeIfAbsent(c, k -> new ConcreteCharacter(k));
    }
}
```

When to Use

- When your system has a large number of similar objects.
- When most of the object state can be made intrinsic (shared).
- When memory usage is critical.

Key Benefits

- Saves memory.
- Improves performance.
- Scalable with large numbers of objects.

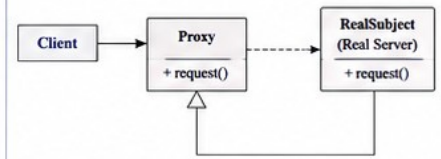
Think & Discuss

Where can you apply Flyweight Pattern in real life? (E.g., game development, text editors, icons, etc.)

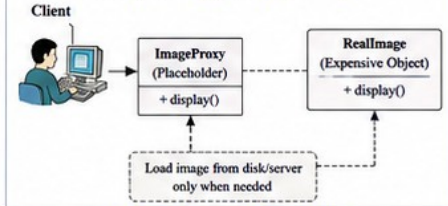
4. PROXY (SERVER) PATTERN

Proxy pattern provides a surrogate or placeholder for another object to control access to it. The proxy controls access to the real object (server).

Structure / UML Diagram



Example: Image Proxy (Server)



Java Code Example (Simplified)

```
interface Image {
    void display();
}
class RealImage implements Image {
    private String fileName;
    public RealImage(String fileName) {
        this.fileName = fileName;
        loadFromDisk();
    }
    private void loadFromDisk() {
        System.out.println("Loading " + fileName + " ...");
    }
    public void display() {
        System.out.println("Displaying " + fileName();
    }
}
class ImageProxy implements Image {
    private RealImage realImage;
    private String fileName;
    public ImageProxy(String fileName) { this.fileName = fileName; }
    public void display() {
        if (realImage == null) realImage = new RealImage(fileName);
        realImage.display();
    }
}
```

When to Use

- When object creation is expensive.
- When you need access control.
- When you need lazy loading.
- When you need remote proxies (e.g., server communication).

Key Benefits

- Controls access to objects.
- Allows lazy initialization.
- Reduces cost of object creation.
- Improves performance.

Think & Discuss

How can Proxy Pattern be used in a real-world application like a caching server or firewall?

Quick Comparison

- Decorator adds behavior dynamically by wrapping objects.
- Facade provides a simple interface to a complex subsystem.
- Flyweight shares common objects to save memory.
- Proxy controls access to another object and manages its creation.

Proxy Pattern

- ✓ Provides a substitute object controlling access to another object.
- ✓ Used for security, performance optimization, and resource management.

Decorator Pattern

- ✓ Dynamically adds responsibilities to objects at runtime.
- ✓ Uses aggregation instead of creating many subclasses.
- ✓ Allows flexible combinations of behaviours.

Decorator Pattern Steps

1. Design component interface.
2. Implement concrete component.
3. Implement abstract decorator.
4. Create concrete decorators adding behaviours.

Module 1 Summary

- Design patterns provide reusable solutions for common design problems.
- Creational patterns manage object creation.
- Structural patterns organize object relationships.
- Good pattern selection improves maintainability and flexibility.

Sample Question

Bloom's Level	Question
Remember	1. What is a design pattern? Name the three major categories of GoF design patterns.
Understand	2. Explain the purpose of the Gang of Four (GoF) design pattern catalog.
Understand	3. Explain the difference between creational and structural design patterns with suitable examples.
Apply	4. Suppose an application requires exactly one shared configuration object throughout the system. Which design pattern would you use? Explain how you would apply it.
Analyze	5. Analyze the following situation: a program directly creates many different concrete objects using new statements throughout the client code. Identify the design problem and determine whether the Factory Method Pattern could improve the design.
Evaluate	6. A developer uses the Singleton Pattern for every shared object in an application. Evaluate this design decision and discuss its advantages and possible disadvantages.
Create	7. Design a simple Factory Method-based system for creating different types of university courses such as TheoryCourse, LabCourse, and ProjectCourse. Describe the classes and their relationships.

Bloom's Level	Question
Remember	1. List the seven structural/creational patterns covered in Module 1.
Understand	2. Explain the main purpose of the Facade Pattern and give one practical example.
Understand	3. Explain how the Adapter Pattern allows two incompatible interfaces to work together.
Apply	4. A legacy payment system provides makePayment(), while a new application expects pay(). Which pattern would you apply and how would it solve the problem?
Analyze	5. Analyze a file-system design containing File and Folder objects. Explain why the Composite Pattern may be appropriate for representing this hierarchy.
Evaluate	6. Compare the Proxy and Facade Patterns. Evaluate which pattern would be more appropriate for controlling access to an expensive remote service and justify your answer.
Create	7. Design a home-theater system using the Facade Pattern. Identify the subsystem classes, Facade class, and the operations exposed to the client.

Bloom's Level	Focus	Typical Question Verbs
Remember	Recall facts/concepts	Define, List, Name, Identify
Understand	Explain concepts	Explain, Describe, Summarize, Discuss
Apply	Use knowledge in a situation	Apply, Demonstrate, Implement, Use
Analyze	Break down and find relationships/problems	Analyze, Compare, Differentiate, Examine
Evaluate	Make a justified judgment	Evaluate, Critique, Justify, Assess
Create	Produce a new design/solution	Design, Develop, Create, Propose

Bloom's Level	Question
Remember	1. Define Singleton, Factory Method, Adapter, Facade, Composite, Proxy, and Decorator Patterns.
Understand	2. Explain how the Decorator Pattern adds responsibilities to an object dynamically.
Understand	3. Explain the difference between inheritance and aggregation in the context of the Decorator Pattern.
Apply	4. A coffee application needs to dynamically add milk, sugar, and whipped cream to a basic coffee object. Which design pattern would you use? Explain the implementation approach.
Analyze	5. Analyze a system in which separate subclasses have been created for every possible combination of features, such as CoffeeWithMilk, CoffeeWithSugar, CoffeeWithMilkAndSugar, and CoffeeWithMilkSugarCream. Identify the design problem and suggest a suitable pattern.
Evaluate	6. Evaluate the use of the Decorator Pattern compared with creating multiple subclasses for every combination of additional functionality.
Create	7. Design a Java-based notification system using the Decorator Pattern in which basic notifications can dynamically be enhanced with email, SMS, and push-notification features.

Sample Question

Bloom's Level	Question
Remember	1. What is the main purpose of creational design patterns? Name two creational patterns discussed in Module 1.
Understand	2. Explain how the Singleton Pattern controls object creation.
Understand	3. Explain the difference between the Facade Pattern and the Adapter Pattern.
Apply	4. A university system requires a single object to manage application-wide configuration settings. Which pattern would you select? Explain how you would implement it.
Analyze	5. Analyze a system in which the client directly creates and manages several objects belonging to a complex subsystem. Identify the design problem and explain how the Facade Pattern could simplify the design.
Evaluate	6. Evaluate whether the Adapter Pattern or Facade Pattern is more appropriate when two existing classes have incompatible interfaces. Justify your answer.
Create	7. Design a Java-based file-management system using the Composite Pattern. Show how individual files and folders can be treated uniformly.

Bloom's Level	Question
Remember	1. What are the three major categories of the GoF design patterns? Give two examples from each category.
Understand	2. Explain the role of the Factory Method Pattern in separating object creation from object usage.
Understand	3. Explain how a Proxy object controls access to a Real Subject.
Apply	4. A web application loads very large images, but the images should only be loaded when they are actually displayed. Which design pattern would you apply? Explain your approach.
Analyze	5. Analyze a design where a program creates a separate subclass for every possible combination of additional features. Explain why this design may become difficult to maintain and identify a suitable pattern.
Evaluate	6. Compare inheritance-based extension with the Decorator Pattern. Evaluate which approach is more suitable when responsibilities need to be added dynamically at runtime.
Create	7. Create a Java notification system in which a basic notification can dynamically be extended with SMS, email, and push-notification capabilities using the Decorator Pattern.

Bloom's Level	Question
Remember	1. Define the Adapter, Composite, Proxy, and Decorator Patterns.
Understand	2. Explain the purpose of Pattern Languages and how multiple design patterns can work together.
Understand	3. Explain why design patterns are considered reusable design solutions rather than ready-made programs.
Apply	4. An application needs to communicate with an old payment service whose interface is incompatible with the application's current payment interface. Select and apply an appropriate design pattern.
Analyze	5. Analyze the following situation: a client directly communicates with five subsystem classes to perform one high-level operation. Identify the design problem and explain how the Facade Pattern can improve it.
Evaluate	6. Evaluate the following statement: **“The Singleton Pattern should be used whenever an object is accessed from multiple parts of a program.”** Give reasons for your answer.
Create	7. Design an e-commerce system that uses at least **two appropriate design patterns** from Module 1. Explain why you selected each pattern and describe how the patterns cooperate.

Module 2: Behavioral Design Patterns

BDP: Behavioural design patterns focus on how objects communicate and interact with each other.

They define how responsibilities and algorithms are distributed among objects.

They help make communication between objects: Flexible; Maintainable; Loosely coupled; Easier to extend.

Major Behavioral Patterns:

1. Template Method Pattern
2. Chain of Responsibility Pattern
3. State Pattern
4. Command Pattern
5. Observer Pattern

Key idea: Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

WHAT ARE BEHAVIOURAL DESIGN PATTERNS?

They define how responsibillities and algorithms are distributed among objects.

Behavioural design patterns focus on how objects **communicate** and **interact** with each other.

THEY HELP MAKE COMMUNICATION BETWEEN OBJECTS

PLEXIBLE
Easily adapt to changing requirements.

MAINTAINABLE
Changes in one part do not affect others.

LOOSELY COUPLED
Objects depend on each other minimally.

EASIER TO EXTEND
New behavior can be added with minimal changes.

KEY IDEA

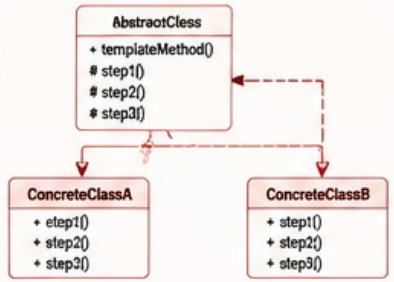
Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

MAJOR BEHAVIOURAL PATTERNS (Examples)

1. TEMPLATE METHOD PATTERN

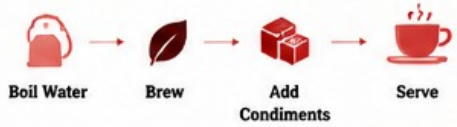
Defines the skeleton of an algorithm in a base class but lets subclasses override specific steps of the algorithm without changing its structure.

UML DIAGRAM



EXAMPLE

Making tea or coffee follows the same steps (boil water, brew, add condiments) but some steps vary.



JAVA CODE EXAMPLE

```
// Abstract Class
abstract class Beverage {
  // Template Method
  final void prepareRecipe() {
    boilWater();
    brew();
    addCondiments();
    serve();
  }
  void boilWater() {
    System.out.println("Boiling ");
  [water";
  abstract void brew();
  abstract void addCondiments();
  void serve() {
   System.out.println("Serving drink");
}
```

```
// Concrete Class A
class Tea extends Beverage {
  void brew() {
    System.out.println("Steeping tea");
  }
  void addCondiments() {
    system.out.println("Adding lemon");
  }
```

```
// Concrete Class B
class Coffee extends Beverage {
  void brew() {
    System.out.println("Dripping coffee");
  }
  void addCondiments() {
    dtotear.oat,t.prtnt("-hing
  ("wntchico("Adding sugar and milk"));
}
```

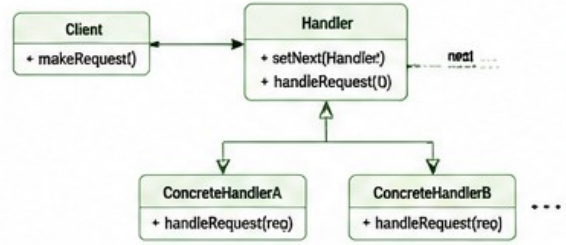
USE WHEN: You have an algorithm with fixed steps, but some steps vary.

BENEFITS: Code reuse, avoids duplication, easy to maintain.

2. CHAIN OP RESPONSIBILITY PATTERN

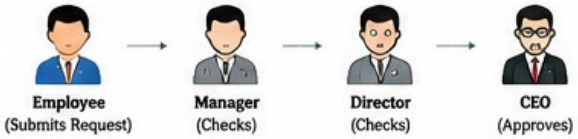
Passes a request along a chain of handlers until one of them handles the request.

UML DIAGRAM



EXAMPLE

A leave request is handled by Manager, then Director, then CEO if needed.



JAVA CODE EXAMPLE

```
// Handler Interface
abstract class Handler {
  protected Handler next;
  public void setNext(Handler next) {
    this.next = next;
  public abstract void handleRequest(int
    amount);
}
```

```
// Concrete Handlers
class Manager extends Handler {
  public void handleRequest(int amount) {
    if (amount == 1000) {
      System.out.println("Manager approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
  }
```

```
class Director extends Handler {
  public void handleRequest(int amount) {
    if (amount == 5000) {
      System.out.println("Director approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
  }
}
class CEO extends Handler {
  public void handleRequest(int amount) {
    if (amount == 5000) {
      System.out.println("CEO approved: " + amount);
    } else if (next != null) next.handleRequest(amount);
    }
  }
```

```
// Client
public class Client {
  public static void main(String[] args) {
    Handler director = new Manager();
    Handler ceo = new CEO();
    manager.setNext(director);
    manager.handleRequest(800);   // Manager approves
    manager.handleRequest(3000);  // Director approves
    manager.handleRequest(10000); // CEO approves
  }
```

USE WHEN: More than one object can handle a request, and the handler is not known in advance.

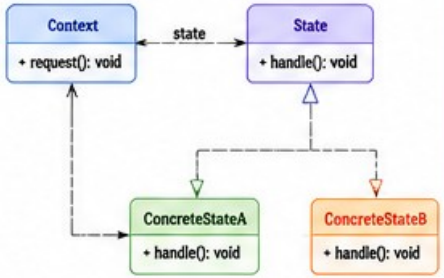
BENEFITS: Decouples sender and receiver, easy to add new handlers, follows OCP.

Key Idea: Behavioural patterns focus on object communication and behavior, rather than object creation or structural organization.

3. STATE PATTERN

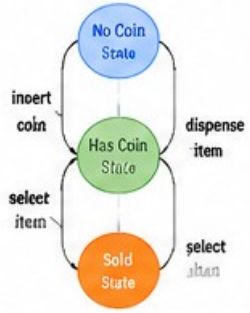
Allows an object to alter its behavior when its internal state changes. The object will appear to change its class.

UML DIAGRAM



EXAMPLE

Vending Machine



JAVA CODE EXAMPLE

```
// State Interface
interface State {
  void insertCoin();
  void selectItem();
  void dispense();
}
```

```
// Concrete State: NoCoinState
class NoCoinState implements State {
  private VendingMachine machine;
  public NoCoinState(VendingMachine m) {
    this.machine = m;
  }
  public void insertCoin() {
    System.out.println("Coin inserted.");
    machine.setState(machine.getHasCoinState());
  }
  public void selectItem() {
    System.out.println("Insert coin first.");
  }
  public void dispense() {
    System.out.println("Insert coin first.");
  }
}
```

```
// Context
class VendingMachine {
  private State noCoinState;
  private State hasCoinState;
  private State soldState;
  private State state;
  public VendingMachine() {
    noCoinState = new NoCoinState(this);
    hasCoinState = new HasCoinState(this);
    soldState = new SoldState(this);
    state = noCoinState;
  }
  public void setState(State s) { state = s; }
  public State getHasCoinState() { return
  public void insertCoin() { state.insertCoin(); }
  public void selectItem() { state.selectItem(); }
  public void dispense() { state.dispense(); }
}
```

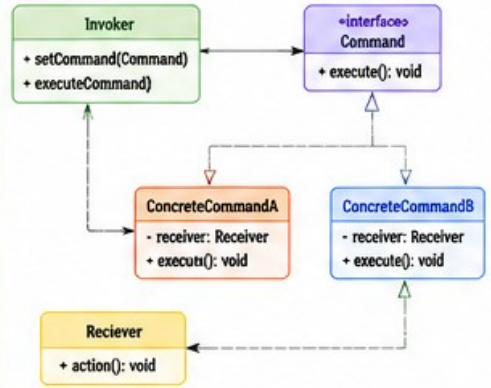
How it works: The context delegates the request to the current state object. When state changes, a different state object handles the requests.

Benefits: Eliminates large conditional statements, easy to add new states.

4. COMMAND PATTERN

Encapsulates a request as an object, thereby letting you parameterize clients with different requests, queue or log requests, and support undo operations.

UML DIAGRAM



EXAMPLE

Remote Control



Invoker (Press Button) — Receiver (TV)

- Button 1 → Turn On TV
- Button 2 → Turn Off TV
- Undo → Previous Command

JAVA CODE EXAMPLE

```
// Command Interface
interface Command {
  void execute();
  void undo();
}
```

```
// Concrete Command: TurnOn
class TurnOnCommand implements Command {
  private TV tv;
  private boolean wasOn;
  public TurnOnCommand(TV tv) {
    this.tv = tv;
  }
  public void execute() {
    wasOn = tv.isOn();
    tv.turnOn();
  }
  public void undo() {
    if(!wasOn) tv.turnOff();
  }
```

```
// Receiver
class TV {
  private boolean on = false;

  public void turnOn() {
    on = true;
    System.out.println("TV is ON");
  }
  public void turnOff() {
    on = false;
    System.out.println("TV is OFF");
  }
  public boolean isOn() { return
    on; }
}
```

```
// Invoker
class RemoteControl {
  private Command command;
  private Command lastCommand;
  public void setCommand(Command cmd) {
    this.command = cmd;
  }
  public void pressButton() {
    if(command != null) {
      command.execute();
      lastCommand = command;
    }
  }
  public void pressUndo() {
    if(lastCommand != null) {
```

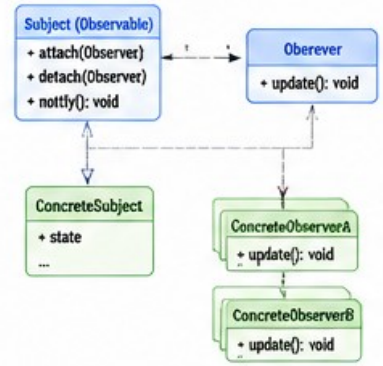
How it works: The invoker sends a request by calling execute() on a command object. The command knows how to perform the action on the receiver and can undo it.

Benefits: Decouples sender and receiver, supports undo/redo, queuing, logging, and macro commands.

5. OBSERVER PATTERN

Defines a one-to-many dependency so that when one object changes state, all its dependents are notified and updated automatically.

UML DIAGRAM



EXAMPLE

News Agency



JAVA CODE EXAMPLE

```
// Subject Interface
interface Subject {
  void attach(Observer o);
  void detach(Observer o);
  void notifyObservers();
}

// Observer Interface
interface Observer {
  void update(String news);
}
```

```
// Concrete Subject
class NewsAgency implements Subject {
  private List<Observer> observers =
    new ArrayList<>();
  private String news;

  public void attach(Observer o) {
    observers.add(o);
  public void detach(Observer o) {
    observers.remove(o);
  public void setNews(String news) {
    this.news =
    notifyObservers();
  }
  public void notifyObservers() {
    for(Observer o : observers)
      o.update(news);
  }
}
```

```
// Concrete Observer
class EmailSubscriber implements Observer {
  private String email;
  public EmailSubscriber(String email) {
    this.email = email;
  public void update(String news) {
    System.out.println("Email to " + email
      + ": " + news);
  }
}
// Another Observer
class SMSSubscriber implements Observer {
  private String phone;
  public SMSSubscriber(String phone) {
    this.phone = phone;
  public void update(String news) {
    System.out.println("SMS to " + phone
      + ": " + news);
  }
}
```

How it works: When the subject's state changes, it notifies all registered observers, and they update themselves.

Benefits: Loosely coupled, supports broadcast communication, easy to add new observers.

1. TEMPLATE METHOD PATTERN

Definition: The Template Method Pattern **defines the skeletom of an algorithm in a superclass** while allowing **subclasses to redefine certain steps** of the algorithm without **changing its overall structure.**

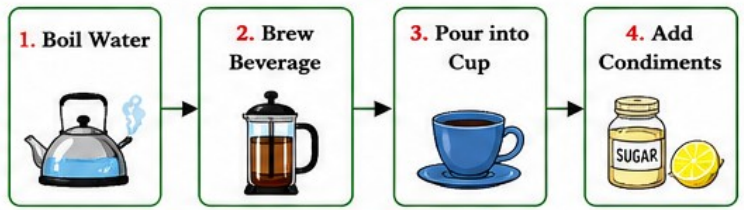
≫ Main Idea

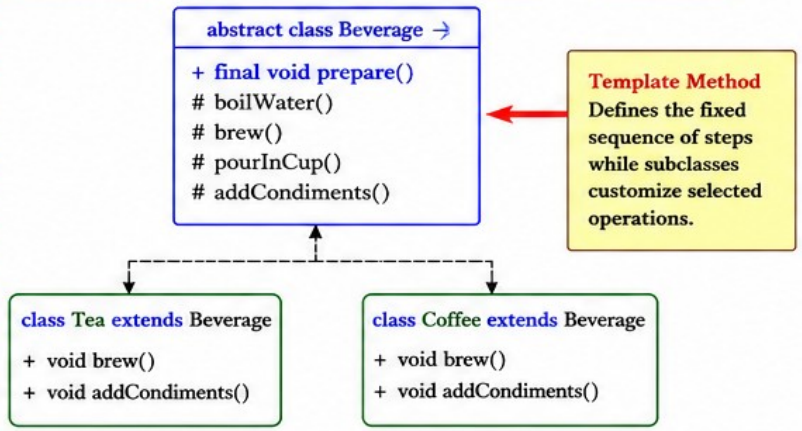
- Algorithm **structure is fixed,** but individual **steps can vary.**

Example: Preparing a Beverage

- A general process may be: Prepare Beverage → Boil Water → Brew Beverage → Pour into Cup → Add Condiments
- **For tea:** Boil Water → Steep Tea → Pour → Add Lemon
- **For coffee:** Boil Water → Brew Coffee → Pour → Add Sugar/Milk
- The overall algorithm remains the **same,** but some steps differ.

Structure: Beverage → Tea / Coffee





Sample Code (Java)

```
abstract class Beverage {
    final void prepare() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();
    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }
    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

```
class Tea extends Beverage {
    void brew() { System.out.println("Steeping tea"); }
    void addCondiments() { System.out.println("Adding lemon"); }
}
```

```
class Coffee extends Beverage {
    void brew() { System.out.println("Brewing coffee"); }
    void addCondiments() { System.out.println("Adding sugar and milk");}
}
```

Benefits

- ✓ Reuses common algorithm structure.
- ✓ Avoids duplicate code.
- ✓ Allows subclasses to customize specific steps.
- ✓ Maintains a consistent algorithm.
- ✓ Supports code reuse through inheritance.

When to Use

- ≫ Several classes follow the same overall process.
- ≫ Some steps differ between classes.
- ≫ You want to avoid repeating common algorithm code.

How It Works – Execution Example

```
Tea t = new Tea();
t.prepare();
```

Output:

1. Boiling water
2. Steeping tea
3. Pouring into cup
4. Adding lemon



```
Coffee c = new Coffee();
c.prepare();
```

Output:

1. Boiling water
2. Brewing coffee
3. Pouring into cup
4. Adding sugar and milk



In Short:

Template Method Pattern is like a recipe (algorithm) with **fixed steps,** where some steps are **common** (boil water, pour in cup) and some steps are **optional/variable** (brew, add condiments). Subclasses provide their own implementation for the variable steps, while the **overall process remains the same.**

Real-World Analogy

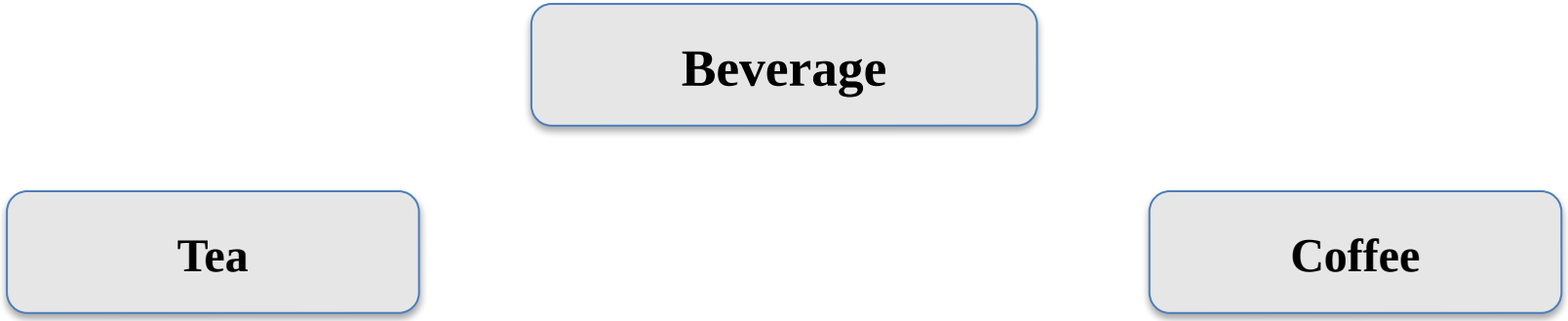
Cooking is the same process, but the ingredients and some steps change depending on the dish.



1. Template Method Pattern — Definition

- **The Template Method Pattern defines the skeleton of an algorithm in a superclass while allowing subclasses to redefine certain steps of the algorithm without changing its overall structure.**
- **Main Idea:** Algorithm structure is fixed, but individual steps can vary.
- **Example:** Preparing a Beverage
- **A general process may be:** Prepare Beverage → Boil Water → Brew Beverage → Pour into Cup → Add Condiments
- **For tea:** Boil Water → Steep Tea → Pour → Add Lemon
- **For coffee:** Boil Water → Brew Coffee → Pour → Add Sugar/Milk
- The overall algorithm remains the same, but some steps differ.

Template Method Pattern — Structure and Benefits



- **Structure:** Beverage → Tea / Coffee
- **Template Method:** Defines the fixed sequence of steps while subclasses customize selected operations.
- **Benefits:** Reuses common algorithm structure; Avoids duplicate code; Allows subclasses to customize specific steps; Maintains a consistent algorithm; Supports code reuse through inheritance.
- **When to Use:** Several classes follow the same overall process; Some steps differ between classes; You want to avoid repeating common algorithm code.

Template Method Pattern — Java Example

```
abstract class Beverage {

    final void prepare() {
        boilWater();
        brew();
        pourInCup();
        addCondiments();
    }

    abstract void brew();
    abstract void addCondiments();

    void boilWater() {
        System.out.println("Boiling water");
    }

    void pourInCup() {
        System.out.println("Pouring into cup");
    }
}
```

2. Chain of Responsibility Pattern — Definition

- ✓ **The Chain of Responsibility Pattern passes a request through a chain of handlers until one handler can process the request**.
- ✓ **Main Idea:** Give multiple objects a chance to handle a request without coupling the sender to a specific receiver.
- ✓ **Example:** University Complaint System: Student → Class Teacher → Department Chairman → Dean → Vice Chancellor
- ✓ A complaint is passed along the chain until the appropriate authority handles it.
- ✓ **Basic Structure:** Client → Handler A → Handler B → Handler C → Handler D

✓ The Chain of Responsibility Pattern **passes a request through a chain of handlers** until one handler **can process the request.**

Main Idea

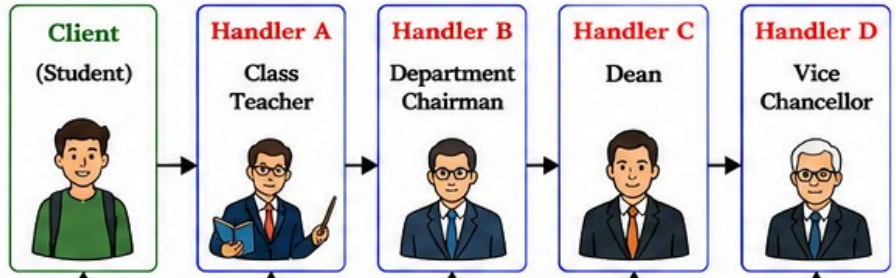
Give multiple objects a chance to handle a request **without coupling** the sender to a specific receiver.

Example: University Complaint System

Student → **Class Teacher** → **Department Chairman** → **Dean** → **Vice Chancellor**

➢ A complaint is passed along the chain until the **appropriate authority** handles it.

Basic Structure



If cannot handle, pass the request to the next handler.

If handled, the chain stops.

Sample Code (Java)

```
abstract class Handler {
    protected Handler next;
    public void setNext(Handler next) {
       this.next = next;
    }
   public abstract void handleRequest(String request);
}

class ClassTeacher extends Handler {
   public void handleRequest(String request) {
      if (request.equals("academic issue")) {
         System.out.println("Class Teacher handled: " + request);
      } else if (next != null) {
         next.handleRequest(request);
      }
   }
}
// Similarly: DepartmentChairman, Dean, ViceChancellor classes
// Each class decides whether to handle or pass to next handler
```

Benefits

- ✓ Reduces coupling between sender and receiver.
- ✓ Multiple objects can handle a request.
- ✓ Handler order can be changed easily.
- ✓ New handlers can be added without modifying the client.
- ✓ Supports flexible request processing.

How It Works – Step by Step

1. **Student** (client) sends a complaint.
2. Class Teacher checks: Can I handle?
   - If **yes** → handles and **stops**.
   - If **no** → passes to next (Department Chairman).
3. **Department Chairman** checks: Can I handle?
   - If **no** → passes to Dean.
4. **Dean** checks: Can I handle?
   - If **no** → passes to Vice Chancellor.
5. **Vice Chancellor** checks: Can I handle?
   - If **yes** → handles and **stops**.
6. If **no** handler can process → request remains **unhandled**.

Example Flow

Complaint: "hostel facility issue"
↓
Stodent
↓
Class Teacher (cannot handle)
↓
Department Chairman (cannot handle)
↓
Dean (cannot handle)
↓
Vice Chancellor (handles)

✅ **Request Handled**

Common Applications

- ✓ Approval systems
- ✓ Exception handling
- ✓ Customer support systems
- ✓ Authentication / authorization
- ✓ Event processing

In Short

Chain of Responsibility Pattern forms a "**chain**" of objects.

A request **travels** along the chain until an object handles it.

If no one handles it, the request **remains unhandled**.

Real-Life Analogy



When you have a problem in a **bank**,
you first talk to the **staff**.
If they cannot solve it,
they forward you to the **manager**,
then to the **branch head**,
and finally to the **regional manager** if needed.

Key Points to Remember

- ✓ Request passes along a **chain**.
- ✓ Each handler decides: **handle** or pass.
- ✓ Loose coupling between client and handlers.
- ✓ Easy to add, remove, or reorder handlers.
- ✓ More flexible and maintainable code.

Chain of Responsibility — Java Structure and Benefits

Benefits:

- ✓ Reduces coupling between sender and receiver;
- ✓ Multiple objects can handle a request;
- ✓ Handler order can be changed easily;
- ✓ New handlers can be added without modifying the client;
- ✓ Supports flexible request processing.
- ✓ Common Applications: Approval systems;
- ✓ Exception handling; Customer support systems;
- ✓ Authentication/authorization; Event processing.

3. State Pattern — Definition and Example

- ✓ **The State Pattern allows an object to change its behavior when its internal state changes**.
- ✓ **Main Idea:** The object appears to change its class when its state changes.
- ✓ **Example:** Online Order
- ✓ An order can have different states: Pending; Paid; Shipped; Delivered.
- ✓ The behavior of the order depends on its current state.



STATE PATTERN

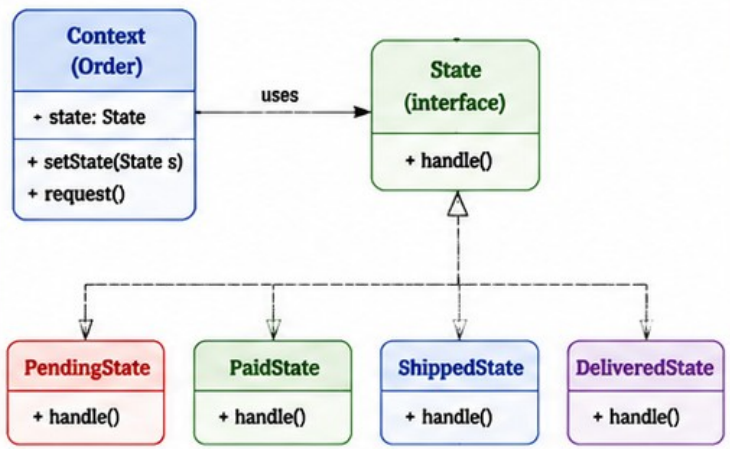
✔ The State Pattern **allows an object to change its behavior when its internal state changes.**

✔ **Main Idea:** The object appears to change its class when its state changes.

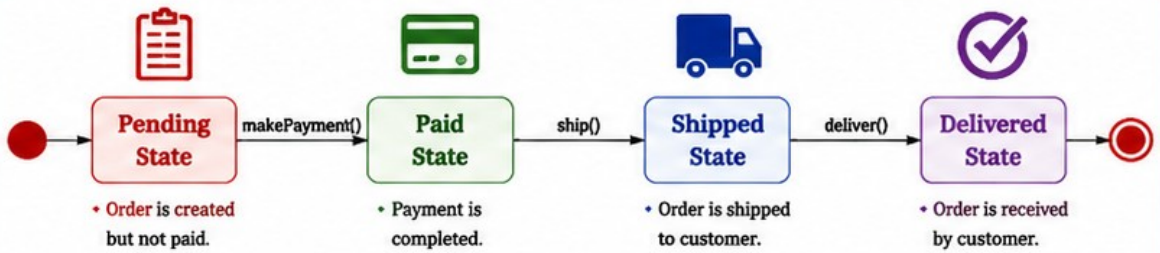
EXAMPLE: ONLINE ORDER

- An order can have different states: **Pending**; **Paid**; **Shipped**; **Delivered**.
- The behavior of the order depends on its current state.

STRUCTURE



STATE TRANSITION DIAGRAM (ONLINE ORDER)



JAVA CODE EXAMPLE

```
interface State {
    void handle();
}
class PendingState implements State {
    public void handle() {
        System.out.println("Order is pending");
    }
}
class PaidState implements State {
    public void handle() {
        System.out.println("Order has been paid");
    }
```

```
class Order {
    private State state;
    public Order() {
        state = new PendingState();
    }
    public void setState(State state) {
        this.state = state;
    }
    public void request() {
        state.handle();
    }
}
```

```
// Client
public class Client {
    public static void main(String[] args) {
        Order order = new Order();
        order.request();  // Order is pending
        order.setState(new PaidState());
        order.request();  // Order has been paid
        order.setState(new ShippedState());
        order.request();  // Order is shipped
        order.setState(new DeliveredState());
        order.request();  // Order is delivered
    }
}
```

BENEFITS

- **Encapsulates state-specific behavior.**
- **Reduces large if-else or switch statements.**
- **Makes state transitions easier to manage.**
- **New states can be added easily.**
- **Improves maintainability.**

WHEN TO USE

Use State Pattern when an object's behavior changes significantly depending on its current state.

HOW IT WORKS

1. The **Context** (Order) holds a reference to a **State** object.
2. The current **State** object handles the **request()** based on its behavior.
3. When a state transition occurs, the **Context** changes its **State** object.
4. The new **State** object handles future requests accordingly.

State Pattern — Structure and Benefits



- **Structure:** Context → Current State → Pending / Paid / Shipped
- **Benefits:** Encapsulates state-specific behavior; Reduces large if-else or switch statements; Makes state transitions easier to manage; New states can be added easily; Improves maintainability.
- **When to Use:** Use State Pattern when an object's behavior changes significantly depending on its current state.

State Pattern — Java Example

```
interface State {
    void handle();
}

class PendingState implements State {
    public void handle() {
        System.out.println("Order is pending");
    }
}

class PaidState implements State {
    public void handle() {
        System.out.println("Order has been paid");
    }
}
```

4. Command Pattern — Definition

The Command Pattern encapsulates a request as an object.

This allows requests to be: Stored; Scheduled; Queued; Executed; Undone; Redone; Logged.

Main Components: Client/Sender → Command → Invoker → Receiver → Performs Task

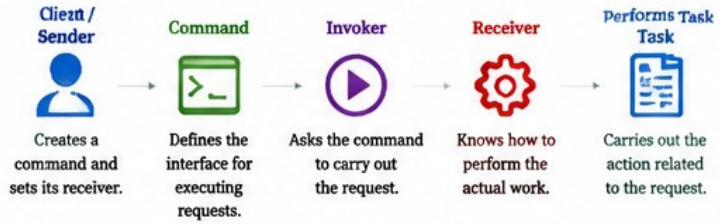
Example: Text Editor: User → Paste Button → PasteCommand → Document

4

COMMAND PATTERN

Definition: The Command Pattern encapsulates a request as an object.
This allows requests to be: **Stored**; **Scheduled**; **Queued**; **Executed**; **Undone**; **Redone**; **Logged**.

1. MAIN COMPONENTS



Example: Text Editor



2. MAJOR PURPOSES

1. **Store and Schedule Requests**
Commands can be stored in lists or queues and executed later.



2. **Undo and Redo**
Commands can maintain a history so that previous operations can be undone and redone.



3. **Logging**
Commands can be stored in a log so recent operations can potentially be replayed after a system failure.

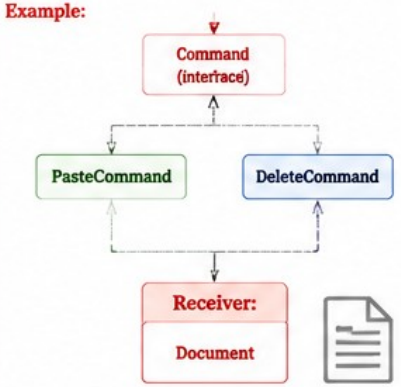


3. COMMAND INTERFACE / SUPERCLASS

```
interface Command {
  void execute();
  void unexecute();
  boolean isReversible();
}
```

- **execute()** → performs the command
- **unexecute()** → reverses the command
- **isReversible()** → determines whether the command can be undone

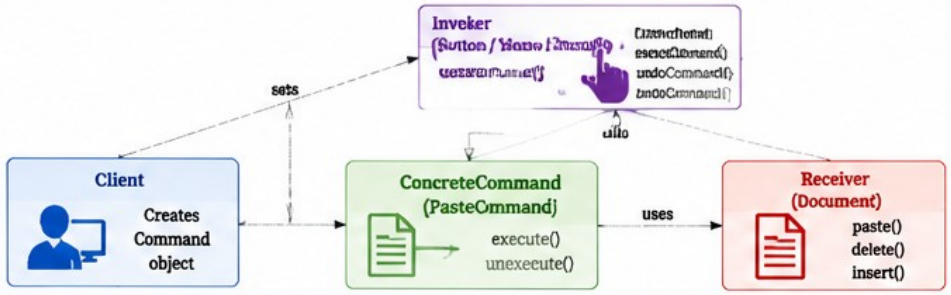
Example:




```
class PasteCommand extends Command {
  private Document doc;
  private String content;
  public void execute() { doc.paste(content); }
  public void unexecute() { doc.delete(content); }
  public boolean isReversible() { return true; }
}
class DeleteCommand extends Command {
  private Document doc;
  private int index; private String backup;
  public void execute() { backup = doc.delete(index); }
  public void unexecute() { doc.insert(index, backup); }
  public boolean isReversible() { return true; }
}
```

4. HOW IT WORKS (FLOW)

1. Client creates a concrete command object and sets the receiver.
2. Invoker stores the command object.
3. Invoker calls execute() on the command.
4. Command calls the receiver to perform the actual task.
5. (Optional) Command can be unexecute() to undo the task.



5. BENEFITS

1. **Commands become objects** — Requests can be manipulated as objects.
2. **Supports Undo/Redo** — Commands can maintain history and reversal operations.
3. **Supports Queueing** — Commands can be placed in queues and executed later.
4. **Reduces Coupling** — The command object separates the requester from the receiver.
5. **Separates UI from Application Logic** — Button clicks can create command objects instead of directly containing application logic.
6. **Easy to Maintain** — New commands can be added independently.

Command Pattern — Purpose

Major Purposes

1. Store and Schedule Requests

Commands can be stored in lists or queues and executed later.

Example: Create Alarm Command → Command Queue → Scheduled Time → Execute Alarm

2. Undo and Redo

Execute → History List → Undo → Redo List

Commands can maintain a history so that previous operations can be undone and redone.

3. Logging

Commands can be stored in a log so recent operations can potentially be replayed after a system failure.



Command Pattern — Structure

Command Interface / Superclass

- execute() → performs the command
- unexecute() → reverses the command
- isReversible() → determines whether the command can be undone

Example: Command → PasteCommand / DeleteCommand → Receiver: Document

Command Pattern — Benefits

1. **Commands become objects** — Requests can be manipulated as objects.
2. **Supports Undo/Redo** — Commands can maintain history and reversal operations.
3. **Supports Queueing** — Commands can be placed in queues and executed later.
4. **Reduces Coupling** — The command object separates the requester from the receiver.
5. **Separates UI from Application Logic —** Button clicks can create command objects instead of directly containing application logic.
6. **Easy to Maintain** — New commands can be added independently.



5. Observer Pattern — Definition and Example

The Observer Pattern defines a one-to-many relationship in which a Subject maintains a list of Observers and automatically notifies them when its state changes.

Main Idea: One object changes → many dependent objects are notified.

Example: Blog Subscription: Blog (Subject) → Subscriber 1 / Subscriber 2 / Subscriber 3 (Observers)

When the blog changes: Blog Updated → notify() → update() on all subscribers.

```
interface Observer { void update(); }

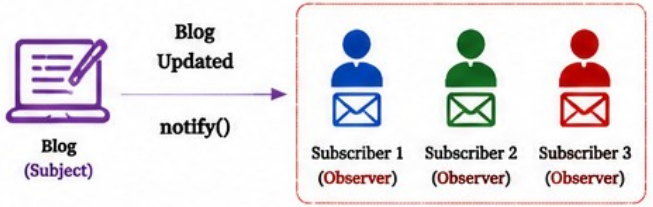
Subject: registerObserver(); unregisterObserver(); notify();
```

OBSERVER PATTERN

Definition: The Observer Pattern defines a one-to-many relationship in which a Subject maintains a list of Observers and automatically notifies them when its state changes.
Main Idea: One object changes → many dependent objects are notified.

1. EXAMPLE: BLOG SUBSCRIPTION

Blog (Subject) → Subscriber 1 / Subscriber 2 / Subscriber 3 (Observers)



2. MAIN ROLES AND OPERATIONS

Subject
Maintains a collection of observers.
Main operations:
- registerObserver(Observer o)
- unregisterObserver(Observer o)
- notify()

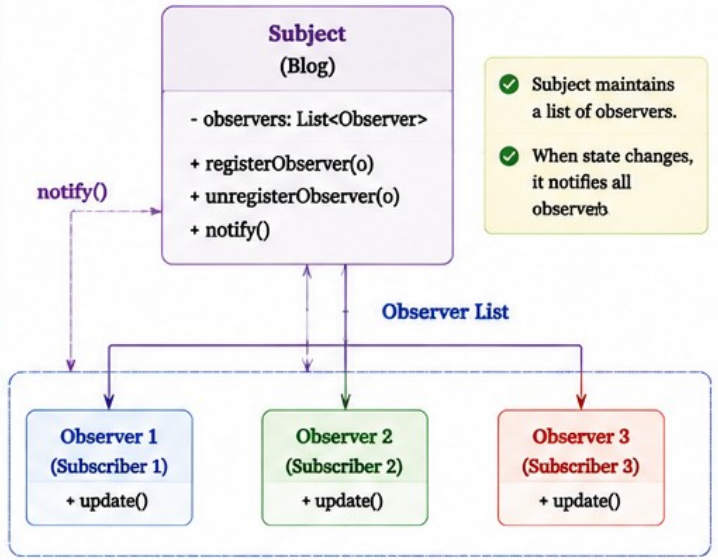
Observer
Defines update().

3. INTERFACE (EXAMPLE)

```
interface Observer {
    void update();
}

interface Subject {
    void registerObserver(Observer o);
    void unregisterObserver(Observer o);
    void notify();
}
```

4. STRUCTURE DIAGRAM



notify() calls update() on all registered observers

5. DYNAMIC SUBSCRIBE / UNSUBSCRIBE



Observers can subscribe and unsubscribe dynamically.
The Subject does not need to know the concrete classes of observers.

6. JAVA CODE EXAMPLE

```
import java.util.*;

interface Observer {
    void update();
}

class Subject {
    private ArrayList<Observer> observers =
            new ArrayList<>();

    public void registerObserver(Observer o) {
            observers.add(o);
    }

    public void unregisterObserver(Observer o) {
            observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o :                ) {
            o.update();
        }
    }
}
```

7. KEY BENEFITS

- Establishes a one-to-many dependency between objects.
- Loose coupling between Subject and Observers.
- Automatic notification when state changes.
- Supports dynamic addition and removal of observers.
- Easy to extend: new observers can be added without changing the subject.
- Widely used in event handling, UI frameworks, messaging systems, etc.

Use Observer Pattern when a change in one object requires changing many others, and the number of dependent objects is unknown or changes at runtime.

Real World Examples: GUI event handling, stock price updates, weather stations, news notifications, social media feeds, etc.

Key Takeaway: One changes, many get notified!
Keep your system flexible, scalable and decoupled.

Observer Pattern — Structure

Subject: Maintains a collection of observers.

Main operations: registerObserver(); unregisterObserver(); notify().

Observer: Defines update().

Structure: Subject → Observer List → notify() → Observer 1, Observer 2, Observer 3 → update()

Observers can subscribe and unsubscribe dynamically.

Observer Pattern — Java Example

```
class Subject {
    private ArrayList<Observer> observers =
        new ArrayList<>();

    public void registerObserver(Observer o) {
        observers.add(o);
    }

    public void unregisterObserver(Observer o) {
        observers.remove(o);
    }

    public void notifyObservers() {
        for (Observer o : observers) {
            o.update();
        }
    }
}
```

Observer Pattern — Benefits & Applications

Benefits: Automatically notifies dependent objects; Reduces direct dependency between subject and observers; Supports one-to-many communication; Observers can subscribe/unsubscribe dynamically; Makes notification management easier; Useful when many objects depend on one object's state.

Applications: Blog/news subscriptions; GUI event handling; Notification systems; Stock-price monitoring; Weather monitoring; Real-time dashboards.

Comparison of Behavioural Patterns

Template Method → How an algorithm is performed (Same algorithm, variable steps)

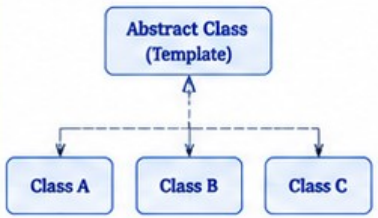
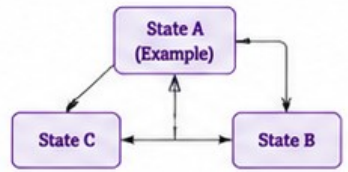
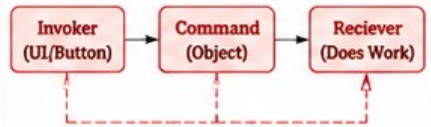
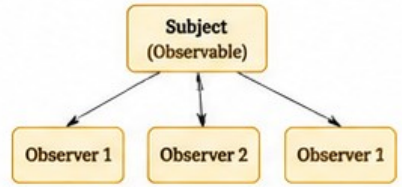
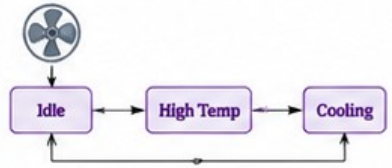
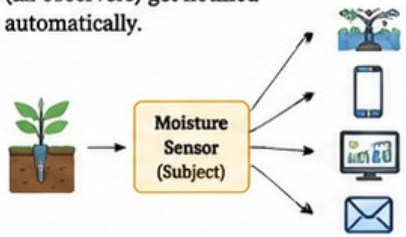
Chain of Responsibility → Who will handle the request? (Handler chain)

State → What should I do in this state? (State-dependent behavior)

Command → Turn a request into an object (Request → Object)

Observer → Who needs to know about a change? (One-to-many notification)

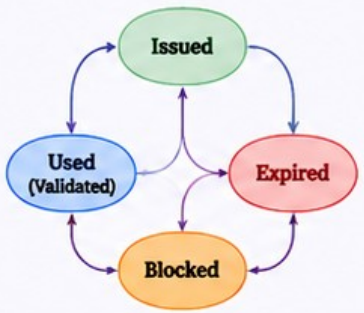
COMPARISON OF BEHAVIORAL DESIGN PATTERNS

	1. TEMPLATE METHOD PATTERN	2. CHAIN OF RESPONSIBILITY PATTERN	3. STATE PATTERN	4. COMMAND PATTERN	5. OBSERVER PATTERN
What it is (All about)	**How** an algorithm is performed (Same algorithm, variable steps)	**Who** will handle the request? (Handler chain)	**What** should I do in this state? (State-dependent behavior)	**Turn** a request into an object (Request → Object)	**Who** needs to know about a change? (One-to-many notification)
Core Idea	Define the skeleton of an algorithm in a base class and let subcleases implement specific steps.	Pass a request along a chain of handlers until one handler processes it.	Allow an object to change its behavior when its internal state changes.	Encapsulate a request as an object so it can be stored, queued, executed, undone, or logged.	Define a one-to-many dependency so that when one object changes state, all its dependents are notified.
Structure (Visual)		 Request comes → Passed along until handled	 Object changes state → behavior changes	 Execute / Undo / Redo / Log	 Change in Subject → notfiy() → update() all observers
Real-life Example (Agricultural Automation System)	**Irrigation Scheduling System** The system has a fixed algorithm: Check soil moisture → Check weather forecast → Decide irrigation → Log result. Different crop types (Rice, Wheat, Maize) inplement how they calculate "Decide irrigation" step differently. 	**Alarm Handling System** When an alarm occurs (e.g., high temperature), it goes through a chain: Sensor Check → Local Controller → Farm Manager → Cloud Server. Whoever can handle the alarm, handles it. 	**Greenhouse Ventilation System** The fan behavior depends on the state: Idle → Low Temp → High Temp → Cooling. The system changes its state based on temperature and acts accordingly. 	**Farm Operation Control System** User presses "Start Pump" button. A StartPumpCommand object is created and sent to the pump controller. The command can be stored, queued, executed later, undone (stop pump) or logged. 	**Soil Moisture Monitoring System** When soil moisture value changes, many components should know: Irrigation Controller, Mobile App, Dashboard, and SMS Alert Service (all observers) get notified automatically. 
Best When to Use	When the overall algorithm is fixed but certain steps vary between implementations.	When multiple objects can handle a request and you don't know in advance who will handle it.	When an object's behavior changes based on its internal state.	When you need to parameterize objects with operations, support undo/redo, queue, or logging.	When a change in one object needs to be broadcast to many dependent objects.

DHAKA
Urban Management

Behavioral Design Patterns in a Dhaka Urban Management System Project

One Smart City – Five Behavioral Pattens – Five Real Solutions

Pattern	1. Template Method Pattern	2. Chain of Responsibility Pattern	3. State Pattern	4. Command Pattern	5. Observer Pattern
Key Idea	How an algorithm is performed (Same algorithm, variable steps)	Who will handle the request? (Handler chain)	What should I do in this state? (State-dependent behavior)	Turn a request into an object (Request → Object)	Who needs to know about a change? (One-to-many notification)
Dhaka Project Scenario	**Daily Waste Collection** Different zones of Dhaka follow the same waste collection process, but each zone may implement specific steps differently.	**Citizen Complaint Handling** A complaint is passed through departments until the right authority handles it.	**Bus / Metro Ticket Status** A ticket system behaves differently depending on its current state.	**Traffic Signal Control** Each operation (change signal, turn on camera, open barrier) is a command that can be queued, scheduled, logged and undone.	**Air Quality Monitoring** When air quality changes, all relevant departments and citizens are automatically notified.
How it Works		 Request passes through a chain until the responsible office handles			
Main Participants	Abstract Template (Base Process) → Zone A / Zone B / Zone C	ComplaintHandler → CallCenter → WardOffice → CityCorporation → SpecialAuthority	Ticket (Context) → State (Issued / Used / Expired / Blocked)	Invoker (Traffic App) → Command → Receiver (Traffic Controller)	Subject → Observer List → notify() → All Observers

Code (Simplified)	`abstract class WasteCollection {`
	`  pubic final void collect() {`
	`  planRoute();`
	`  collectWaste(); // variable step`
	`  transport();`
	`  dispose();`
	`  }`
	`  protected abstract void collectWaste();`
	`}`
		`interface ComplaintHandler {`
		`  void setNext(ComplaintHandler h);`
		`  void handle(Complaint c);`
		`}`
			`interface TicketState {`
			`  void process(Card c);`
			`}`
			`class IssuedState implements`
			`  TicketState { ... }`
				`interface Command {`
				`  void execute();`
				`  void unexecute(); }`
				`}`
				`class StartGreenCommand`
				`  implements Command { ... }`
					`class AirQualitySubject {`
					`  private List<Observer> obs =`
					`  new ArrayList<<();`
					`void registerObserver(Observer o) {...}`
					`void notifyObservers() {`
					`  for (Observer o : obs) o.update();`
					`}`

Benefit	Ensures a standard process yet allows zone-specific customization.	Flexible and scalable complaint processing.	Keeps system behavior clear and easy to extend for new states (e.g., refunded).	Supports scheduling, logging and undo/redo of traffic control actions.	Ensures real-time information sharing with all stakeholders automatically.

Smart Dhaka, Smarter Management – Design Patterns Make Urban Systems Flexible, Scalable and Maintainable.

Problem Scenario

Suppose You are designing a system for money withdrawal from bank account using check with the following authorization scenario:

- ✓ For Tk. < 10,000 only cashier's authorization is sufficient to withdraw money from account.
- ✓ For Tk. 10,000 to 10,00,000 authorization from both cashier and senior officer is needed.
- ✓ For Tk > 10,00,000 authorization from Senior officer and Manager is needed.
- ✓ **Implement this behaviour using appropriate software design pattern.**

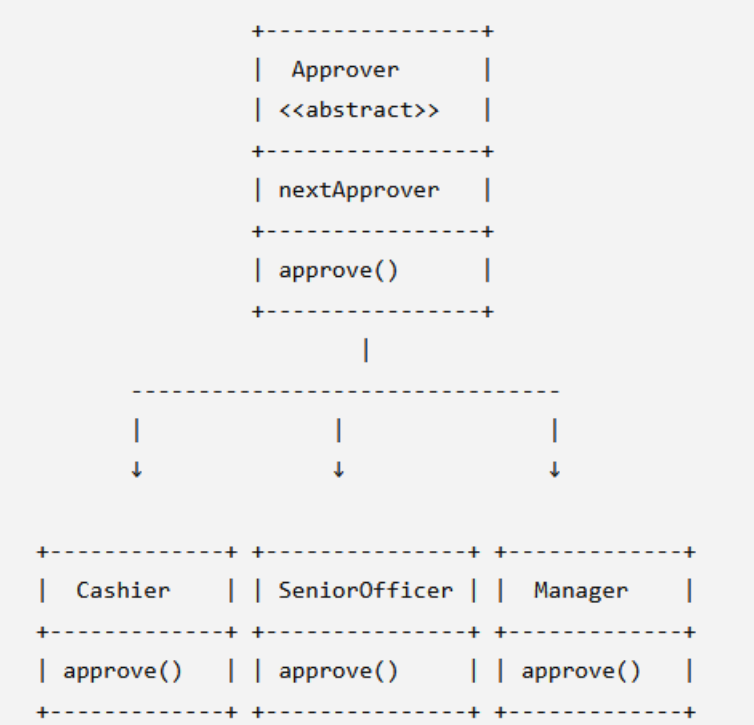
This Scenario is a classic example of the **Chain of Responsibility Design Pattern**.
The reason is:

- ✓ A withdrawal request passes through a sequence of authorization levels.
- ✓ Each authorization object decides whether it can handle the request.
- ✓ If it cannot handle it, it forwards the request to the next authority.
- ✓ The sender does not need to know which object will finally approve the request.

1. Concept

The **Chain of Responsibility pattern** creates a chain of handler objects. Each handler checks whether it can process a request; otherwise, it passes the request to the next handler in the chain.
In this bank withdrawal system:
Cashier | ↓ Senior Officer | ↓ Manager

UML



Abstract Handler: abstract class Approver
{
protected Approver nextApprover;
public void setNextApprover(Approver approver) {
this.nextApprover = approver; }
public abstract void approve(double amount);}

Cashier Class: class Cashier extends Approver{
public void approve(double amount){
if(amount < 10000){
System.out.println("Cashier approved withdrawal of Tk. " + amount);
} else if(nextApprover != null){
nextApprover.approve(amount); } }}

class SeniorOfficer extends Approver{
public void approve(double amount){
if(amount >= 10000 && amount <= 1000000){
System.out.println("Senior Officer approved withdrawal of Tk. " + amount); }
else if(nextApprover != null)
{ nextApprover.approve(amount); } }}

UML and Java Implementation

```
class Manager extends Approver {
 public void approve(double amount) {
 if(amount > 1000000){
          System.out.println(          "Manager approved
withdrawal of Tk. "          + amount);       }
       else {
          System.out.println(          "Withdrawal cannot
be approved");       }    }}
```

Output: Cashier approved withdrawal of Tk. 5000Senior Officer approved withdrawal of Tk. 50000Manager approved withdrawal of Tk. 5000000

```
public class BankSystem {
public static void main(String args[]) {
 Approver cashier = new Cashier();
Approver senior = new SeniorOfficer();
Approver manager = new Manager();       //
Create chain
cashier.setNextApprover(senior);
senior.setNextApprover(manager);       //
Withdrawal requests
cashier.approve(5000);
cashier.approve(50000);
cashier.approve(5000000);    }}
```

Real World Component	Design Pattern Component
Cashier	First Handler
Senior Officer	Second Handler
Manager	Final Handler
Withdrawal Request	Request Object
Authorization Process	Chain of Responsibility

UML and Java Implementation

Why Chain of Responsibility is Appropriate?

1.Flexible authorization flow

If the bank introduces another authority, for example:
Cashier | Senior Officer | Assistant Manager | Manager
we can add a new handler without modifying existing classes.

2.Loose coupling

The withdrawal request does not directly depend on Cashier, Senior Officer, or Manager.

3.Follows Open/Closed Principle

New approval levels can be added without changing existing code.

Problem -2 Scenario

Question 1: Dhaka Urban Management System (DUMS)
Scenario: Smart Citizen Complaint Notification System
The Dhaka Urban Management System receives thousands of citizen complaints every day regarding waste collection, traffic problems, water supply, and emergency services.
When the status of a complaint changes, different users should automatically receive updates:

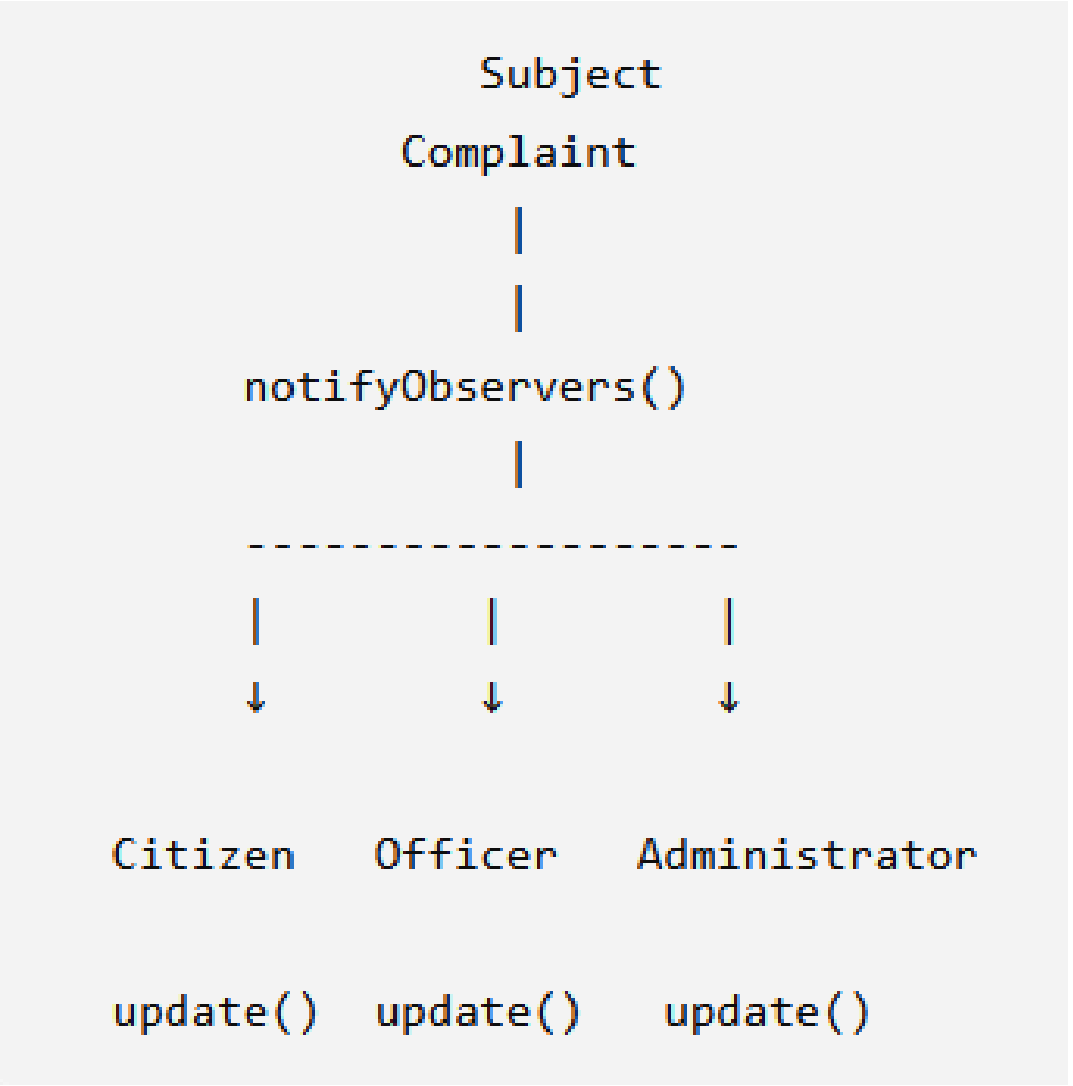
- Citizen receives complaint status notification.
- Department officer receives update information.
- City administrator receives monitoring alerts.

The complaint management system should notify all registered users automatically whenever the complaint status changes.
Requirement:
Implement this behavior using an appropriate software design pattern.

Design Pattern: Observer Pattern
Reason:
The Observer Pattern is suitable because:

- One object (**Complaint**) changes its state.
- Multiple dependent objects (**Citizen, Officer, Administrator**) need automatic updates.
- The complaint system should not tightly depend on specific notification users.

Problem -2 : UML and Java Implementation



Observer Interface

```
interface Observer {
  void update(String message);}
```

Concrete Observer:

```
class Citizen implements Observer {
 public void update(String message){
  System.out.println(       "Citizen Notification: " + message);
 }}
class Officer implements Observer {    public void
update(String message){       System.out.println(       "Officer
Alert: " + message);    }}
class Administrator implements Observer {    public void
update(String message){       System.out.println(       "Admin
Dashboard Update: " + message);    }}
```

Problem -2 : UML and Java Implementation

Subject Class

```
import java.util.*;
class Complaint {
ArrayList<Observer> observers =new ArrayList<>();
public void addObserver(Observer o){
 observers.add(o);}
public void changeStatus(String status){    for(Observer o:
observers){
o.update(       "Complaint status changed to "       + status);    }}}
```

Main Class

```
public class DUMS {
public static void main(String args[]){
Complaint complaint=new Complaint();
complaint.addObserver(new Citizen());
complaint.addObserver(new Officer());
complaint.addObserver(new Administrator());
complaint.changeStatus("Completed");}}
```

Citizen Notification:

```
Complaint status changed to
CompletedOfficer Alert:
Complaint status changed to
CompletedAdmin Dashboard Update:
Complaint status changed to Completed
```

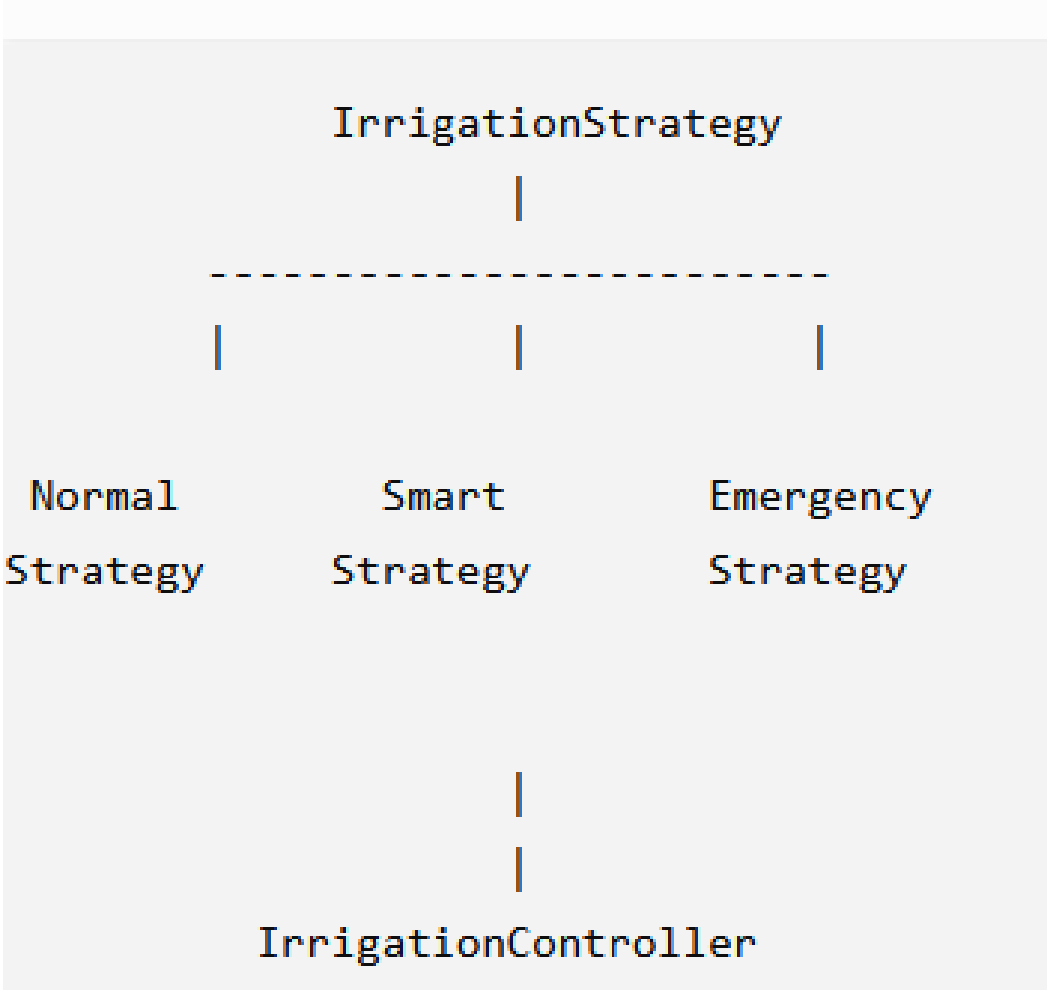
Problem -3 Scenario

Question 2: Agricultural Automation System (AAS)
Scenario: Intelligent Irrigation Strategy Selection
An Agricultural Automation System controls irrigation based on different farming conditions.
The farmer can select different irrigation strategies:
1.Normal irrigation
2.Water-saving irrigation
3.Emergency irrigation during drought
The system should allow changing irrigation algorithms without modifying the main irrigation controller.
Requirement:
Implement this behavior using an appropriate software design pattern.

Solution
Design Pattern: Strategy Pattern
Reason:
Strategy Pattern is suitable because:

- Different irrigation algorithms exist.
- The algorithm can change dynamically.
- The irrigation controller does not need to know the internal implementation.

UML and Java Implementation for Scenario-3



interface IrrigationStrategy
{void irrigate();}

class NormalIrrigation implements IrrigationStrategy{
public void irrigate(){
System.out.println("Normal irrigation activated");}}

class WaterSavingIrrigationimplements
IrrigationStrategy{public void irrigate()
{System.out.println("Water saving irrigation activated");}}

class EmergencyIrrigationimplements
IrrigationStrategy{public void irrigate()
{System.out.println("Emergency irrigation activated");}}

Problem - 3: UML and Java Implementation

Context class:

```
class IrrigationController{
private IrrigationStrategy strategy;
public void setStrategy(IrrigationStrategy
strategy){this.strategy=strategy;}public void
execute(){
strategy.irrigate();}}
```

Citizen Notification:

Water saving irrigation activatedEmergency irrigation activated

Main Class

```
public class FarmSystem{public static void main(String args[])
{IrrigationController controller =new
IrrigationController();controller.setStrategy(new
WaterSavingIrrigation());controller.execute();controller.setStrateg
y(new EmergencyIrrigation());controller.execute();}}
```

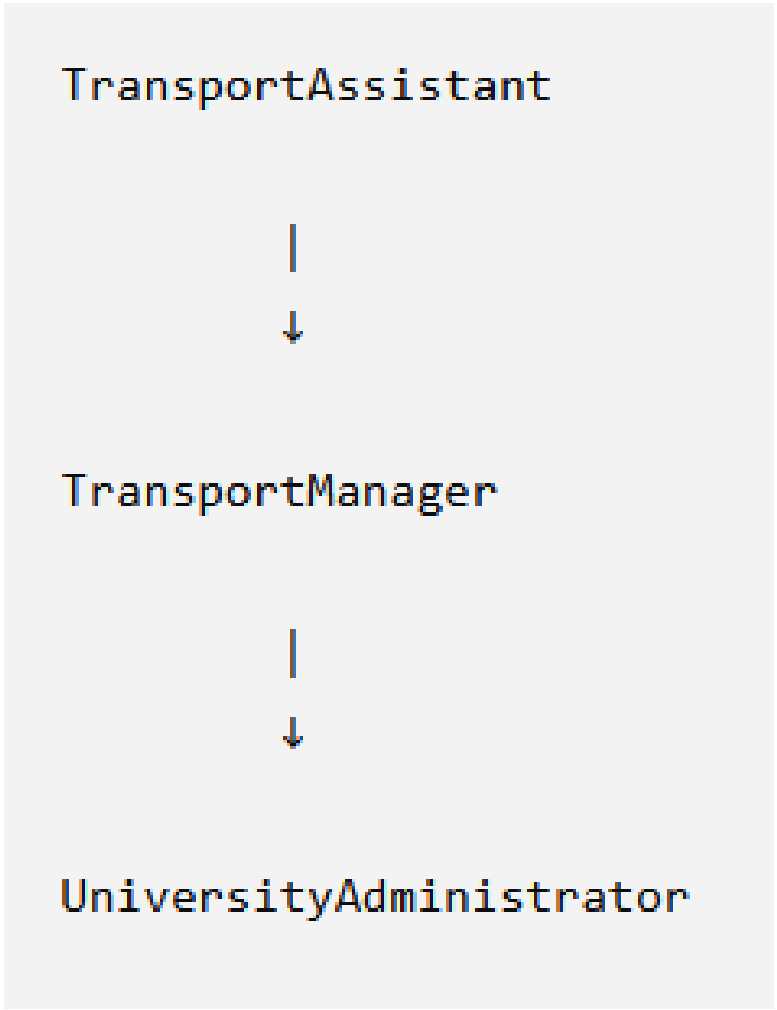
Problem -4 Scenario

Scenario: University Transport Approval System
A smart campus transportation system manages bus ticket approval.
Different transport requests require different approval levels:
•Student transport request below 500 BDT → Transport Assistant approval
•Request from 500 to 5000 BDT → Transport Manager approval
•Request above 5000 BDT → University Administrator approval
The request should automatically move through the approval chain until the responsible authority handles it.
Requirement:
Implement this behavior using an appropriate software design pattern.

Solution
Design Pattern: Chain of Responsibility Pattern

Reason:
The Chain of Responsibility pattern is suitable because:
•Multiple approval levels exist.
•Each handler decides whether it can process the request.
•The request moves to the next authority if required.

UML and Java Implementation for Scenario-4



abstract class Approver{protected Approver next;public void setNext(Approver next){this.next=next;}public abstract void approve(double amount);}

Transport Assistant:
class Assistant extends Approver{public void approve(double amount) {if(amount<500) {System.out.println("Assistant approved request");} else{next.approve(amount);}}}

Transport Manager:
class Manager extends Approver{public void approve(double amount){if(amount<=5000) {System.out.println("Manager approved request");}else{next.approve(amount);}}}

UML and Java Implementation for Scenario-4

Adminstrator:

```
class Administrator extends Approver{public
void approve(double amount)
{System.out.println("Administrator approved
request");}}
```

Main:

```
public class CampusTransport{public static
void main(String args[]){Approver
assistant=new Assistant();Approver
manager=new Manager();Approver
admin=new
Administrator();assistant.setNext(manager);ma
nager.setNext(admin);assistant.approve(7000);
}}
```

Output:

Administrator approved request

System	Scenario	Behavioral Pattern
Dhaka Urban Management System	Complaint status notification	Observer Pattern
Agricultural Automation System	Irrigation algorithm selection	Strategy Pattern
Smart Campus Transportation System	Transport approval workflow	Chain of Responsibility Pattern

These three questions together cover three major behavioral patterns:

1.Observer → Object communication

2.Strategy → Algorithm selection

3.Chain of Responsibility → Request processing workflow

They are appropriate for an Object-Oriented Design Pattern Lab exam because students can identify the pattern from the real-world scenario and implement it using Java.

Other Pattern Scenario

Observer Patttern Scenario:
In the Dhaka Urban Management System, citizens can submit complaints regarding waste collection, traffic problems, water supply, and emergency services.
Whenever the complaint status changes (e.g., Submitted → Processing → Completed), multiple users should automatically receive updates:
•Citizen receives notification about complaint progress.
•Responsible department officer receives task updates.
•City administrator receives monitoring information.
The complaint management system should notify all registered users automatically without modifying the complaint class whenever a new type of user is added.
Requirement:
What problem does the Observer Design Pattern solve? Explain how the Observer Pattern can be applied in the Dhaka Urban Management System. Draw the supporting UML diagram and implement the solution.
Expected Pattern: Observer Pattern
Concept:
One object (Subject) maintains a list of dependent objects (Observers) and automatically notifies them when its state changes.

Other Pattern Scenario

Strategy Pattern
Agricultural Automation System (AAS)
Scenario:An Agricultural Automation System controls irrigation based on different farming conditions.The irrigation controller supports different irrigation strategies:Normal irrigation during regular weather conditions.Water-saving irrigation during water shortage.Emergency irrigation during drought situations.The system should allow farmers to change the irrigation strategy dynamically without modifying the irrigation controller code.Requirement:What problem does the Strategy Design Pattern solve? Explain how the Strategy Pattern can be used for dynamic irrigation decision-making in the Agricultural Automation System. Draw the supporting UML diagram and implement the solution.

Expected Pattern:Strategy Pattern

Concept:Encapsulates different algorithms and allows them to be exchanged at runtime.

Other Pattern Scenario

Command Pattern

Smart Food Management System

Scenario:A Smart Food Management System provides an online food ordering service.Customers can perform different actions:Place food order.Cancel order.Modify order.Track delivery.The system should store customer requests so that previous operations can be logged, cancelled, or executed later.The ordering system should allow different commands without directly coupling the customer interface with restaurant operations.Requirement:What problem does the Command Design Pattern solve? Explain how the Command Pattern can be applied in the Smart Food Management System for handling customer orders. Draw the supporting UML diagram and implement the solution.

Expected Pattern:Command Pattern

Concept:Encapsulates a request as an object, allowing operations to be stored, queued, logged, or undone.

Other Pattern Scenario

State Pattern
Smart Traffic Management System (DUMS Module)
Scenario:A Smart Traffic Management System controls traffic signals at different road intersections.A traffic signal changes its behavior depending on its current state:Green State → Allow vehicles to move.Yellow State → Prepare vehicles to stop.Red State → Stop vehicles.The behavior of the traffic light should change automatically when its internal state changes without using complex conditional statements.Requirement:What problem does the State Design Pattern solve? Explain how the State Pattern can be used to model traffic signal behavior in the Smart Traffic Management System. Draw the supporting UML diagram and implement the solution.

Expected Pattern:State Pattern

Concept:Allows an object to change its behavior when its internal state changes.

Other Pattern Scenario

Adapter Pattern

Agricultural Automation System

Scenario:A Smart Agricultural Automation System uses an existing irrigation controller that accepts data from a standard soil moisture sensor.A newly purchased IoT soil sensor provides moisture information using a different interface. The existing irrigation controller cannot directly communicate with this new sensor.

Requirement:What problem does the Adapter Design Pattern solve? Explain how the Adapter Pattern can integrate the new IoT soil sensor without modifying the existing irrigation controller. Draw the supporting UML diagram and implement the solution.

Sample Question

What problem does Adapter design pattern solve? Explain with an example and draw supporting UML diagram.

Suppose You are designing a system for money withdrawal from bank account using check with the following authorization scenario:

- For Tk. < 10,000 only cashier's authorization is sufficient to withdraw money from account.
- For Tk. 10,000 to 10,00,000 authorization from both cashier and senior officer is needed.
- For Tk > 10,00,000 authorization from Senior officer and Manager is needed.

Implement this behaviour using appropriate software design pattern.

Sample Quesiton

Bloom's Level	Question
Remember	1. What is a behavioural design pattern? Name five behavioural patterns discussed in Module 2.
Understand	2. Explain the main purpose of the Template Method Pattern.
Understand	3. Explain how the Chain of Responsibility Pattern processes a request through multiple handlers.
Apply	4. Design a university complaint-handling process using the Chain of Responsibility Pattern, where complaints may be handled by a Class Teacher, Department Chairman, Dean, or Vice Chancellor.
Analyze	5. Analyze a program containing a large number of if-else statements to determine an object's behavior based on its current state. Explain how the State Pattern could improve the design.
Evaluate	6. Evaluate the advantages and disadvantages of using the State Pattern instead of a large switch statement.
Create	7. Design a Java-based online order system using the State Pattern with Pending, Paid, Shipped, and Delivered states.

Bloom's Level	Question
Remember	1. Define the Command Pattern and identify its main components.
Understand	2. Explain the purpose of the Command Pattern and how it encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support undo and redo operations.
Apply	4. Apply the Command Pattern to a text editor that supports Copy, Paste, Delete, and Undo operations. Identify the Command, Invoker, and Receiver.
Analyze	5. Analyze how the Command Pattern reduces coupling between a graphical user-interface button and the object that performs the actual operation.
Evaluate	6. Evaluate the benefits of using the Command Pattern for queueing, logging, scheduling, and undo/redo operations.
Create	7. Design a Java-based remote-control system using the Command Pattern where different buttons can execute different commands such as LightOn, LightOff, and FanOn.

Bloom's Level	Question
Remember	1. Define the Command Pattern and identify its main components.
Understand	2. Explain the purpose of the Command Pattern and how it encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support undo and redo operations.
Apply	4. Apply the Command Pattern to a text editor that supports Copy, Paste, Delete, and Undo operations. Identify the Command, Invoker, and Receiver.
Analyze	5. Analyze how the Command Pattern reduces coupling between a graphical user-interface button and the object that performs the actual operation.
Evaluate	6. Evaluate the benefits of using the Command Pattern for queueing, logging, scheduling, and undo/redo operations.
Create	7. Design a Java-based remote-control system using the Command Pattern where different buttons can execute different commands such as LightOn, LightOff, and FanOn.

Sample Quesiton

Bloom's Level	Question
Remember	1. Name five behavioural design patterns discussed in Module 2.
Understand	2. Explain the main idea of the Template Method Pattern.
Understand	3. Explain why the Chain of Responsibility Pattern reduces coupling between a request sender and request receiver.
Apply	4. A university has several levels of approval: Teacher → Chairman → Dean → Vice Chancellor. Apply the Chain of Responsibility Pattern to this approval system.
Analyze	5. Analyze a program where an object's behavior is controlled by a large switch statement based on its current state. Identify the problem and explain how the State Pattern could improve it.
Evaluate	6. Evaluate the advantages and disadvantages of using the Chain of Responsibility Pattern when many handlers are present in the chain.
Create	7. Design a Java-based customer-support system using the Chain of Responsibility Pattern with different support levels such as Level 1, Level 2, and Level 3 support.

Bloom's Level	Question
Remember	1. What is the Command Pattern? Name its major participants.
Understand	2. Explain how the Command Pattern encapsulates a request as an object.
Understand	3. Explain how the Command Pattern can support command queueing and scheduling.
Apply	4. Apply the Command Pattern to a text editor that provides Copy, Paste, Delete, and Undo operations. Identify the Client, Command, Invoker, and Receiver.
Analyze	5. Analyze a GUI application in which every button directly contains the application logic for performing its operation. Explain the coupling problem and how the Command Pattern can solve it.
Evaluate	6. Evaluate whether the Command Pattern is appropriate for implementing an undo/redo mechanism. Explain your reasoning.
Create	7. Create a Java-based restaurant ordering system using the Command Pattern where orders can be created, queued, executed, and cancelled.

Bloom's Level	Question
Remember	1. Define the Observer Pattern and identify the Subject and Observer.
Understand	2. Explain the one-to-many relationship used by the Observer Pattern.
Understand	3. Explain how the Observer Pattern reduces direct dependency between a subject and its dependent objects.
Apply	4. Apply the Observer Pattern to an online news system where multiple users receive notifications whenever a new article is published.
Analyze	5. Analyze a weather-monitoring system where the weather station directly calls update methods on every display component. Explain the design problems and suggest how the Observer Pattern could improve the system.
Evaluate	6. Compare the Observer Pattern with manually notifying every dependent object. Evaluate the advantages of using the Observer Pattern.
Create	7. Design a Java-based stock-price monitoring system using the Observer Pattern. Include a StockMarket Subject and at least three different Observer classes.

Module 3:
Working with Design Patterns and Anti-patterns

Module 3: Working with Design Patterns and Anti-patterns

- **Module Focus**
  - Understand how design patterns are applied in real software designs.
  - Understand MVC as a design pattern for separating user-interface responsibilities.
  - Apply general design principles to evaluate software structure.
  - Analyze and critique code that exhibits poor design.
  - Recognize anti-patterns and code smells before they make a system difficult to maintain.
  - Use refactoring to improve internal code structure without changing external behavior.
- Key idea: Good design is not only about selecting a pattern; it is also about recognizing when a design has become unnecessarily coupled, complex, duplicated, or difficult to maintain.

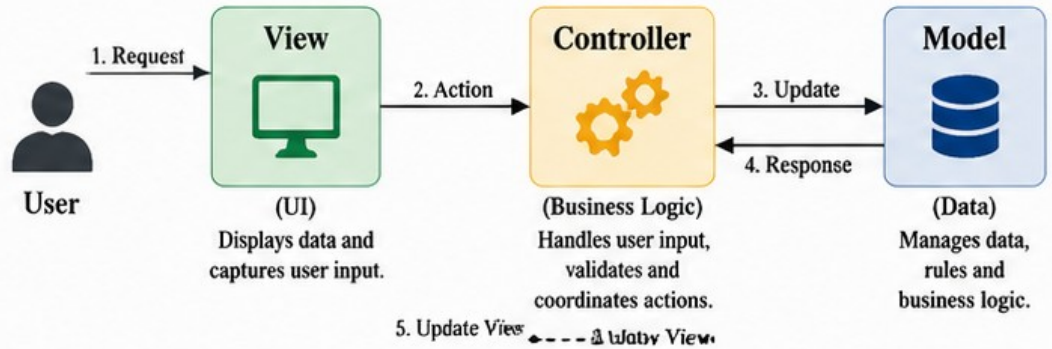
Module 3: Working with Design Patterns and Anti-patterns

Key idea: Good design is not only about selecting a pattern;
it is also about recognizing when a design has become
unnecessarily coupled, complex, duplicated, or difficult to maintain.

Module Focus

- Understand how design patterns are applied in real software designs.
- Understand MVC as a design pattern for separating user-interface responsibilities.
- Apply general design principles to evaluate software structure.
- Analyze and critique code that exhibits poor design.
- Recognize anti-patterns and code smells before they make a system difficult to maintain.
- Use refactoring to improve internal code structure without changing external behavior.

MVC Design Pattern



MVC separates concerns: *View* handles ***presentation***, *Controller* handles interaction, and ***Model*** **handles *data*** and logic.

General Design Principles

Single Responsibility Principle (SRP)
A class should have one reason to change.

Open/Closed Principle (OCP)
Open for extension, closed for modification.

Liskov Substitution Principle (LSP)
Subtypes must be substitutable for their base types.

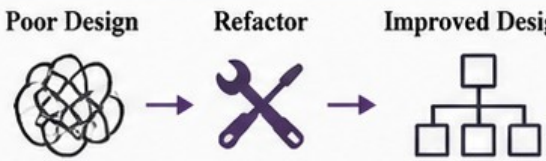
Interface Segregation Principle (ISP)
Clients should not be forced to depend on interfaces they don't use.

Dependency Inversion Principle (DIP)
Depend on abstractions, not on concretions.

Anti-patterns / Code Smells

- **Spaghetti Code:** Unstructured flow, hard to follow.
- **God Object:** A class that knows or does too much.
- **Duplicate Code:** Same code repeated in many places.
- **Long Method:** Method is too long, does too many things.
- **Tight Coupling:** Classes depend heavily on each other.

Refactoring

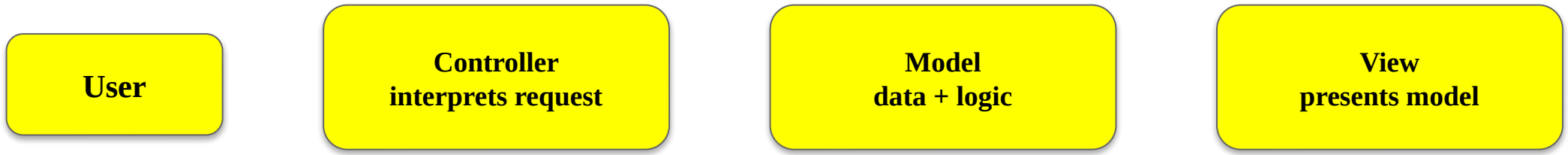


Improve *internal structure* without changing *external behavior*.

Good software design improves understandability, maintainability, testability, and scalability.
Always review, refactor, and evolve your design continuously.

MVC Design Pattern — Overview

- ✓ **The Model–View–Controller (MVC) pattern divides the responsibilities of a system that offers a user interface into three parts: Model, View, and Controller.**
- ✓ **Model:** Contains the underlying **data and logic** that users want to see and manipulate through an interface.
- ✓ **View:** Allows **users** to see all or part of the model.
- ✓ **Controller:** Interprets **user requests and changes** the model.
- ✓ **Main Goal:** Separate responsibilities so that user-interface presentation, request handling, and application data/logic are not unnecessarily mixed together.



Model–View–Controller (MVC) Design Pattern

What is MVC?

The Model–View–Controller (MVC) pattern divides the responsibilities of a system that offers a user interface into three parts: Model, View, and Controller.

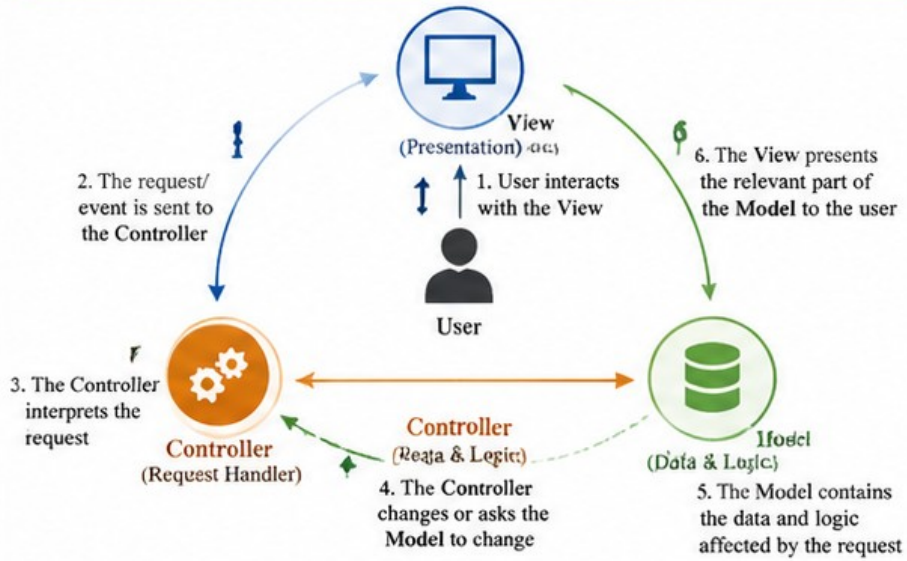
Model: Contains the underlying **data** and **logic** that users want to see and manipulate through an interface.

View: Allows **users** to see all or part of the model.

Controller: Interprets **user requests** and changes the model.

Main Goal:
Separate responsibilities so that user-interface presentation, request handling, and application data/logic are not unnecessarily mixed together.

MVC Interaction Flow



Result:
Each part has a clearer responsibility, making the system easier to **understand** and **modify**.

Scenario: A User Clicks "Add Student"

1. **View:** Displays the student form and collects the user's input.
2. **Controller:** Receives the ***Add Student*** request, validates/coordinates the request, and asks the Model to perform the operation.
3. **Model:** Represents **student data** and performs the required application/data operations.
4. **View:** Displays the updated student list or confirmation.

Separation of Responsibilities
- View should focus on presentation.
- Controller should coordinate requests.
- Model should contain the underlying data and logic.

PHP Laravel MVC vs Java MVC (Comparison)

Aspect	PHP Laravel MVC	Java MVC (e.g., Spring MVC)
Framework Type	Full-stack MVC framework	MVC implemented in enterprise frameworks
Routing / Request Handling	Handled by Laravel **Router** (routes/web.php, **routes/api.php**)	Handled by **DispatcherServlet** and @RequestMapping
Controller	PHP classes (extend Controller), methods return View/Response	Java classes with @Controller or @RestController
View	Blade templates (server-side rendering)	JSP, Thymeleaf, or other view technologies
Model	Eloquent ORM (ActiveRecord pattern)	**JPA/Hibernate (Entity pattern)**
Data Binding	Automatic via Request validation and Eloquent	Automatic via **Model** attributes and data binding
Configuration	Convention over Configuration	More configuration (XML/Java Config/Annotations)
Example Flow	Route → Controller → Model (Eloquent) → View (Blade)	Request → Controller → Model (Service/Repository) → View
Use Cases	Rapid web development, small to medium applications	Large-scale, enterprise, highly modular applications

Why MVC?

- Separation of concerns (clearly defined roles)
- Easier maintenance and modifications
- Better testability (each part can be tested independently)
- Supports team collaboration (frontend, backend, database teams can work separately)

MVC — Request and Response Flow

Typical interaction flow:

1. User interacts with the View.
2. The request/event is sent to the Controller.
3. The Controller interprets the request.
4. The Controller changes or asks the Model to change.
5. The Model contains the data and logic affected by the request.
6. The View presents the relevant part of the Model to the user.

Result: Each part has a clearer responsibility, making the system easier to understand and modify.



MVC — Simple Example: Student Management System

- **Scenario:** A user clicks “Add Student”.
- **View:** Displays the student form and collects the user's input.
- **Controller:** Receives the Add Student request, validates/coordinates the request, and asks the Model to perform the operation.
- **Model:** Represents student data and performs the required application/data operations.
- **View:** Displays the updated student list or confirmation.
- **Separation of responsibilities:**
  - View should focus on presentation.
  - Controller should coordinate requests.
  - Model should contain the underlying data and logic.

General Design Principles — Why They Matter

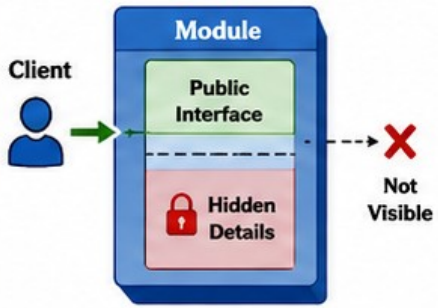
- Design principles provide guidance for creating reusable, flexible, understandable, and maintainable software.
- **Information Hiding:** Modules should expose the minimum information needed to use them correctly and hide the rest.
- **Separation of Concerns:** Organize software so different concerns are separated into different sections and addressed effectively.
- **Dependency Inversion:** High-level modules should depend on high-level generalizations rather than low-level implementation details.
- **Open/Closed Principle:** Classes should be open for extension but closed to change.
- **Interface Segregation:** A class should not be forced to depend on methods it does not use.
- **Coupling Objects Principle:** Classes should achieve code reuse through aggregation rather than inheritance.

General Design Principles — Why They Matter

Design principles provide guidance for creating reusable, flexible, understandable, and maintainable software.

1 Information Hiding

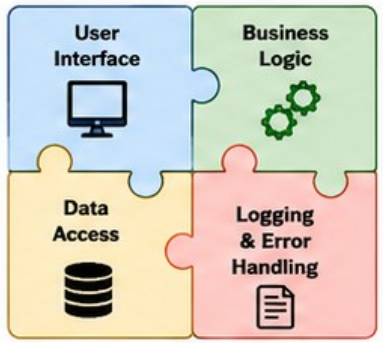
Modules should expose the minimum information needed to use them correctly and hide the rest.



Expose what is necessary, hide what is not.

2 Separation of Concerns

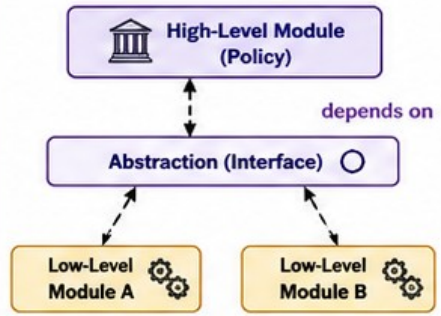
Organize software so different concerns are separated into different sections and addressed effectively.



Each concern changes for a different reason.

3 Dependency Inversion

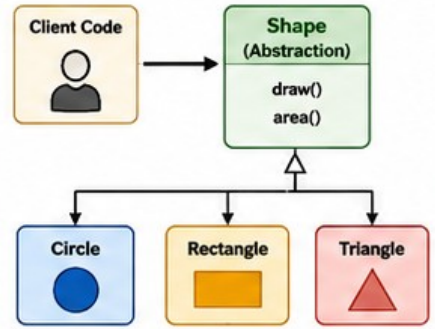
High-level modules should depend on high-level generalizations rather than low-level implementation details.



Depend on abstractions, not on concretions.

4 Open/Closed Principle

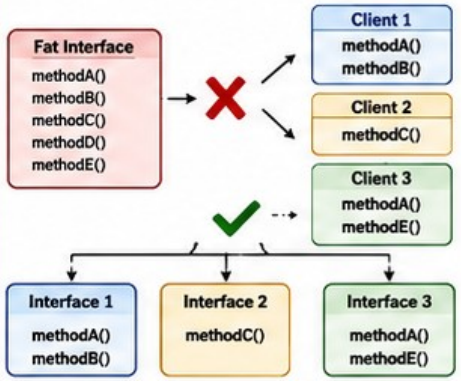
Classes should be open for extension but closed to change.



Extend behavior via new classes, not by modifying existing code.

5 Interface Segregation

A class should not be forced to depend on methods it does not use.



Many small interfaces are better than one large interface.

General Design Principles — DRY and Least Knowledge

6 Don't Repeat Yourself (DRY)

- ✓ Write programs that perform the same tasks with less repeated code.
- ✓ Reuse shared blocks of code so systems are easier to maintain.

Without DRY (Duplicated Code)

```
void totalOrder1() {
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}
void totalOrder2() {
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}
```

With DRY (Reusable Code)

```
int calculateTotal(){
  int sum = 0;
  for(Item i : items){
    sum += i.getPrice();
  }
  return sum;
}

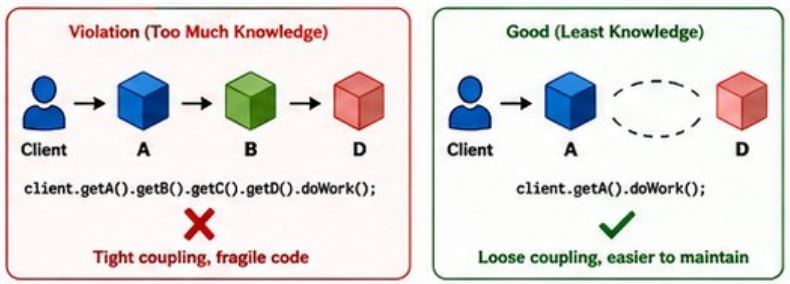
void totalOrder1(){
  int sum = calculateTotal();
}

void totalOrder2(){
  int sum = calculateTotal();
}
```

One change in one place — everywhere it is used.

7 Principle of Least Knowledge / Law of Demeter

- ✓ Classes should know about and interact with as few other classes as possible.
- ✓ Avoid unnecessary navigation through chains of dependent objects.



Talk only to your immediate friends.

Why these principles matter:

- Less duplication means fewer places to update.
- Less knowledge reduces coupling.
- Simpler dependencies make testing and refactoring easier.
- Results in software that is more maintainable, flexible, and scalable.



Good design is not about adding more code. It is about organizing code smartly. | Better Design ⇒ Better Quality ⇒ Better Software

General Design Principles — DRY and Least Knowledge

✓ Don't Repeat Yourself (DRY)

- Write programs that perform the same tasks with less repeated code.
- Reuse shared blocks of code so systems are easier to maintain.

✓ Principle of Least Knowledge / Law of Demeter

- Classes should know about and interact with as few other classes as possible.
- Avoid unnecessary navigation through chains of dependent objects.
- **Why these principles matter:**
  - Less duplication means fewer places to update.
  - Less knowledge reduces coupling.
  - Simpler dependencies make testing and refactoring easier.

How to Analyze and Critique a Design — Checklist

Step 1: Identify Responsibilities

Ask: What does each class actually do? Does it have one clear purpose?

Step 2: Identify Dependencies

Ask: Which classes know about each other? Are there unnecessary dependencies or cycles?

Step 3: Check Coupling and Cohesion

Ask: Are responsibilities tightly connected inside a class, and are unrelated responsibilities separated?

Step 4: Look for Code Smells: Check for duplication, long methods, large classes, message chains, primitive obsession, switch statements, and inappropriate intimacy.

Step 5: Check Principles: Review information hiding, separation of concerns, DRY, least knowledge, dependency inversion, open/closed, and interface segregation.

Step 6: Refactor: Improve the internal structure while preserving external behavior.

Analyzing and Critiquing Code — What Is Bad Design?

When reviewing code, ask:

1. Does one class have too many unrelated responsibilities?

2. Are responsibilities placed in the wrong class?

3. Are classes tightly coupled through unnecessary dependencies?

4. Is the same logic duplicated in multiple locations?

5. Are long methods, long parameter lists, or long call chains making behavior difficult to understand?

6. Are primitive values being used where meaningful abstractions/classes should exist?

7. Are switch statements scattered throughout the system?

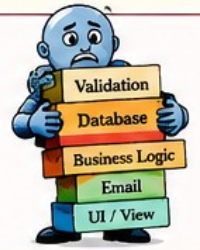
8. Is the code generalized for requirements that do not yet exist?

9. Does inheritance force subclasses to receive behavior they do not need?

Analyzing and Critiquing Code — What Is Bad Design?

When reviewing code, ask these important questions:

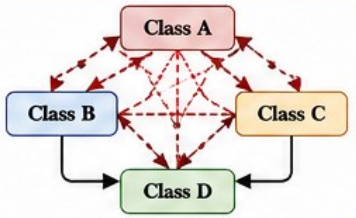
1 Does one class have too many unrelated responsibilities?



2 Are responsibilities placed in the wrong class?

Wrong Class ✗ → Right Class ✓

3 Are classes tightly coupled through unnecessary dependencies?



4 Is the same logic duplicated in multiple locations?



5 Are long methods, long parameter lists, or long call chains making behavior difficult to understand?

```
method(a, b, c, d, e,
       f, g, h, i, j, k, l,
       m, n, o, p, q, r)
{
  ... many lines of code ...
  methodA().methodB()
           .methodC()
           .methodD();
}
```

6 Are primitive values being used where meaningful abstractions/classes should exist?

Primitive Obsession

```
String name
int age
int type
double amount
```

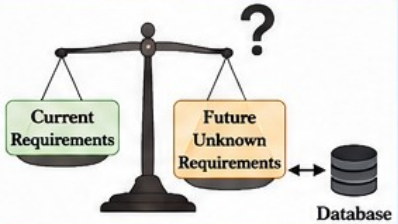
Better: Use Meaningful Classes

```
Person
Age
OrderType
Money
```

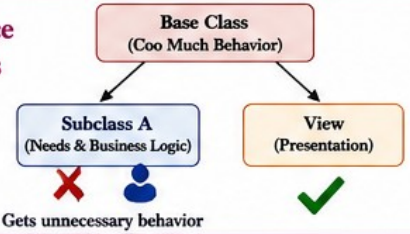
7 Are switch statements scattered throughout the system?

```
switch(type) {
  case A: ...
  case B: ...
  case C: ...
  default: ...
}
```

8 Is the code generalized for requirements that do not yet exist?



9 Does inheritance force subclasses to receive behavior they do not need?

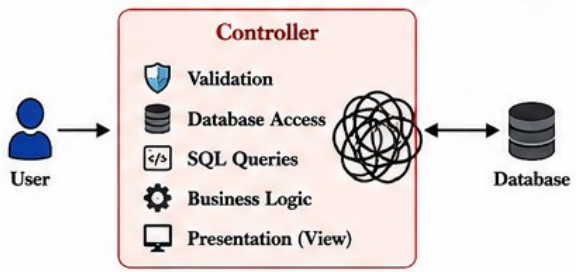


Code Critique Example — Multiple Responsibilities

Critique:

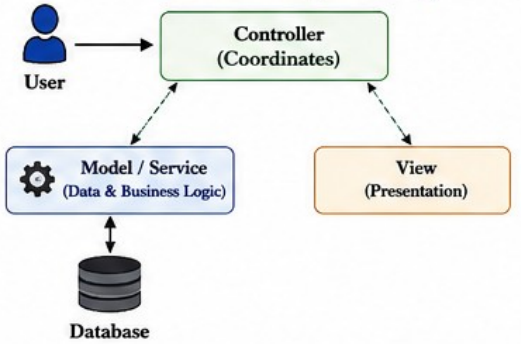
- ✗ Controller handles validation, database access, SQL, and presentation.
- ✗ Responsibilities are mixed together.
- ✗ Changing the database can affect the controller.
- ✗ Changing the presentation can affect the controller.
- ✗ Difficult to test each responsibility independently.

Problem: Everything in One Class (Controller)



Hard to maintain, hard to change, hard to test!

Better Direction: Separate Responsibilities



- ✓ Controller coordinates.
- ✓ Model/service handles data and application logic.
- ✓ View handles presentation.
- ✓ Each part is easier to understand, change, and test.

★ Good design leads to clean, maintainable, flexible, and testable software. → Bad design leads to complexity, bugs, and cost. ★

Code Analysis/ Code Critique Example — Multiple Responsibilities

```
class StudentController {

void addStudent(String name, String email) {
                // validate input
   if (name == null || name.isEmpty()) return;

          // create database connection
Connection db = DriverManager.getConnection(...);

                // SQL operation
                  String sql =
   "INSERT INTO students(name,email) VALUES(?,?)";

                // execute SQL
                    // ...

            // format HTML response
              System.out.println(
    "<html><body>Student Added</body></html>");
                    }
                  }
```

- ✓ **Critique:**
  - ✓ Controller handles validation, database access, SQL, and presentation.
  - ✓ Responsibilities are mixed together.
  - ✓ Changing the database can affect the controller.
  - ✓ Changing the presentation can affect the controller.
  - ✓ Difficult to test each responsibility independently.

- ✓ **Better direction:**
  - ✓ Controller coordinates.
  - ✓ Model/service handles data and application logic.
  - ✓ View handles presentation.

Code Analysis/ Code Critique Example — Scattered Switch Statements

```
double calculatePrice(String type) {
            switch(type) {
        case "REGULAR": return 100;
        case "PREMIUM": return 150;
            default: return 0;
                }
              }

String getDescription(String type) {
            switch(type) {
     case "REGULAR": return "Regular";
     case "PREMIUM": return "Premium";
        default: return "Unknown";
                }
              }

 double getDiscount(String type) {
            switch(type) {
       case "REGULAR": return 0.05;
       case "PREMIUM": return 0.15;
            default: return 0;
                }
              }
```

- ✓ **Critique:**
  - ✓ The same type information is checked repeatedly.
  - ✓ Adding a new type requires changing several methods.
  - ✓ The design can become difficult to maintain.
  - ✓ This is the kind of situation where polymorphism may reduce scattered conditionals.
- ✓ **Possible refactoring:**
  - ✓ Create a common Product/Customer/Plan abstraction.
  - ✓ Move price, description, and discount behavior into the appropriate concrete classes.

Code Analysis/ Code Critique Example — Primitive Obsession

```
void createOrder(
 String customerName,
String customerEmail,
   String country,
  String currency,
   String street,
    String city,
 String postalCode) {

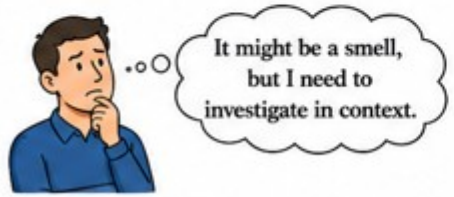
// many operations using raw Strings
                }
```

- **Potential smell:**
  - Many Strings represent different concepts.
  - The code does not clearly express the domain abstractions.
  - Validation and behavior may be duplicated across the system.
- **Possible improvement:**
  - Create meaningful types such as Customer, EmailAddress, Address, PostalCode, and Money where appropriate.
  - Encapsulate validation and related behavior in those abstractions.
- Result: The design communicates the domain more clearly and reduces primitive obsession.

Anti-patterns and Code Smells

Anti-patterns: Patterns of bad code that commonly emerge in software design; they are also described as code smells.

Code smells are warning signs rather than automatic proof that code is wrong.



Why Identify Them?

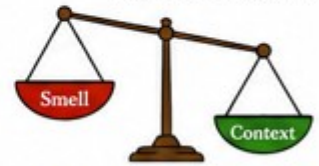
- They help identify bad code and bad design early.
- They indicate places where refactoring may improve reuse, flexibility, and maintainability.
- Regular code review can prevent small problems from becoming major design problems.

Common Code Smells (Examples)

Long Method	Method is too long and does too many things.
Large Class	Class has too many responsibilities and too many fields.
Duplicate Code	Same code appears in multiple places.
Long Parameter List (`f( a, b, c, d, e, f, g )`)	Method has too many parameters, hard to understand and use.
Feature Envy	Method uses more data from other classes than its own.
Data Clumps	Groups of data travel together but have no structure.

Important

A smell should be investigated in context. Some structures may be appropriate when there is a valid design reason.



Understand the reason.
Make the right decision.

Anti-patterns and Code Smells

- **Anti-patterns: Patterns of bad code that commonly emerge in software design; they are also described as code smells.**
- Code smells are warning signs rather than automatic proof that code is wrong.
- **Why Identify Them?**
  - They help identify bad code and bad design early.
  - They indicate places where refactoring may improve reuse, flexibility, and maintainability.
  - Regular code review can prevent small problems from becoming major design problems.
- **Important:** A smell should be investigated in context. Some structures may be appropriate when there is a valid design reason.

Common Anti-patterns / Code Smells in This Module

Large Class — a class that is too long; also called a God, Blob, or Black Hole class.

Long Method — a method containing excessive logic.

Duplicated Code — similar code blocks appear in multiple places with slight differences.

Data Class — a class containing mostly data and little real functionality.

Data Clumps — groups of data repeatedly appear together as fields or method parameters.

Divergent Change — one class must be changed in many different ways for many different reasons.

Shotgun Surgery — one requirement change requires changes across many classes.

Feature Envy — a method is more interested in another class's details than its own class.

Long Parameter Lists — methods require too many parameters.

Comments as a smell — excessive comments may indicate that comments are being used to explain or cover a difficult design.

Code Smells — Inappropriate Intimacy and Message Chains

- **Inappropriate Intimacy:** Occurs when two classes depend too much on each other through two-way communication.
  - If methods in one class call the other and vice versa, consider removing the cycle.
  - Factor methods so both classes can use another class, or at least make communication one-way.
- **Message Chains:** Long chains of calls can create rigidity and complexity and make code harder to test independently.
  - They can violate the Law of Demeter when objects know too much about other objects' dependencies.
  - Navigation through dependencies should not be unnecessarily baked into objects.
- **Important qualification:** A message chain is not automatically bad if it returns a limited set of objects and follows the Law of Demeter.

Code Smells — Primitive Obsession and Switch Statements

- **Primitive Obsession:**
  - Occurs when the design relies too much on built-in types such as int, long, float, and String.
  - Overuse can indicate that meaningful abstractions/classes have not been identified.
  - High-level use of primitives may hide important domain concepts.
- **Switch Statements:**
  - Scattered switch statements become a problem because changing one case may require finding and updating many other switches.
  - When conditionals repeatedly check type codes, consider whether polymorphism could express the behavior more clearly.
- **Better direction:** Identify missing abstractions and move behavior into appropriate classes rather than encoding everything as primitive values or repeated conditionals.

Code Smells — Speculative Generality and Refused Request

Speculative Generality

- Creating a superclass, interface, or code that is not currently needed but may be useful someday.
- It introduces generality that may not help and can over-engineer the system.
- Just-in-Time Design suggests designing only what is needed for the current requirements/iteration.
- If generalization becomes necessary later, refactor when the requirement is real.

Refused Request

- A subclass inherits behavior that it does not need or use.
- This may indicate that the subclass is not appropriate for the superclass.
- Consider a standalone class or move behavior to the subclasses that actually need it.

Example-1: Code Smells — Problem

Problem: A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach

Solution: Likely Code Smell: Long Method

The method contains **100 lines of code** and performs several different responsibilities:

- Input validation
- Database access
- Calculations
- Report generation

This indicates that the method has **too many responsibilities**, making it difficult to understand, test, maintain, and modify.

Appropriate Refactoring Approach: Extract Method

The large method should be divided into several smaller, focused methods, where each method performs one specific task.

The likely code smell is **Long Method** because the method contains about 100 lines and performs several different responsibilities such as validation, database access, calculation, and report generation. The appropriate refactoring technique is **Extract Method**. The large method should be divided into smaller methods such as validateStudent(), getStudentMarks(), calculateAverage(), and generateReport(). This improves **readability, maintainability, reusability, and testability**.

Code Smells — Problem

Before Refactoring Long Method:

```
public void processStudent() {

   // Validation
   if (studentName == null || studentName.isEmpty()) {
      System.out.println("Invalid student name");
      return;
   }

   if (studentId <= 0) {
      System.out.println("Invalid student ID");
      return;
   }

   // Database access
   Connection con = DriverManager.getConnection(
      "jdbc:mysql://localhost:3306/university",
      "root",
      "password"
   );

   PreparedStatement ps = con.prepareStatement(
      "SELECT marks FROM students WHERE id = ?"
   );
   ps.setInt(1, studentId);

   ResultSet rs = ps.executeQuery();

   // Calculation
   double total = 0;
   int count = 0;

   while (rs.next()) {
      total += rs.getDouble("marks");
      count++;
   }

   double average = total / count;

   // Report generation
   System.out.println("Student: " + studentName);
   System.out.println("Student ID: " + studentId);
   System.out.println("Average Marks: " + average);

   // Many more lines...
}
```

After Refactoring:

```
public void processStudent() {

   if (!validateStudent()) {
      return;
   }

   List<Double> marks = getStudentMarks();

   double average = calculateAverage(marks);

   generateReport(average);
}
```

The responsibilities are now separated:

```
private boolean validateStudent() {
   if (studentName == null ||
studentName.isEmpty()) {
      System.out.println("Invalid student name");
      return false;
   }

   if (studentId <= 0) {
      System.out.println("Invalid student ID");
      return false;
   }

   return true;
}

private List<Double> getStudentMarks() {
   List<Double> marks = new ArrayList<>();

   // Database access code
   // Execute SQL query
   // Retrieve marks

   return marks;
}
```

```
private double calculateAverage(List<Double> marks) {
   if (marks.isEmpty()) {
      return 0;
   }
   double total = 0;

   for (double mark : marks) {
      total += mark;
   }

   return total / marks.size();
}
```

```
private void generateReport(double average) {
   System.out.println("Student: " + studentName);
   System.out.println("Student ID: " + studentId);
   System.out.println("Average Marks: " + average);
  }
```

Item	Answer
Code Smell	**Long Method**
Problem	One method performs too many tasks
Refactoring Technique	**Extract Method**
Main benefit	Improves readability, maintainability, and testability
Extracted methods	validateStudent(), getStudentMarks(), calculateAverage(),

Code Smells — Speculative Generality and Refused Request

Speculative Generality

- Creating a superclass, interface, or code that is not currently needed but may be useful someday.
- It introduces generality that may not help and can over-engineer the system.
- Just-in-Time Design suggests designing only what is needed for the current requirements/iteration.
- If generalization becomes necessary later, refactor when the requirement is real.

Refused Request

- A subclass inherits behavior that it does not need or use.
- This may indicate that the subclass is not appropriate for the superclass.
- Consider a standalone class or move behavior to the subclasses that actually need it.

MVC Anti-pattern — When Responsibilities Are Mixed

A common poor MVC design occurs when one class attempts to perform the responsibilities of Model, View, and Controller simultaneously.

Symptoms:

- Database queries inside UI code.
- HTML/UI formatting inside data/model classes.
- Business rules directly inside event-handling code.
- Controllers that become extremely large and perform every operation themselves.

Problems:

- High coupling.
- Difficult testing.
- Difficult maintenance.
- Changes in one concern affect unrelated concerns.

Better approach: Preserve separation of concerns and assign each responsibility to the appropriate MVC component.

Example-2: If a System Suffering from a Large Class and Duplicated Code

1. Identify the Code Smells

There are two major code smells:

① **Large Class** – One class contains too many responsibilities.

② **Duplicated Code** – The same or very similar code is repeated in multiple methods.

Example Scenario

Suppose we have a StudentManager class that handles:

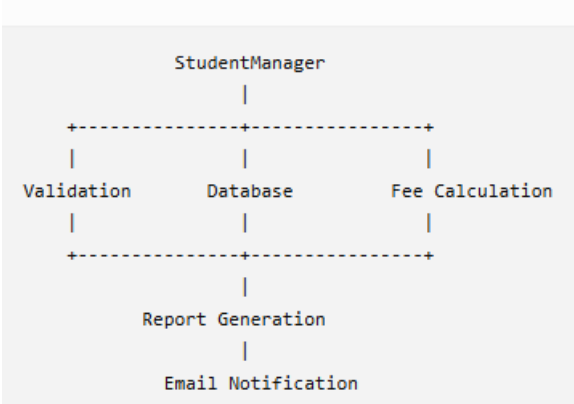
① Student information
② Student validation
③ Database operations
④ Fee calculation
⑤ Report generation
⑥ Email notification

This violates the **Single Responsibility Principle (SRP)**.

2. Before Refactoring

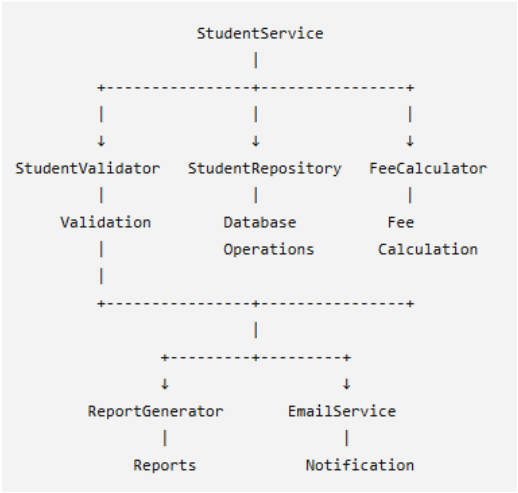
① A poorly designed class might very long code

② For Example: The StudentManager class is doing **too many jobs**:



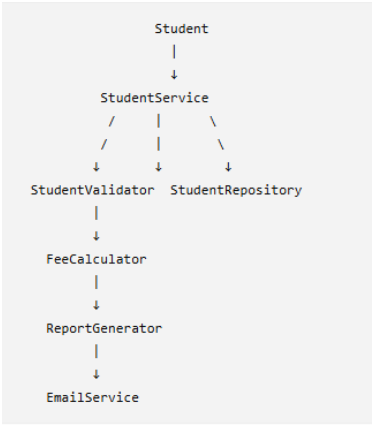
It also contains duplicated validation and fee-calculation code.

6. Final Design: The final structure can be repres as:



3. Refactored Design

① We can redistribute the responsibilities into several classes:



Class	Responsibility
Student	Store student information
StudentValidator	Validate student information
StudentRepository	Database operations
FeeCalculator	Calculate tuition fees
ReportGenerator	Generate student reports
EmailService	Send email notifications
StudentService	Coordinate the overall student operation

If a System Suffering from a Large Class and Duplicated Code: Java Example

Before Refactoring Long Method:

```
public class StudentManager {

    private String name;
    private int studentId;
    private double tuitionFee;

    public void registerStudent() {
        // Student validation
        if (name == null || name.isEmpty()) {
            System.out.println("Invalid student name");
            return;
        }
        if (studentId <= 0) {
            System.out.println("Invalid student ID");
            return;
        }
        // Database operation
        System.out.println("Saving student to database...");
        // Fee calculation
        double fee = tuitionFee;
        if (fee > 50000) {
            fee = fee * 0.90;   // 10% discount
        }
        System.out.println("Final fee: " + fee);
        // Report generation
        System.out.println("Student Report");
        System.out.println("Name: " + name);
        System.out.println("ID: " + studentId);
        // Email notification
        System.out.println("Sending email to student...");
    }

    public void updateStudent() {
        // Same validation code - DUPLICATED
        if (name == null || name.isEmpty()) {
            System.out.println("Invalid student name");
            return;
        }
        if (studentId <= 0) {
            System.out.println("Invalid student ID");
            return;
        }
        // Database operation
        System.out.println("Updating student in database...");
        // Same fee calculation - DUPLICATED
        double fee = tuitionFee;

        if (fee > 50000) {
            fee = fee * 0.90;
        }
        System.out.println("Updated fee: " + fee);
        // Email notification
        System.out.println("Sending email to student...");
    }
}
```

If a System Suffering from a Large Class and Duplicated Code: Java Example

After Refactoring Long Method:

```
public class Student {

    private int studentId;
    private String name;
    private double tuitionFee;

    public Student(int studentId, String name, double tuitionFee) {
        this.studentId = studentId;
        this.name = name;
        this.tuitionFee = tuitionFee;
    }

    public int getStudentId() {
        return studentId;
    }

    public String getName() {
        return name;
    }

    public double getTuitionFee() {
        return tuitionFee;
    }
}
```

```
public class StudentRepository {
    public void save(Student student) {
        System.out.println("Saving student to database...");
    }
    public void update(Student student) {
        System.out.println("Updating student in database...");
    }
}
```

```
public class StudentValidator {

    public boolean isValid(Student student) {

        if (student.getName() == null ||
            student.getName().isEmpty()) {
            return false;
        }

        if (student.getStudentId() <= 0) {
            return false;
        }

        return true;
    }
}
```

```
public class ReportGenerator {

    public void generate(Student student, double fee) {

        System.out.println("Student Report");
        System.out.println("Name: " + student.getName());
        System.out.println("ID: " + student.getStudentId());
        System.out.println("Final Fee: " + fee);
    }
}
```

```
public class FeeCalculator {

    public double calculateFee(double tuitionFee) {

        if (tuitionFee > 50000) {
            return tuitionFee * 0.90;
        }

        return tuitionFee;
    }
}
```

```
public class EmailService {

    public void sendEmail(Student student) {

        System.out.println(
            "Sending email to " + student.getName()
        );
    }
}
```

```
public class StudentService {

    private StudentValidator validator;
    private StudentRepository repository;
    private FeeCalculator calculator;
    private ReportGenerator reportGenerator;
    private EmailService emailService;

    public StudentService() {

        validator = new StudentValidator();
        repository = new StudentRepository();
        calculator = new FeeCalculator();
        reportGenerator = new ReportGenerator();
        emailService = new EmailService();
    }

    public void registerStudent(Student student) {

        if (!validator.isValid(student)) {
            System.out.println("Invalid student information.");
            return;
        }
```

If a System Suffering from a Large Class and Duplicated Code: Java Example

After Refactoring Long Method:

```
public class StudentService {

    private StudentValidator validator;
    private StudentRepository repository;
    private FeeCalculator calculator;
    private ReportGenerator reportGenerator;
    private EmailService emailService;

    public StudentService() {

        validator = new StudentValidator();
        repository = new StudentRepository();
        calculator = new FeeCalculator();
        reportGenerator = new ReportGenerator();
        emailService = new EmailService();
    }
    public void registerStudent(Student student) {

        if (!validator.isValid(student)) {
            System.out.println("Invalid student
information.");
            return;
        }
        repository.save(student);
        double fee =
            calculator.calculateFee(student.getTuitionFee());

        reportGenerator.generate(student, fee);
        emailService.sendEmail(student);
    }
    public void updateStudent(Student student) {
        if (!validator.isValid(student)) {
            System.out.println("Invalid student
information.");
            return;
        }
        repository.update(student);
        double fee =
            calculator.calculateFee(student.getTuitionFee());
        reportGenerator.generate(student, fee);
        emailService.sendEmail(student);
    }
}
```

Problem	Refactoring Solution
Large Class	Extract Class / Responsibility redistribution
Duplicated Code	Extract Method / Move common logic to a single class
Too many responsibilities	Apply **Single Responsibility Principle (SRP)**
Repeated validation	Centralize in StudentValidator
Repeated fee calculation	Centralize in FeeCalculator
Database code mixed with	Move to StudentRepository

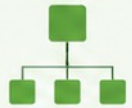
Design Patterns vs. Anti-patterns (Code Smells)

Key Lesson: Do not apply patterns simply because they exist.
Choose a pattern because it solves a real design problem.
Likewise, do not automatically remove every code smell; **understand the context and trade-offs first.**

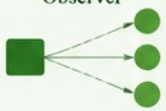
DESIGN PATTERN

Example Patterns

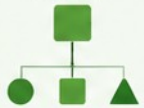
Singleton



Observer



Factory



Reusable solution
Provides a general, reusable solution to a recurring software design problem.

Improves communication
Offers a common vocabulary that helps developers communicate design ideas clearly.

Proven approach
Based on experience and proven in many successful projects.

Used intentionally
Applied when the context is appropriate and the problem fits the pattern.

ANTI-PATTERN / CODE SMELL

Recurring poor design
Represents a tendency toward poor design or implementation.

Warning sign
Signals that the code may be difficult to reuse, modify, test, or maintain.

Investigate the cause
Indicates deeper problems that should be understood and addressed.

Refactor when appropriate
Should be refactored when it hurts maintainability or increases complexity.

Common Code Smells

Long Method



Duplicate Code



Large Class



Large Coupling



Tight Coupling

When to Use a Design Pattern?

- When you recognize a recurring design problem.
- When it improves flexibility, maintainability, or extensibility.
- When the added complexity of the pattern is justified.

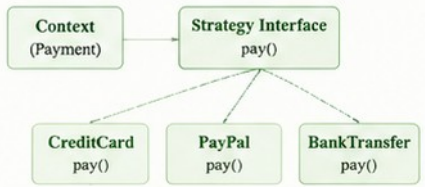


Good design is about making the right choice.

When to Investigate a Code Smell?

- When it appears frequently in the codebase.
- When it makes changes risky or hard to test.
- When it violates good design principles.
- When it affects readability or performance.

Example: Good Design Using a Pattern (Strategy Pattern)



- Open for extension (add new payment easily)
- Closed for modification (existing code unchanged)
- Easy to test and maintain
- Low coupling, high flexibility

Refactor to Improve

Example: Bad Design (Code Smell – Long Method & Tight Coupling)

```
public void proccesPayment(String type) {
   if (type.equals("card")) {
      // do card payment
   } else if (type.equals("paypal")) {
      // do paypal payment
   } else if (type.equals("bank")) {
      // do bank transfer
   }
   // Many more conditions...
}
```

- Hard to extend (changes require modification)
- Violates Single Responsibility
- Difficult to test
- High coupling and complexity
- Prone to bugs

Takeaway: Good software design comes from recognizing **the right pattern for the right problem** and addressing the real issues behind code smells—not by following rules blindly.

Design Patterns vs. Anti-patterns

Design Pattern

Reusable solution to a recurring software design problem.

Improves communication and provides a proven design approach.

Used intentionally when the context is appropriate.

Anti-pattern / Code Smell

Recurring poor design or implementation tendency.

Warns that code may be difficult to reuse, modify, test, or maintain.

Should be investigated and potentially refactored.

Key lesson:

Do not apply patterns simply because they exist.

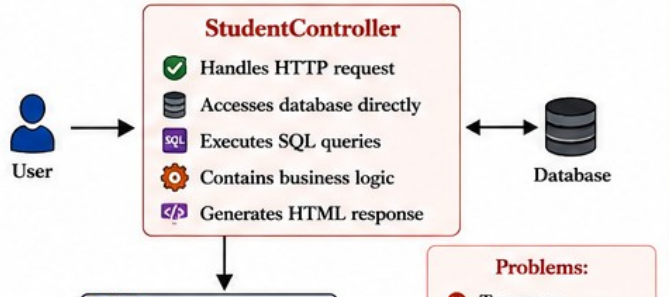
Choose a pattern because it solves a real design problem.

Likewise, do not automatically remove every code smell; understand the context and trade-offs first.

From Bad Design to Better Design — Refactoring

Bad Design (Before Refactoring)

All responsibilities are mixed in one class (StudentController).



Problems:

- Too many responsibilities
- Hard to maintain
- Hard to test
- Changes in database or UI affect controller
- Tightly coupled

Hard to understand, maintain, and extend!

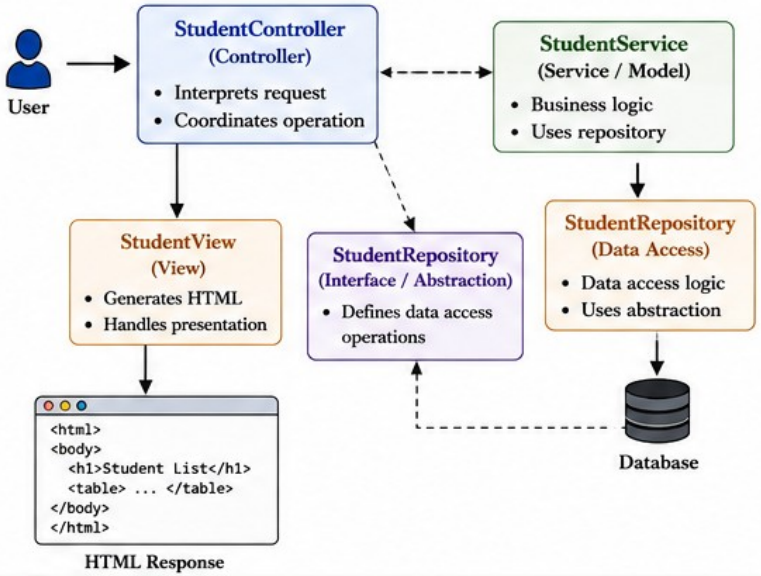
Refactoring Approach

1. Move student data and operations into appropriate model/service classes.
2. Move presentation/HTML generation into the View.
3. Keep the Controller focused on interpreting the request and coordinating the operation.
4. Replace direct low-level dependencies with suitable abstractions where appropriate.
5. Test after each refactoring step to ensure behavior remains unchanged.

Goal: Improve structure without changing what the user expects the system to do.

Better Design (After Refactoring)

Responsibilities are separated into appropriate classes.



Clean, maintainable, testable, and flexible!

From Bad Design to Better Design — Refactoring

Refactoring

Refactoring is the process of changing code so that its external behavior remains the same while its internal structure is improved.

Refactoring approach for the previous example:

1. Move student data and operations into appropriate model/service classes.
2. Move presentation/HTML generation into the View.
3. Keep the Controller focused on interpreting the request and coordinating the operation.
4. Replace direct low-level dependencies with suitable abstractions where appropriate.
5. Test after each refactoring step to ensure behavior remains unchanged.

Goal: Improve structure without changing what the user expects the system to do.

Goal

- Create software that remains reusable, flexible, understandable, and maintainable as requirements change.
- Key idea: Good design is not only about selecting a pattern; it is also about recognizing when a design has become unnecessarily coupled, complex, duplicated, or difficult to maintain.

Module 3 — Summary

- **MVC:** Separates a user-interface system into **Model, View, and Controller** responsibilities.
- **General Design Principles: Information hiding, separation of concerns, dependency inversion, open/closed, interface segregation, coupling objects, DRY, and least knowledge.**
- **Code Analysis:** Look for excessive responsibilities, **coupling, duplication, long methods, complex** dependencies, and inappropriate abstractions.
- **Anti-patterns / Code Smells**
  - Inappropriate intimacy, message chains, primitive obsession, switch statements, speculative generality, refused request, and many other smells.
- **Refactoring:** Improve internal structure while preserving external behavior.
- **Final Goal:** Create software that remains reusable, flexible, understandable, and maintainable as requirements change.

Software Design Concepts Summary with Dhaka Urban Management System (DUMS) Examples

Topic	Short Explanation	Application in DUMS (Dhaka Urban Management System)	Example in DUMS
MVC (Model–View–Controller)	Separates a user-interface system into Model, View, and Controller responsibilities.	Divides DUMS into Model (urban data & logic), View (citizen/ddmin interfaces), Controller (request processing).	Citizen submits a waste complaint → View: Complaint Form → Controller: Processes request → Model: Stores complaint in database → View shows status update.
Information Hiding	Hides internal implementation details and exposes only necessary interfaces.	Protects internal operations of urban service modules.	Traffic module provides *getTrafficStatus()* API without exposing how traffic prediction is calculated.
Separation of Concerns (SoC)	Divides a system into independent parts, each handling a specific responsibility.	Separates traffic, waste, emergency, payment, and citizen service modules.	Waste collection scheduling is handled by Waste Management Module, not mixed with traffic operations.
Dependency Inversion Principle (DIP)	High-level modules should depend on abstractions, not specific implementations.	Urban services communicate through common interfaces.	EmergencyService depends on NotificationService interface rather than directly depending on SMS or Email systems.
Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	New urban services can be added without changing existing modules.	Adding a Smart Parking Module without modifying Traffic Management code.
Interface Segregation Principle (ISP)	Clients should not depend on unnecessary methods.	Provides specific interfaces for different users and services.	Traffic Officer uses *TrafficControlInterface*, while Waste Manager uses WasteCollection*Interface*.
Coupling Management	Reduces dependency between modules to improve flexibility.	DUMS modules communicate through APIs/services instead of direct dependency.	Traffic Management receives road information from a shared City Data Service instead of directly depending on Waste Management.
DRY (Don't Repeat Yourself)	Avoids duplicate code and logic.	Common functions are reused across modules.	A single **NotificationService** is used for complaints, emergency alerts, and service updates.
Least Knowledge Principle	A module should know only necessary information about other modules.	Limits unnecessary communication between urban modules.	Complaint Module only requests service status; it does not access internal Waste Management operations.

Code Analysis: What to Look For

- Excessive responsibilities
- High coupling between modules
- Code duplication
- Long methods and large classes
- Complex dependencies
- Inappropriate abstractions

Anti-patterns / Code Smells

Inappropriate Intimacy – one class depends too much on another class's internal details.

Message Chains – long chains of method calls across multiple objects.

Primitive Obsession – too many primitive types instead of domain objects.

Large Switch Statements – complex conditionals instead of polymorphism.

Speculative Generality – adding unnecessary abstractions for future (possible) needs.

Refused Bequest – inheriting methods that are not used or not applicable.

... and many other smells

Refactoring

Improve *internal* code structure while preserving *external* behavior.

Example: Splitting a large *CityManagement* class into *TrafficService*, *WasteService*, and *EmergencyService* classes.

Final Goal:
Create software that remains **reusable**, **flexible**, **understandable**, and **maintainable** as requirements change.

A Smarter Dhaka, Built on Good Design



- New services can be added easily —
- Modules work independently
- One part can change without breaking others
- Citizens get a unified smart-city experience

Concept	Short Explanation	Application in Dhaka Urban Management System (DUMS)	Example
MVC (Model–View–Controller)	Separates user interface, data, and request handling responsibilities.	Divides DUMS into **Model (urban data & logic), View (citizen/admin interfaces), Controller (request processing)**.	Citizen submits a waste complaint → **View:** Complaint Form → **Controller:** Processes request → **Model:** Stores complaint in database → View shows status update.
Information Hiding	Hides internal implementation details and exposes only necessary interfaces.	Protects internal operations of urban service modules.	Traffic module provides getTrafficStatus() API without exposing how traffic prediction is calculated.
Separation of Concerns (SoC)	Divides a system into independent parts, each handling a specific responsibility.	Separates traffic, waste, emergency, payment, and citizen service modules.	Waste collection scheduling is handled by **Waste Management Module**, not mixed with traffic operations.
Dependency Inversion Principle (DIP)	**High-level modules should depend on abstractions,** not specific implementations.	Urban services communicate through common interfaces.	EmergencyService depends on NotificationService interface rather than directly depending on SMS or Email systems.
Open/Closed Principle (OCP)	Software entities should be open for extension but closed for modification.	New urban services can be added without changing existing modules.	Adding a **Smart Parking Module** without modifying Traffic Management code.
Interface Segregation Principle (ISP)	Clients should not depend on unnecessary methods.	Provides specific interfaces for different users and services.	Traffic Officer uses TrafficControlInterface, while Waste Manager uses WasteCollectionInterface.
Coupling Management	Reduces dependency between modules to improve flexibility.	DUMS modules communicate through APIs/services instead of direct dependency.	Traffic Management receives road information from a shared City Data Service instead of directly depending on Waste Management.
DRY (Don't Repeat Yourself)	Avoids duplicate code and logic.	Common functions are reused across modules.	A single NotificationService is used for complaints, emergency alerts, and service updates.
Least Knowledge Principle	A module should know only necessary information about other modules.	Limits unnecessary communication between urban modules.	Complaint Module only requests service status; it does not access internal Waste Management operations.
Code Analysis	Evaluates code quality by identifying design problems.	Reviews DUMS modules for maintainability and complexity.	Detecting that Citizen Service class contains login, payment, complaint, and notification responsibilities.
Anti-patterns / Code Smells	Poor design practices that make software difficult to maintain.	Identifies problematic structures before they affect system quality.	A single **CityManagement class** controlling traffic, waste, emergency, and payment creates excessive responsibility.
Inappropriate Intimacy	One class depends too much on another class's internal details.	Prevents modules from accessing unnecessary internal data.	Traffic module directly modifying Waste Vehicle database records.
Message Chains	Long sequences of object calls indicate poor design.	Avoids complicated communication paths.	Citizen → Complaint → Department → Manager → Employee → Database (too many dependencies).
Primitive Obsession	Overuse of basic data types instead of meaningful objects.	Encourages proper domain modeling.	Instead of storing location as "23.8103,90.4125" string, create a **Location** class.
Large Switch Statements	Excessive conditional logic often indicates missing abstraction.	Replaces complex decision structures with polymorphism.	Instead of checking if(service=="Traffic"), if(service=="Waste"), create separate service classes.
Speculative Generality	Adding unnecessary features for possible future use.	Keeps DUMS simple and focused.	Avoid creating unused AI modules before actual requirements exist.
Refactoring	Improving internal code structure without changing external behavior.	Improves maintainability of DUMS while keeping services working.	Splitting a large CityManagement class into TrafficService, WasteService, and EmergencyService classes.
Final Goal	Build software that is reusable, flexible, understandable, and maintainable.	Ensures DUMS can evolve with future smart-city requirements.	Adding new IoT sensors or AI analytics without redesigning the entire system.

Sample Question

1. Explain the responsibilities of the Model, View, and Controller in the MVC Pattern.
2. What are the three components of the MVC Pattern?
3. Apply MVC to a university student registration system. Identify suitable responsibilities for the Model, View, and Controller
4. Design the architecture of an MVC-based Student Management System. Identify the major classes and explain how the Model, View, and Controller communicate.
5. Explain the DRY principle and the Principle of Least Knowledge.

Bloom's Level	Question
Remember	1. What are the three components of the MVC Pattern?
Understand	2. Explain the responsibilities of the Model, View, and Controller in the MVC Pattern.
Understand	3. Explain how the MVC Pattern supports separation of concerns.
Apply	4. Apply MVC to a university student registration system. Identify suitable responsibilities for the Model, View, and Controller.
Analyze	5. Analyze a program in which the Controller performs input validation, database operations, business logic, and HTML generation. Identify the design problems.
Evaluate	6. Evaluate the benefits of separating database operations, business logic, request handling, and presentation in an MVC-based application.
Create	7. Design the architecture of an MVC-based Student Management System. Identify the major classes and explain how the Model, View, and Controller communicate.

Bloom's Level	Question
Remember	1. Define an anti-pattern and a code smell. Give three examples of code smells.
Understand	2. Explain the meaning of the Large Class, Long Method, and Duplicated Code smells.
Understand	3. Explain the concepts of Primitive Obsession and Message Chains.
Apply	4. A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach.
Analyze	5. Analyze a system where the same switch statement for customer types appears in five different methods. Identify the code smell and explain why it creates maintenance problems.
Evaluate	6. Evaluate whether every occurrence of a code smell must always be refactored. Explain why the context of the software design should be considered.
Create	7. Create a refactored design for a system suffering from a Large Class and Duplicated Code. Identify how responsibilities should be redistributed among multiple classes.

Bloom's Level	Question
Remember	1. What are the DRY principle, Principle of Least Knowledge, Open/Closed Principle, and Information Hiding?
Understand	2. Explain how the DRY principle improves software maintainability.
Understand	3. Explain why excessive coupling between classes can make software difficult to maintain and modify.
Apply	4. Consider a class that directly accesses several internal objects of another class through a long chain of method calls. Identify the relevant design principle and suggest how it could be improved.
Analyze	5. Analyze the following situation: a superclass provides five methods, but a subclass only needs one of them and ignores the other four. Identify the possible code smell and explain the design problem.
Evaluate	6. Evaluate the following statement: **“Using more design patterns always results in better software design.”** Support your answer with appropriate examples from Modules 1–3.
Create	7. Create a design-improvement plan for a poorly designed Java application containing a Large Class, Duplicated Code, Primitive Obsession, long methods, and tightly coupled classes. Explain which design principles, patterns, and refactoring techniques you would use.

Sample Question

1. A method contains 100 lines of code and performs validation, database access, calculations, and report generation. Identify the likely code smell and suggest an appropriate refactoring approach.
2. Analyze a system where the same switch statement for customer types appears in five different methods. Identify the code smell and explain why it creates maintenance problems.
3. Analyze a method containing 150 lines that performs validation, database operations, calculations, and report generation. Identify the possible smells and explain how they affect maintainability.
4. Evaluate whether comments are always beneficial in software development. Explain how excessive comments may sometimes indicate a design problem.
5. Create a refactoring plan for a Java application suffering from a Large Class, Long Methods, and Duplicated Code. Explain the sequence of improvements you would make.

Bloom's Level	Question
Remember	1. What are the three components of MVC? State the primary responsibility of each component.
Understand	2. Explain how separation of concerns is achieved through the MVC Pattern.
Understand	3. Explain the DRY principle and the Principle of Least Knowledge.
Apply	4. A Student Management System contains database queries directly inside its GUI classes. Apply the principles of MVC and separation of concerns to improve the design.
Analyze	5. Analyze a class that performs database access, user-interface formatting, validation, and business calculations. Identify the possible design problems and code smells.
Evaluate	6. Evaluate the statement: **“A large class is always an example of bad design.”** Discuss the importance of context when identifying code smells.
Create	7. Design an MVC architecture for an online course-registration system. Identify the Model, View, Controller, and major responsibilities of each.

Bloom's Level	Question
Remember	1. Define Large Class, Long Method, Duplicated Code, and Primitive Obsession.
Understand	2. Explain why duplicated code can make software difficult to maintain.
Understand	3. Explain the difference between Divergent Change and Shotgun Surgery.
Apply	4. A program contains the same student-validation code in five different classes. Identify the code smell and propose an appropriate refactoring approach.
Analyze	5. Analyze a method containing 150 lines that performs validation, database operations, calculations, and report generation. Identify the possible smells and explain how they affect maintainability.
Evaluate	6. Evaluate whether comments are always beneficial in software development. Explain how excessive comments may sometimes indicate a design problem.
Create	7. Create a refactoring plan for a Java application suffering from a Large Class, Long Methods, and Duplicated Code. Explain the sequence of improvements you would make.

Bloom's Level	Question
Remember	1. Define Inappropriate Intimacy, Message Chains, Primitive Obsession, Speculative Generality, and Refused Request.
Understand	2. Explain why the Principle of Least Knowledge is useful for reducing unnecessary dependencies.
Understand	3. Explain why speculative generality can lead to unnecessary complexity in a software system.
Apply	4. A class accesses several objects through a long chain such as student.getDepartment().getTeacher().getOffice().getPhone(). Identify the possible design issue and suggest an improvement.
Analyze	5. Analyze a superclass that contains several methods that are not required by one of its subclasses. Identify the likely code smell and explain why it may indicate an inappropriate inheritance relationship.
Evaluate	6. Evaluate the statement: **“Design patterns should always be used to improve software design.”** Discuss situations in which applying a design pattern could actually make a simple system unnecessarily complicated.
Create	7. Create a complete design-improvement strategy for a poorly designed Java application containing tight coupling, duplicated code, primitive obsession, long methods, and inappropriate intimacy. Identify the design principles, patterns, and refactoring techniques you would apply.